

Code PFE : _____

Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique
Université de Manouba
Institut Supérieur des Arts Multimédia de la Manouba



Mémoire de fin d'études

Préparé en vue de l'obtention du diplôme de licence en sciences de l'informatique
Parcours Micro-Informatique et Machines Embarquées (MIME)

PrediTeq : Plateforme SaaS de maintenance prédictive industrielle basée sur le Machine Learning et l'IoT

Réalisé par

Mohamed Firas Abed

Encadré par

M. Kais Brik, ISAMM

M. Saber Abeda, AROTEQ

Année universitaire : 2025 – 2026

Dédicace

Je dédie ce travail à ma famille, pour son soutien constant, sa patience et sa confiance tout au long de ce projet.

Je le dédie également à toutes les personnes qui m'ont encouragé à viser un travail à la fois utile, rigoureux et compréhensible, au-delà de la seule performance technique.

Remerciements

Je tiens à remercier vivement mon encadrant académique, **M. Kais Brik**. Je le remercie pour son accompagnement, ses remarques structurantes et sa disponibilité tout au long de ce projet.

J'adresse également mes remerciements à **M. Saber Abeda** ainsi qu'à l'équipe d'**AROTeq** pour leur confiance, leur retour terrain et l'opportunité de construire un sujet relié à un besoin industriel concret.

Je remercie aussi **Dr. Lilia Sfaxi** pour ses conseils méthodologiques, en particulier sur la progression à adopter lorsque les données réelles sont encore rares : partir d'une génération réaliste, valider le preprocessing et l'analyse du signal, structurer l'extraction de caractéristiques, puis avancer par étapes vers la détection d'anomalies et la modélisation du RUL.

Je remercie enfin toutes les personnes qui ont contribué, de près ou de loin, à la maturation de PrediTeq : par des échanges techniques, des retours de démonstration, des conseils de présentation ou des remarques de fond sur le réalisme du produit et de ses scénarios.

Résumé

Ce projet de fin d'études présente PrediTeq, une plateforme de maintenance prédictive industrielle fondée sur l'apprentissage automatique, ou *Machine Learning*, et sur l'Internet des objets industriel, ou *Internet of Things* (IoT). La plateforme est générale par conception et conçue pour être réutilisable sur différentes machines. Le moteur d'un stockeur vertical rotatif constitue le premier cas de référence. Il ancre la simulation physique, le produit actuel et la démonstration, sans fixer la limite finale de la plateforme. La solution relie trois blocs cohérents : une chaîne d'apprentissage hors ligne, un service d'inférence temps réel et une interface web de supervision.

PrediTeq est conçu comme une plateforme logicielle en mode service, ou *Software as a Service* (SaaS). Cette orientation signifie que le pipeline scientifique doit alimenter un produit partageable, gouvernable et compréhensible par l'utilisateur final. L'architecture va dans ce sens : le service temps réel s'appuie sur un contrat de données générique, et la chaîne d'apprentissage peut être réentraînée pour une autre famille de machines.

Le noyau scientifique du projet repose sur une simulation physiquement guidée du moteur de référence. Cette simulation génère 200 trajectoires cohérentes avec les constantes nominales de la machine, les cas de charge et les cycles d'usage retenus. Le pipeline extrait ensuite des caractéristiques, détecte les dérives, construit un indice de santé et estime une durée de vie résiduelle. Le point méthodologique essentiel est que la cible du pronostic est dérivée d'un signal observable de santé et non d'une vérité cachée inaccessible en exploitation.

Les résultats exportés sont solides. Le modèle de durée de vie résiduelle atteint un coefficient de détermination de 0,980 sur le jeu de test, avec une erreur absolue moyenne de 1,65 jour. La validation croisée par groupes donne un coefficient moyen de 0,982. L'ensemble hybride de détection d'anomalies atteint une mesure $F1$ de 0,964. La validation externe sur NASA CMAPSS, ou *Commercial Modular Aero-Propulsion System Simulation*, conserve un coefficient de détermination de 0,886. Ces résultats n'autorisent pas à revendiquer une validation sur toute machine industrielle. Ils montrent en revanche que la méthode est déjà partiellement éprouvée au-delà du seul cas interne du stockeur.

Sur le plan applicatif, le service backend charge les artefacts au démarrage, crée un moteur d'inférence par machine, reçoit les mesures, calcule l'indice de santé, historise les valeurs et pilote les alertes. L'interface web expose ensuite un tableau de bord, un diag-

nostic avancé, un calendrier de maintenance, un centre d’alertes, un module de rapports, un simulateur à trois scénarios et une page d’acquisition expérimentale. Une première chaîne live cohérente avec l’architecture de la plateforme a déjà été préparée et vérifiée fonctionnellement : une source locale ou un poste externe publie des mesures, le transport Message Queuing Telemetry Transport (MQTT) les achemine, puis le backend et l’interface restituent l’état machine. Cette étape prouve la faisabilité technique du flux de bout en bout, mais elle ne vaut pas encore validation terrain sur une machine industrielle réelle. Le prochain jalon consiste à raccorder le contrat CSV réel fourni par le partenaire industriel, puis à stabiliser cette chaîne sur une première machine de référence.

Le projet atteint ainsi un niveau de maturité plus précis. Le pipeline est validé scientifiquement. Le produit est démontré fonctionnellement. La chaîne live est préparée et vérifiée sur un essai fonctionnel, sans être encore industrialisée sur le terrain. Le simulateur reste un outil de démonstration guidée. La validation sur données réelles reste l’étape suivante avant un déploiement progressif de flotte chez AROTEQ puis une ouverture à d’autres contextes industriels.

Mots-clés : maintenance prédictive, plateforme SaaS, indice de santé, durée de vie résiduelle, apprentissage automatique, Internet des objets industriel

Abstract

This end-of-study project presents PrediTeq, an industrial predictive-maintenance platform built on Machine Learning and the Industrial Internet of Things (IoT). The platform is general by design and intended to be reusable across different machines. The drive motor of a vertical rotary storage carousel is used as the first reference case. It grounds the physical simulation, the current product, and the demonstration, without defining the final limit of the platform. The solution combines three coherent layers : an offline learning pipeline, a runtime inference service, and a web supervision interface.

PrediTeq is designed as a software-as-a-service platform. This means that the scientific pipeline must feed a shareable, governable, and readable product. The architecture already reflects that goal through a generic runtime data contract and a retrainable pipeline for other machine families.

The scientific core relies on a physically guided simulation of the reference motor. This simulation generates 200 trajectories that remain consistent with the machine's nominal constants, load cases, and usage cycles. The pipeline then extracts features, detects drifts, computes a health index, and estimates remaining useful life. The key methodological point is that the prognostic target is derived from an observable health signal rather than from a hidden label unavailable in real operation.

Exported results are strong. The remaining-useful-life model reaches an R^2 of 0.980 on the holdout set, with a mean absolute error of 1.65 day. Group-based cross-validation yields a mean R^2 of 0.982. The hybrid anomaly detector reaches an $F1$ score of 0.964. External validation on NASA CMAPSS FD001 still gives an R^2 of 0.886. These results do not prove the platform on every machine type, but they do show that the method already transfers beyond the internal stocker case.

On the application side, the backend loads the artifacts at startup, creates one inference engine per machine, receives telemetry, computes the health index, stores histories, and triggers alerts. The web interface then exposes a dashboard, advanced diagnostics, maintenance planning, alerts, reports, a three-scenario simulator, and an experimental acquisition page. A first complete live chain has already been prepared and functionally verified : a local source, or an external workstation, publishes measurements, MQTT carries them, and the backend together with the web interface restitutes the machine state. This proves the end-to-end technical feasibility of the flow, but it does not yet count as

field validation on real industrial telemetry. The next milestone is to connect the real CSV contract provided by the industrial partner and stabilize that chain on a first reference machine.

PrediTeq therefore reaches a more precise maturity level. The pipeline is scientifically validated. The product is functionally demonstrated. The live chain is prepared and functionally verified, but not yet industrialized on field telemetry. The simulator remains a guided demonstration layer. Real-field validation on actual telemetry remains the next step before rollout to other AROTEQ machines and, later, broader industrial use.

Table des figures

1.1	AROTeq : identité visuelle et environnement d'atelier.	5
1.2	Organigramme d'AROTeq.	6
1.3	Positionnement du stage dans le flux projet.	6
1.4	Technologies industrielles.	7
1.5	Système de test industriel.	8
1.6	Familles de maintenance	9
1.7	Vue d'ensemble de PrediTeq	12
1.8	Cycle de travail retenu	13
1.9	Planning simplifié du travail	13
2.1	Lecture produit du pronostic	17
2.2	Architecture MQTT	19
2.3	Capteurs visés et capteurs de banc	20
2.4	Technologies de l'interface web : React, TypeScript et Vite	20
2.5	Logo de FastAPI	21
2.6	Logo de Supabase	21
2.7	Pile retenue : React, FastAPI, Supabase et MQTT	21
3.1	Cas d'utilisation principaux	25
3.2	Cycle de surveillance temps réel	32
4.1	Architecture générale de PrediTeq	34
4.2	Diagramme de classes simplifié du noyau runtime	35
4.3	Séquence simplifiée de la supervision temps réel	37
5.1	Base physique de la simulation	41
5.2	Profils et charges	42
5.3	Chronologie des anomalies	45
5.4	Courbes d'indice de santé	46
5.5	RUL réelle et RUL prédite	49
5.6	Résumé d'explicabilité	50
5.7	Sensibilité des réglages	51

5.8	Validation NASA CMAPSS	51
5.9	Calibration des intervalles	52
5.10	Tableau de bord PrediTeq	55
5.11	Structure du simulateur	56
5.12	Banc d'essai expérimental	57
5.13	Scénario Experiment	58
5.14	Validation fonctionnelle de la chaîne live à deux postes	61
5.15	Technologies de déploiement, de messagerie et de versionnement retenues. .	63
5.16	Architecture de déploiement	63
5.17	Chaîne préparée d'intégration réelle	64
7.1	Capture du code source reel montrant les constantes moteur, la charge maximale, le cycle 44 s et les 200 trajectoires	98
7.2	Capture du code source reel montrant les phases ascent/descent/pause ainsi que le calcul de la puissance et du courant	100
7.3	Capture du code source reel montrant la construction du tableau de features et la liste des variables fournies au modele	101
7.4	Capture du code source reel montrant le poids HYBRID_ALPHA et la combi- naison du score Isolation Forest avec la RMS vibratoire	102
7.5	Capture du code source reel montrant que la cible RUL est construite a partir de <code>hi_smooth</code> et de resumes de tendance observables	104
7.6	Capture du code source reel montrant la validation <code>GroupKFold</code> qui garde des trajectoires entieres hors apprentissage	105

Liste des tableaux

1.1	Fiche synthétique d'AROTeq	5
1.2	Axes de contexte utiles au projet	8
1.3	Lecture comparative de l'existant	11
3.1	Acteurs du système	24
3.2	Besoins fonctionnels retenus	26
3.3	Besoins non fonctionnels	27
3.4	Contraintes majeures du projet	28
3.5	User stories prioritaires	30
4.1	Structures principales de persistance	36
4.2	Organisation fonctionnelle de l'interface	38
5.1	Environnement de réalisation	40
5.2	Constantes de référence	41
5.3	Paramètres de simulation	42
5.4	Étapes de la chaîne d'apprentissage	43
5.5	Variables principales extraites par le pipeline	43
5.6	Paramétrage du pronostic	47
5.7	Résultats principaux du pronostic	48
5.8	Parcours utilisateur principal	53
5.9	Pages principales de la plateforme	54
5.10	Scénarios machine de démonstration	56
5.11	Maturité actuelle des briques PrediTeq	59
5.12	Synthèse des apports personnels	67
6.1	Cartographie fonctionnelle du dépôt	71
6.2	Détails utiles des scénarios machine	73
6.3	Trace simple entre besoins, pages produit et preuves de validation	76
6.4	Principe de gestion des visuels technologiques	77
7.1	Chaine complete PrediTeq, du moteur au produit	79

7.2	Demarche industrielle recommandee et reponse effective de PrediTeq	82
7.3	Choix techniques majeurs, justification et effet pratique	84
7.4	Technologies utilisees et role reel dans le projet	87
7.5	Pages visibles avant authentication	89
7.6	Pages metier accessibles apres connexion	90
7.7	Pages et actions reservees aux administrateurs	91
7.8	Principales surfaces backend exposees par l'API	92
7.9	Difference entre la page Experiment et la vraie chaine live PrediTeq	93
7.10	Comment presenter le meme sous-systeme a un public non technique ou technique	106

Liste des sigles et abréviations

ACS712 Capteur à effet Hall de mesure de courant

API Interface de programmation applicative

CMAPSS Commercial Modular Aero-Propulsion System Simulation, benchmark NASA de pronostic

ECE Erreur de calibration attendue

ESP32 Microcontrôleur 32 bits de la famille Espressif

GMAO Gestion de maintenance assistée par ordinateur

HI Indice de santé

IEC Commission électrotechnique internationale

IF Isolation Forest

IoT Internet des objets

ISAMM Institut Supérieur des Arts Multimédia de la Manouba

ISO Organisation internationale de normalisation

JSON JavaScript Object Notation, format d'échange de données

LabVIEW Environnement logiciel d'instrumentation virtuelle

MAE Erreur absolue moyenne

MEMS Systèmes micro-électromécaniques

ML Apprentissage automatique

MQTT Message Queuing Telemetry Transport, protocole de télémétrie légère

MPU6050 Module MEMS inertiel accéléromètre + gyroscope

NASA Agence spatiale américaine

PFE Projet de fin d'études

PHM Pronostic et gestion de santé

PLC Automate programmable industriel

RF Forêt aléatoire

RMSE Racine de l'erreur quadratique moyenne

RMS Valeur efficace

Liste des sigles et abréviations

RUL Durée de vie résiduelle

SaaS Logiciel en tant que service

SHAP Explications additives de Shapley

USB Universal Serial Bus

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	v
Liste des figures	vii
Liste des tableaux	ix
Liste des sigles et abréviations	xi
Introduction générale	1
Apports personnels et périmètre réalisé	2
Organisation du mémoire	2
1 Cadre général du projet	4
1.1 Introduction	4
1.2 Présentation de l'organisme d'accueil	4
1.3 Présentation du projet et problématique	9
1.4 Étude de l'existant	10
1.5 Critique de l'existant	11
1.6 Solution proposée	12
1.7 Méthodologie de travail	13
1.8 Conclusion	14
2 État de l'art	15
2.1 Introduction	15
2.2 Maintenance industrielle et prédiction	15
2.3 Concepts clés de pronostic	16
2.4 Méthodes de Machine Learning retenues	17

2.5	IoT, instrumentation et communication	18
2.6	Environnement logiciel	20
2.7	Conclusion	22
3	Analyse des besoins	23
3.1	Introduction	23
3.2	Identification des acteurs	24
3.3	Besoins fonctionnels	26
3.4	Besoins non fonctionnels	27
3.5	Contraintes du projet	28
3.6	User stories prioritaires	29
3.7	Cycle de surveillance visé	31
3.8	Conclusion	33
4	Conception	34
4.1	Introduction	34
4.2	Architecture générale de la solution	34
4.3	Noyau runtime	35
4.4	Conception de la persistance	36
4.5	Séquence de supervision temps réel	37
4.6	Conception de l'interface et des droits	38
4.7	Conclusion	38
5	Réalisation	39
5.1	Introduction	39
5.2	Environnement matériel et logiciel	39
5.3	Réalisation du noyau scientifique	41
5.4	Réalisation de la plateforme de supervision	52
5.5	Validation fonctionnelle, discussion critique et ouverture industrielle	57
5.6	Conclusion	64
	Conclusion générale et perspectives	65
	Apports personnels majeurs	67
	Bibliographie et webographie	69
6	Annexes pratiques	71
6.1	Annexe A : cartographie résumée du dépôt	71
6.2	Annexe B : commandes utiles	72
6.3	Annexe C : trois machines du simulateur	73
6.4	Annexe D : protocole de démonstration	73

6.5	Annexe E : chaîne v1 d'intégration réelle	74
6.6	Annexe F : traçabilité user stories, pages et preuves de test	76
6.7	Annexe G : origine des logos technologiques	77
7	Dossier ultime de soutenance	78
7.1	Vue d'ensemble ordonnée du projet	79
7.2	Demarche methodologique suivie	81
7.3	Choix structurants et justification des methodes	84
7.4	Cartographie technologique exacte	87
7.5	Ce que les utilisateurs peuvent faire dans l'application	88
7.6	Surfaces backend et logique partagee	92
7.7	IoT, Experiment, MQTT et PC relais cote client	93
7.8	Preuves par extraits de code	97
7.9	Meme projet, deux niveaux de discours	105
7.10	Points d'honnetete, limites et ordre conseille pour la presentation	107

Introduction générale

La maintenance prédictive cherche à répondre à une question simple : intervenir avant la panne sans remplacer trop tôt un équipement encore exploitable. Dans un atelier réel, cette question devient vite exigeante. Les mesures sont bruitées. Les historiques de défaillance sont rares. Les résultats n'ont de valeur que s'ils restent compréhensibles pour l'exploitant.

C'est dans ce contexte qu'a été développé PrediTeq. Le projet est pensé comme une plateforme générale par conception de maintenance prédictive industrielle, fondée sur l'apprentissage automatique, ou *Machine Learning*, et sur l'Internet des objets industriel, ou *Internet of Things* (IoT). Le moteur d'un stockeur vertical rotatif en constitue le premier cas de référence. Ce choix ancre la simulation physique, le produit actuel et la démonstration. Il ne fixe pas pour autant la limite finale de la plateforme. L'architecture relie trois blocs complémentaires : un apprentissage hors ligne, un service de supervision temps réel et une interface web de pilotage.

Le projet est aussi pensé comme une plateforme logicielle en mode service, ou *Software as a Service* (SaaS). Le pipeline ne vaut donc pas seulement par ses métriques. Il doit aussi alimenter un produit capable de surveiller une machine, d'afficher un diagnostic, de publier un horizon de maintenance et de garder une trace exploitable des décisions. Cette ambition générale est visible dans la conception même de PrediTeq : le service temps réel s'appuie sur un contrat de données générique, et la chaîne d'apprentissage peut être réadaptée à une autre famille de machines.

La progression retenue reste volontairement concrète. Le pipeline commence par simuler des trajectoires cohérentes avec les constantes du moteur et les cas de charge de la machine. Il extrait ensuite des caractéristiques, détecte les dérives, calcule un indice de santé et estime une durée de vie résiduelle. Le service applicatif reprend enfin cette logique sur flux live pour produire un état machine, des alertes et une lecture de maintenance.

Les niveaux de maturité du projet sont distincts. L'apprentissage hors ligne ne se confond pas avec l'usage temps réel. Le simulateur applicatif ne remplace pas un essai terrain durable. La page d'acquisition expérimentale ne remplace pas non plus une intégration industrielle complète. Cette séparation permet de distinguer ce qui est validé scientifiquement, ce qui est démontré fonctionnellement et ce qui reste à industrialiser.

L'ouverture vers la donnée réelle n'est plus seulement esquissée. Une première chaîne

live cohérente avec l'architecture de la plateforme a déjà été préparée et vérifiée fonctionnellement : une source locale, ou un poste externe, publie des mesures, le transport Message Queuing Telemetry Transport (MQTT) les achemine, puis le backend et l'interface restituent l'état machine. Cette étape montre que le flux de bout en bout fonctionne. Elle prépare un déploiement de flotte, sans remplacer encore la qualification terrain sur une machine industrielle réelle.

Apports personnels et périmètre réalisé

Dans sa phase finale, le projet a été consolidé autour de plusieurs apports structurants :

- une simulation portée à 200 trajectoires, avec une lecture physiquement guidée de la dégradation ;
- une formulation du pronostic fondée sur un signal de santé observable, afin d'éviter toute fuite d'information vers une vérité cachée ;
- une séparation plus nette entre apprentissage hors ligne, exécution temps réel et comportement produit ;
- une démonstration lisible articulée autour de trois scénarios machine et d'une page d'acquisition expérimentale ;
- la préparation et la validation fonctionnelle d'une première chaîne live entre une source locale, un pont de normalisation, le transport MQTT, le backend cloud et le frontend public.

Le prochain jalon n'est donc plus d'inventer cette chaîne, mais de la qualifier sur une première machine de référence à partir du contrat CSV réel fourni par l'industriel.

Organisation du mémoire

Le mémoire suit l'organisation suivante :

- le **chapitre 1** présente le cadre général du projet, l'organisme d'accueil, la problématique, l'existant, la critique de l'existant, la solution proposée et la méthodologie de travail ;
- le **chapitre 2** expose l'état de l'art : maintenance prédictive, indice de santé, durée de vie résiduelle, méthodes retenues et environnement d'Internet des objets ;
- le **chapitre 3** formalise l'analyse des besoins, les acteurs, les fonctions attendues et les contraintes du système ;
- le **chapitre 4** présente la conception de la solution, de l'architecture et des flux de supervision ;

- le **chapitre 5** rassemble la réalisation du pipeline, du service applicatif, de la plateforme web, des validations et de l'ouverture industrielle.

Le document se termine par une conclusion générale, une bibliographie et des annexes pratiques destinées à faciliter la reprise du projet.

Chapitre 1

Cadre général du projet

1.1 Introduction

Ce chapitre présente le cadre industriel et organisationnel du projet PrediTeq. Il situe d'abord l'organisme d'accueil et la place du stage. Il expose ensuite le besoin traité, l'existant et les limites des approches déjà disponibles. Il précise enfin la solution proposée et la méthodologie de travail retenue. Le stage a été conduit dans un positionnement de développement logiciel au contact direct des besoins d'intégration industrielle.

1.2 Présentation de l'organisme d'accueil

1.2.1 Présentation de la société AROTEQ

AROTÉQ est une entreprise tunisienne implantée à Ben Arous. Elle intervient dans l'ingénierie industrielle, l'automatisation, l'intégration électrique et logicielle, ainsi que la réalisation de machines spéciales. Son activité couvre la conception, la fabrication, l'intégration et la mise en service de solutions industrielles sur mesure.

Ce contexte est directement lié au projet PrediTeq. Les équipements suivis par AROTEQ combinent instrumentation, automatisme, acquisition de données et contraintes d'exploitation. La disponibilité des machines y a donc une valeur immédiate pour la production.

Table 1.1 – Fiche synthétique d'AROTÉQ

Champ	Contenu
Nom de l'entreprise	AROTÉQ
Localisation	Ben Arous, Tunisie
Spécialité	Ingénierie industrielle, machines spéciales, systèmes de test, automatisation et intégration robotique
Compétences clés	Conception mécanique et électrique, automatisme, systèmes embarqués, vision industrielle, supervision et intégration de solutions intelligentes
Partenaires mentionnés dans la documentation	Keyence, Bosch Rexroth, Festo, Equiptest
Cadre du stage	Projet de maintenance prédictive basé sur l'IoT, l'acquisition embarquée et le Machine Learning, mené dans un environnement où logiciel, automatisme et instrumentation interagissent en continu



Figure 1.1 – AROTEQ : identité visuelle et environnement d'atelier.

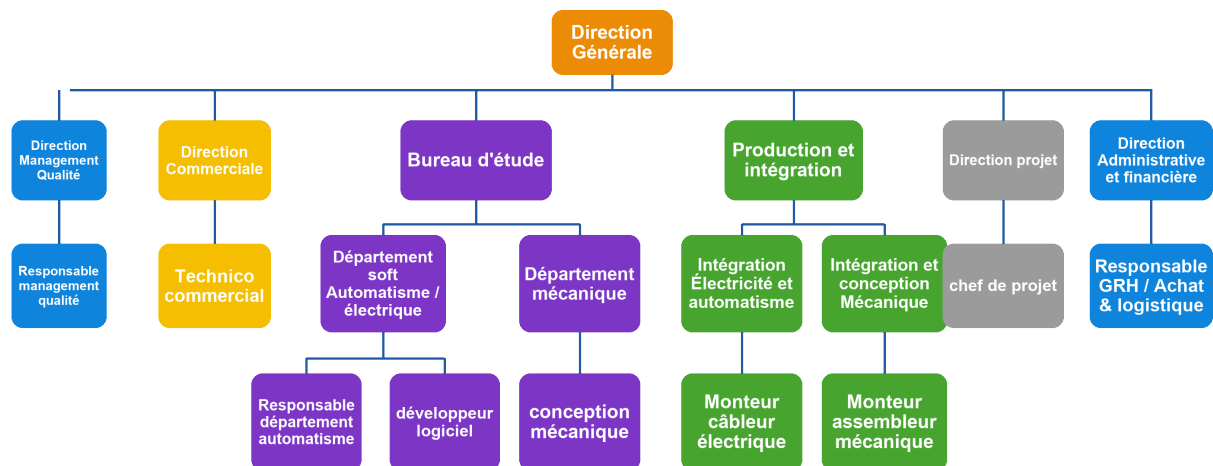


Figure 1.2 – Organigramme d'AROTÉQ.

1.2.2 Positionnement du stage

Le stage a été réalisé au sein du bureau d'étude, avec un rattachement principal au département logiciel et automatisme. Le travail y a été mené dans un rôle de développeur logiciel. Ce positionnement est important. Il place le projet au contact des besoins d'intégration, des contraintes d'exploitation et des échanges entre logiciel, électricité et instrumentation.

Le travail demandé ne relevait donc pas d'une étude théorique isolée. Il devait produire un résultat scientifique défendable, une plateforme démontrable et une base réutilisable pour une intégration industrielle future.

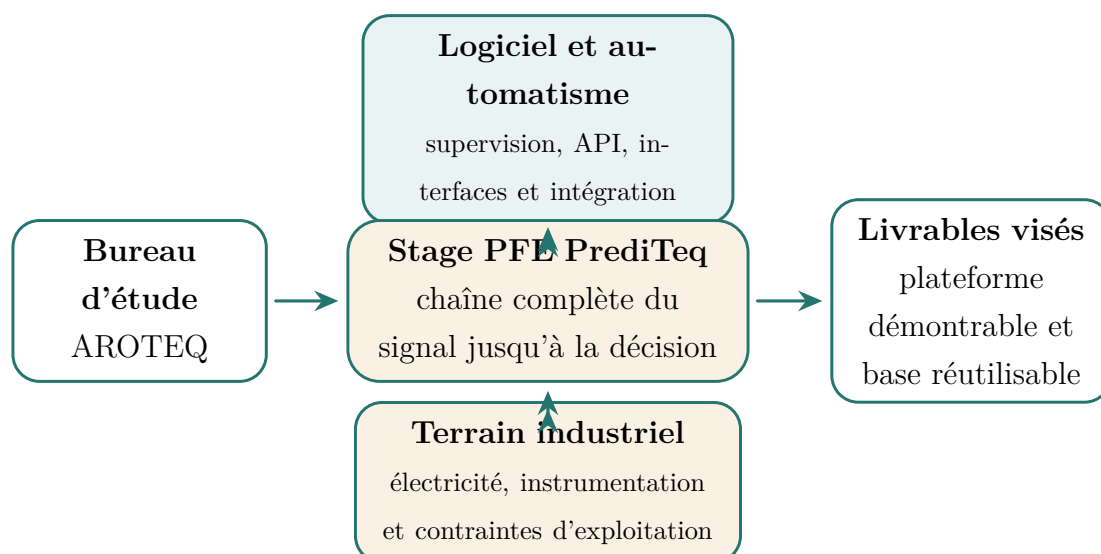


Figure 1.3 – Positionnement du stage dans le flux projet.

1.2.3 Domaines d'activité et intérêt pour le projet

Les documents internes d'AROTÉQ montrent un portefeuille cohérent avec PrediTeq :

- conception et fabrication de machines spéciales ;
- réalisation de postes automatisés ;
- développement de systèmes de test et de contrôle ;
- intégration de solutions de robotique et d'automatisation ;
- mise en oeuvre de solutions combinant instrumentation, supervision, IoT et systèmes embarqués.

Dans un tel environnement, une panne n'est pas seulement un incident technique. Elle affecte la cadence, la qualité et la disponibilité. La maintenance prédictive y prend donc un sens concret.

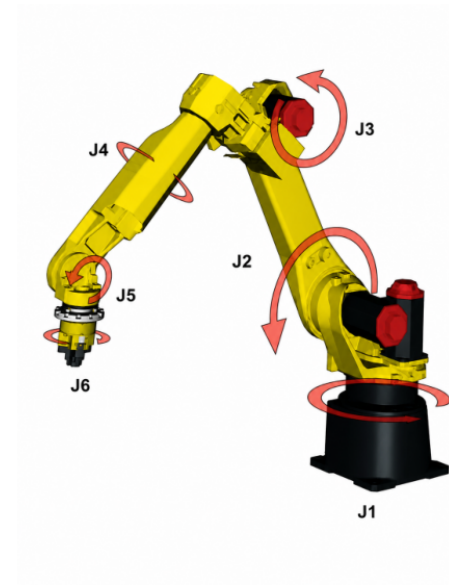


Figure 1.4 – Technologies industrielles.



Figure 1.5 – Système de test industriel.

Table 1.2 – Axes de contexte utiles au projet

Axe	Lecture utile pour le projet
Automatisation industrielle	Nécessité de systèmes fiables, répétables et simples à superviser.
Intégration électrique et logicielle	Le projet doit relier signaux, logique applicative, communication et visualisation.
Disponibilité machine	Une panne détectée trop tard entraîne arrêt, coût et désorganisation.
Systèmes de test et de contrôle	Le contexte valorise naturellement le suivi d'état, la mesure et l'interprétation des signaux.
Ouverture vers le terrain	La future chaîne LabVIEW ou automate programmable industriel impose de penser l'interopérabilité des flux.

1.3 Présentation du projet et problématique

1.3.1 Contexte du besoin

Le cas d'usage retenu concerne le moteur d'entraînement d'un stockeur vertical rotatif. Ce choix joue un rôle de point d'appui. Il fournit à PrediTeq une première machine de référence, cohérente avec l'environnement AROTEQ et suffisamment concrète pour ancrer la simulation, le produit et la démonstration. Le projet n'est toutefois pas pensé comme un outil réservé à ce seul équipement.

Dans ce type d'équipement, l'enjeu n'est pas seulement de constater qu'une machine fonctionne encore. Il faut aussi savoir si son état se dégrade et s'il reste une marge d'action avant une zone critique. Cette logique vaut d'ailleurs au-delà du stockeur. Elle correspond plus largement à tout contexte industriel où une dérive progressive doit être détectée puis transformée en décision de maintenance.

Une maintenance corrective intervient après la panne. Une maintenance préventive intervient selon une périodicité fixe. La maintenance prédictive cherche une voie intermédiaire. Elle déclenche l'action lorsque les signaux et leur évolution montrent qu'une dérive est en cours.

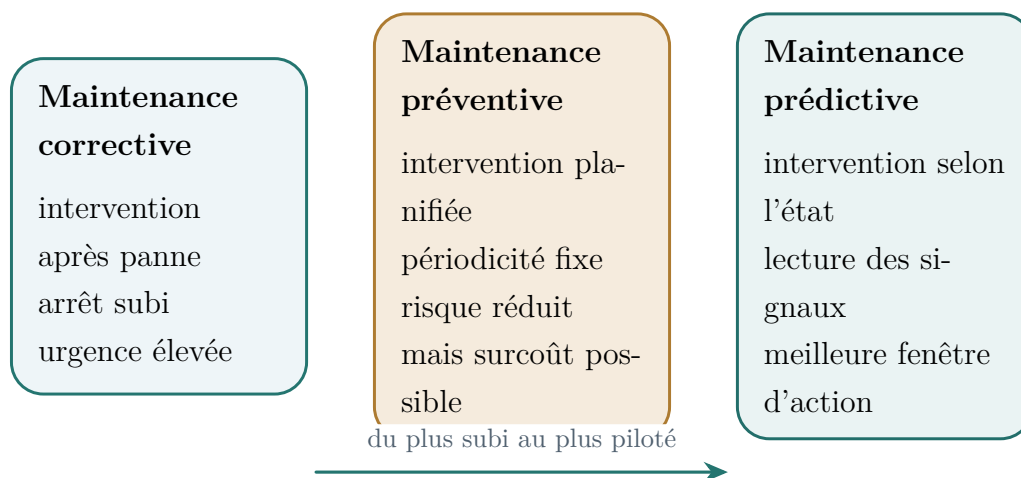


Figure 1.6 – Familles de maintenance

1.3.2 Problématique

La problématique centrale du projet peut être formulée ainsi :

Comment concevoir une plateforme de maintenance prédictive capable d'estimer de manière crédible l'état de santé et la durée de vie résiduelle d'une machine industrielle, en la pilotant d'abord sur le moteur d'un stockeur vertical rotatif, alors que l'historique réel de défaillances instrumentées reste limité et qu'une démonstration logicielle complète doit être disponible en parallèle ?

Cette problématique implique quatre besoins :

1. produire une base d'apprentissage cohérente malgré le manque initial de données terrain longues ;
2. transformer les signaux en indicateurs lisibles pour un non-spécialiste ;
3. intégrer ces indicateurs dans une plateforme logicielle complète ;
4. préparer une ouverture progressive vers un flux industriel réel.

1.3.3 Objectifs du projet

L'objectif général est de proposer une plateforme logicielle en mode service, ou *Software as a Service* (SaaS), capable de surveiller l'état d'une machine, d'estimer un horizon de maintenance et de présenter cette information dans une interface exploitable. Le moteur du stockeur vertical rotatif sert ici de première base de référence, et non de limite définitive.

Cet objectif se décline en objectifs plus précis :

- construire un pipeline de simulation, d'apprentissage et d'export cohérent pour une première famille de machines ;
- charger ces artefacts dans un backend de supervision temps réel ;
- proposer une interface web de diagnostic, de planification et de suivi ;
- offrir une démonstration lisible avec plusieurs scénarios machine ;
- préparer un raccordement futur à une source industrielle réelle ;
- préparer l'extension vers d'autres machines AROTEQ, puis vers des usages industriels plus généraux.

1.4 Étude de l'existant

1.4.1 Familles de solutions existantes

Les solutions de maintenance prédictive se répartissent souvent en trois familles :

- les offres industrielles propriétaires, robustes mais souvent coûteuses et peu ouvertes ;
- les suites de gestion de maintenance et de supervision, utiles pour l'organisation mais plus limitées en pronostic natif ;
- les démonstrateurs académiques, solides sur le plan méthodologique mais peu reliés à un produit logiciel complet.

Cette lecture de l'existant est importante pour situer PrediTeq. Le projet ne doit ni être comparé à une simple maquette de laboratoire, ni être présenté comme une suite industrielle déjà finalisée.

1.4.2 Lecture comparative

Table 1.3 – Lecture comparative de l'existant

Critère	Suite industrielle fermée	Prototype ML pur	PrediTeq
Simulation physique-ment guidée	Variable	Parfois	Oui
Pipeline d'apprentissage traçable	Variable	Oui	Oui
Backend métier	Souvent oui	Rare	Oui
Frontend multi-pages	Oui	Rare	Oui
Démonstration guidée	Faible	Variable	Oui
Ouverture vers un flux industriel réel	Variable	Rare	Oui
Adaptation à une PME industrielle	Faible	Moyenne	Forte

1.5 Critique de l'existant

L'étude de l'existant fait apparaître deux limites récurrentes.

La première limite concerne les solutions industrielles fermées. Elles offrent des fonctions avancées, mais leur coût, leur faible ouverture et leur dépendance à un fournisseur les rendent moins adaptées à un projet de PFE centré sur la compréhension complète de la chaîne.

La seconde limite concerne les prototypes purement académiques. Ils valident souvent une méthode de traitement ou de prédiction, mais ils laissent de côté le backend, la persistance, la gouvernance des accès et la démonstration produit.

Le besoin traité par PrediTeq se situe précisément entre ces deux extrêmes. Il faut une solution techniquement sérieuse, mais aussi lisible, démontrable et réutilisable.

1.6 Solution proposée

1.6.1 Vue d'ensemble de PrediTeq

PrediTeq répond à ce besoin par une architecture en trois blocs :

- un apprentissage hors ligne pour la simulation, l'entraînement, la validation et l'export des artefacts ;
- un backend de supervision temps réel pour charger les modèles, recevoir les mesures et produire l'état machine ;
- une interface web pour la visualisation, le diagnostic, la planification et l'administration.

Cette architecture est pensée pour dépasser le seul cas de référence. Le service temps réel repose sur un contrat de données commun à la supervision. La chaîne d'apprentissage, elle, peut être réentraînée pour une autre famille de machines à partir de ses propres signaux et de ses propres hypothèses physiques. Le stockeur vertical rotatif sert donc de point de départ crédible, pas de frontière définitive.

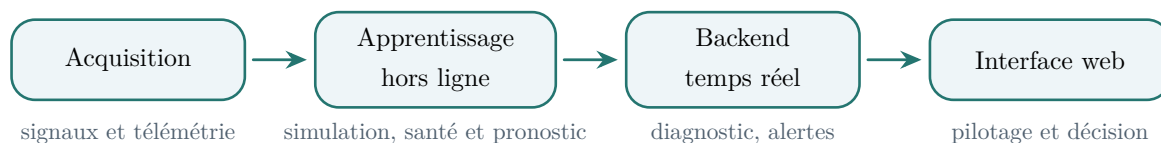


Figure 1.7 – Vue d'ensemble de PrediTeq

1.6.2 Apports attendus

La solution proposée apporte trois niveaux de valeur.

Elle apporte d'abord une valeur scientifique. Elle formalise une chaîne complète de simulation, d'indice de santé et de durée de vie résiduelle.

Elle apporte ensuite une valeur logicielle. Elle transforme cette chaîne en service de supervision, avec persistance, comptes, alertes et interface.

Elle apporte enfin une valeur de démonstration et d'ouverture. Le projet dispose d'un simulateur guidé, d'une page d'acquisition expérimentale et d'une trajectoire claire vers une source industrielle réelle.

Son positionnement doit donc être formulé avec mesure. PrediTeq n'est pas encore validé sur toute famille de machines industrielles. En revanche, la plateforme est générale par conception et sa logique de supervision est déjà partiellement éprouvée au-delà du seul cas interne. Le benchmark NASA CMAPSS montre que la méthode conserve de la valeur sur un autre type de machine. La prochaine étape préparée consiste à déployer cette logique sur d'autres machines AROTEQ, puis à élargir le périmètre d'usage.

1.7 Méthodologie de travail

1.7.1 Choix d'une conduite agile

Le projet combine plusieurs types de travail : simulation, backend, frontend, démonstration et intégration progressive. Une conduite strictement séquentielle aurait rendu les ajustements difficiles. Une organisation inspirée de Scrum était donc plus adaptée [11].

Cette approche permet de livrer par incréments, de vérifier chaque bloc et d'ajuster le produit à mesure que les besoins deviennent plus concrets.

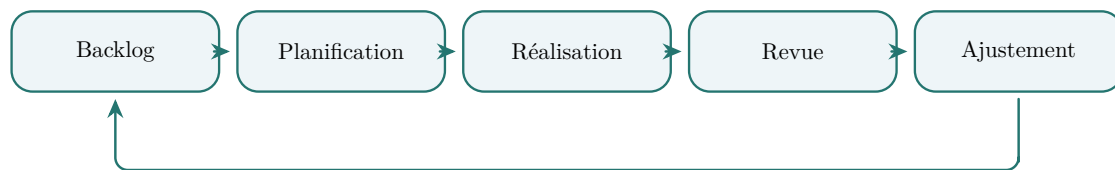


Figure 1.8 – Cycle de travail retenu

1.7.2 Découpage du travail

Le développement s'est structuré autour de grands incréments :

- construction du pipeline et du socle d'indice de santé ;
- mise en place du backend temps réel et du simulateur ;
- réalisation du frontend produit ;
- ajout de la page d'acquisition expérimentale ;
- préparation de la future intégration industrielle.

Incrément	S1	S2	S3	S4	S5
Simulation et pipeline ML	X	X			
Backend temps réel et simulateur		X	X		
Frontend produit			X	X	
Page Experiment et démonstration				X	X
Préparation de l'intégration réelle					X

Figure 1.9 – Planning simplifié du travail

1.8 Conclusion

Ce chapitre a présenté le cadre industriel du projet, le besoin traité et les limites des solutions existantes. Il a montré que PrediTeq répond à une situation intermédiaire entre prototype académique et produit industriel. Il a aussi précisé la solution proposée et la méthodologie de travail retenue. Le chapitre suivant expose maintenant l'état de l'art nécessaire pour comprendre les notions de maintenance prédictive, d'indice de santé, de durée de vie résiduelle et d'acquisition.

Chapitre 2

État de l’art

2.1 Introduction

PrediTeq se situe au croisement de plusieurs domaines : maintenance industrielle, analyse de signaux, apprentissage automatique et Internet des objets, ou *Internet of Things* (IoT), appliqués à une plateforme de supervision. Ce chapitre présente les notions utiles à la lecture du projet. Il précise d’abord les bases de la maintenance prédictive. Il introduit ensuite l’indice de santé et la durée de vie résiduelle. Il expose enfin les méthodes retenues et l’environnement technologique de la solution.

2.2 Maintenance industrielle et prédiction

2.2.1 Maintenance corrective, préventive et prédictive

La maintenance corrective intervient après la panne. Elle est simple à comprendre, mais elle expose l’atelier à des arrêts subis.

La maintenance préventive intervient selon une périodicité fixe. Elle réduit le risque de panne brutale, mais elle peut conduire à intervenir trop tôt.

La maintenance prédictive cherche une lecture plus fine. Elle s’appuie sur les signaux de la machine et sur leur évolution pour choisir le moment d’intervention le plus pertinent.

2.2.2 Pourquoi cette approche est pertinente ici

Dans le cas d’un stockeur vertical rotatif, l’enjeu n’est pas seulement de savoir si la machine tourne encore. Il faut aussi savoir dans quel état de fatigue elle se trouve et combien de temps il reste avant une zone critique.

Cette logique conduit naturellement à deux indicateurs complémentaires :

- un indicateur d’état courant ;
- un indicateur d’horizon de maintenance.

2.3 Concepts clés de pronostic

2.3.1 Indice de santé

L'indice de santé, ou *Health Index* (HI), est une variable synthétique qui résume l'état d'une machine. Il ne s'agit pas d'une mesure physique directe. C'est un indicateur reconstruit à partir des signaux observables.

Son intérêt est double. Il condense plusieurs symptômes en une lecture simple. Il sert aussi de base à la définition des zones d'état et à la construction du pronostic.

Dans PrediTeq, les zones du HI suivent une lecture compatible avec l'esprit de l'ISO 10816-3 :2009 [4] :

- HI supérieur ou égal à 0,8 : situation saine ;
- HI entre 0,6 et 0,8 : situation correcte à surveiller ;
- HI entre 0,3 et 0,6 : situation dégradée ;
- HI inférieur à 0,3 : situation critique.

2.3.2 Durée de vie résiduelle

La durée de vie résiduelle, ou *Remaining Useful Life* (RUL), représente le temps estimé avant l'entrée dans une zone critique. C'est l'indicateur qui transforme la lecture d'état en horizon d'action.

Dans PrediTeq, ce pronostic repose sur un point méthodologique important. La cible d'apprentissage finale est construite à partir d'un signal de santé lissé et observable, puis d'un franchissement persistant du seuil critique. Le modèle n'apprend donc pas directement sur une variable cachée inaccessible en exploitation.

Cette construction rapproche l'apprentissage de ce qui sera réellement disponible plus tard dans le service temps réel.

2.3.3 Publication progressive du pronostic

Le produit n'affiche pas toujours immédiatement un nombre de jours. Trois situations doivent être distinguées :

- une machine encore très saine reste dans une lecture de référence stable ;
- une machine récemment démarrée passe par une phase d'initialisation, le temps de constituer un historique suffisant ;
- une machine disposant d'un historique exploitable peut recevoir un pronostic publié avec son intervalle de confiance.

Cette logique protège l'utilisateur contre des chiffres prématurés. Elle privilégie un résultat plus tardif, mais plus crédible.

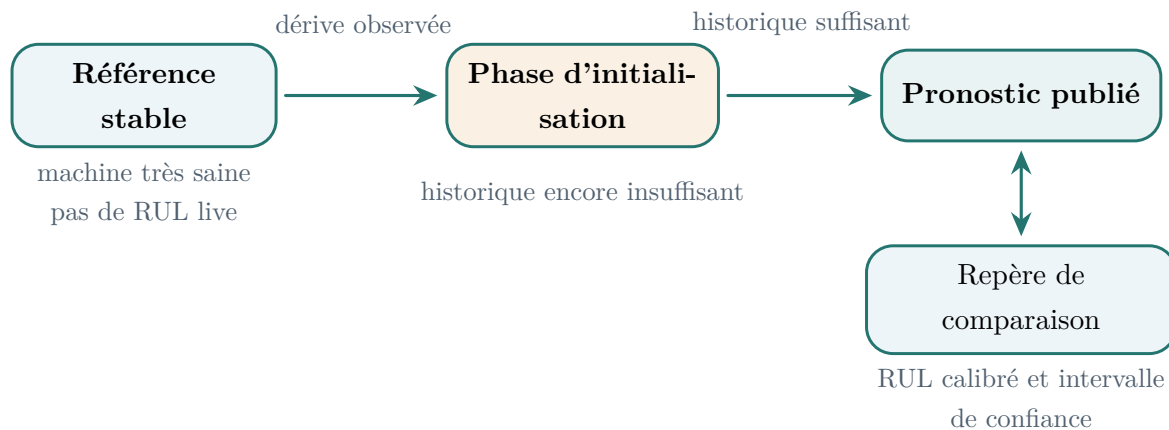


Figure 2.1 – Lecture produit du pronostic

2.4 Méthodes de Machine Learning retenues

2.4.1 Apprentissage supervisé et non supervisé

PrediTeq combine deux logiques complémentaires :

- une logique non supervisée pour repérer les comportements anormaux ;
- une logique supervisée pour estimer la durée de vie résiduelle.

La première répond à la question : « ce comportement ressemble-t-il encore à une zone saine ? »

La seconde répond à la question : « combien de temps reste-t-il avant un état critique ? »

2.4.2 Détection d'anomalies et score hybride

La détection d'anomalies s'appuie sur une forêt d'isolement, ou *Isolation Forest* [2]. Cette méthode est bien adaptée lorsque l'on décrit mieux le comportement sain que l'ensemble des pannes possibles.

Dans PrediTeq, cette lecture n'est pas laissée seule. Elle est combinée à un indicateur vibratoire direct pour former un score hybride. Cette combinaison permet de conserver une lecture physique simple tout en profitant d'une analyse plus riche des signaux.

2.4.3 Régression pour la durée de vie résiduelle

Le pronostic de durée de vie résiduelle repose sur une forêt aléatoire, ou *Random Forest* [1]. Ce choix est cohérent avec les besoins du projet :

- gérer des relations non linéaires ;
- accepter des variables de nature différente ;
- conserver un bon compromis entre performance, robustesse et lisibilité.

La méthode retenue reste ainsi compatible avec un mémoire d’ingénierie où l’explicabilité et la stabilité comptent autant que le score final.

2.4.4 Pourquoi ne pas commencer par des modèles plus lourds

Des approches plus complexes, comme les réseaux récurrents ou les autoencodeurs, auraient été possibles. Elles n’étaient pas le meilleur point de départ ici.

Le projet avait d’abord besoin d’une chaîne lisible, défendable et progressive. Il fallait d’abord stabiliser la simulation, le prétraitement, les caractéristiques, la détection d’anomalies et l’indice de santé. Ce choix méthodologique est plus adapté au niveau réel des données disponibles.

2.4.5 Validation par groupes et validation externe

L’évaluation ne doit pas mélanger des fragments d’une même trajectoire entre apprentissage et test. C’est pourquoi PrediTeq s’appuie sur une validation croisée par groupes (GroupKFold) [8]. Cette précaution évite une fuite d’information.

Le projet ajoute aussi une validation externe sur NASA CMAPSS, ou *Commercial Modular Aero-Propulsion System Simulation* [6]. Cette étape ne cherche pas à assimiler un moteur de stockeur à un moteur aéronautique. Elle vérifie plus simplement que la logique du pipeline conserve une valeur méthodologique au-delà du seul jeu de simulation interne.

2.5 IoT, instrumentation et communication

2.5.1 Trois niveaux d'acquisition

L'Internet des objets (IoT) n'est pas ici une couche accessoire. Il relie la théorie prédictive à l'usage concret. Dans PrediTeq, il faut distinguer trois niveaux d'acquisition :

1. la simulation hors ligne, qui produit les trajectoires d'apprentissage ;
2. le simulateur applicatif, qui rejoue des scénarios machine pour la démonstration ;
3. l'acquisition physique, qui prend aujourd'hui la forme d'un banc expérimental, puis demain d'un flux industriel réel.

2.5.2 Choix du protocole de transport

Le protocole Message Queuing Telemetry Transport (MQTT) est retenu pour le transport des mesures live. Ce choix est justifié par sa légèreté, sa logique de publication-abonnement et son usage courant dans les architectures IoT.

Dans l'ouverture industrielle visée, une source locale industrielle, ou un poste local, écrit une mesure dans un CSV. Un pont de normalisation la transforme ensuite en message métier, puis un broker MQTT privé la transmet au backend cloud et à l'interface.

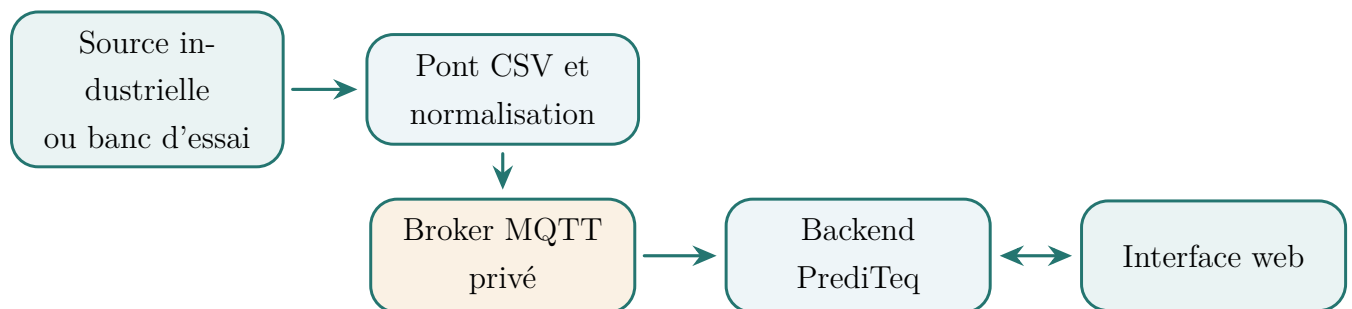


Figure 2.2 – Architecture MQTT

2.5.3 Capteurs industriels et capteurs de banc

Le projet distingue les capteurs visés en exploitation et les capteurs retenus pour la démonstration de banc. Pour la page Experiment, le choix s'est porté sur deux capteurs simples :

- un capteur de courant ACS712 5A ;
- un capteur inertiel MPU6050 pour la vibration.

Leur rôle n'est pas de valider toute l'instrumentation industrielle finale. Leur rôle est de prouver qu'un flux live simple peut être reçu, affiché et interprété dans l'application.

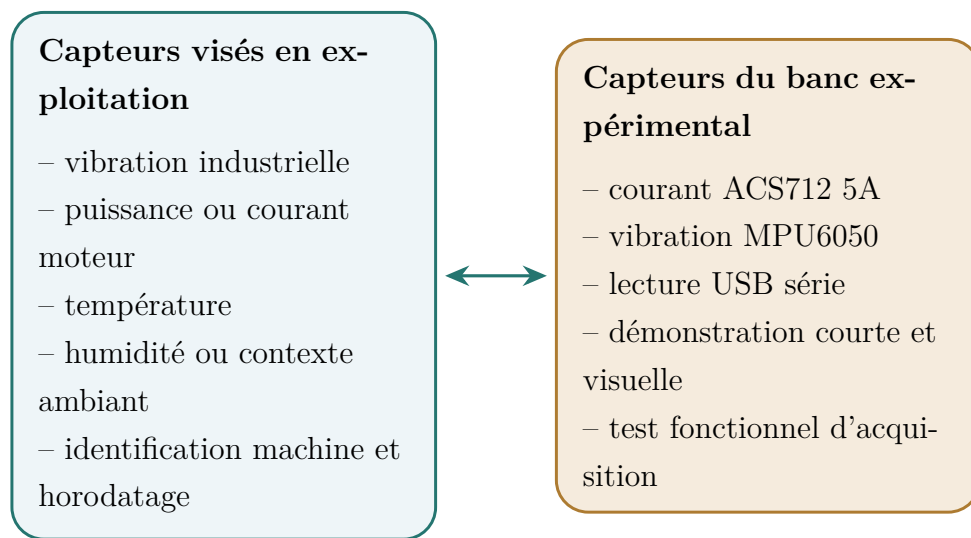


Figure 2.3 – Capteurs visés et capteurs de banc

2.6 Environnement logiciel

2.6.1 React, TypeScript et Vite



Figure 2.4 – Technologies de l'interface web : React, TypeScript et Vite

React, TypeScript et Vite structurent l'interface web. Ce choix convient à une application de supervision riche en états, en composants réutilisables et en pages spécialisées.

2.6.2 FastAPI



FastAPI

Figure 2.5 – Logo de FastAPI

FastAPI porte le backend de supervision. Cette solution est adaptée au chargement des artefacts d'apprentissage, à la publication de routes métier et à la gestion d'une supervision temps réel lisible.

2.6.3 Supabase



Supabase

Figure 2.6 – Logo de Supabase

Supabase assure la persistance et la gouvernance. Cette base centralise les machines, les historiques, les comptes, les seuils et les journaux utiles au suivi du produit.

2.6.4 MQTT et cohérence de la pile



React



FastAPI



Supabase



MQTT

Figure 2.7 – Pile retenue : React, FastAPI, Supabase et MQTT

La pile React, FastAPI, Supabase et MQTT suit une logique simple :

- React pour l'expérience de supervision ;
- FastAPI pour le service de diagnostic ;

- Supabase pour la persistance et la gouvernance ;
- MQTT pour le transport léger du live.

L’ensemble forme une base cohérente pour une plateforme de maintenance prédictive.

2.7 Conclusion

Ce chapitre a présenté les notions nécessaires à la lecture du projet : maintenance prédictive, indice de santé, durée de vie résiduelle, détection d’anomalies, régression, acquisition et pile logicielle. Il a montré que les choix de PrediTeq reposent sur une chaîne cohérente plutôt que sur une accumulation d’outils. Le chapitre suivant formalise maintenant les besoins, les acteurs et les contraintes du système.

Chapitre 3

Analyse des besoins

3.1 Introduction

L'analyse des besoins part des usages réellement couverts par PrediTeq. La plateforme ne se limite pas à un calcul prédictif. Elle doit recevoir des mesures, construire un diagnostic, publier un pronostic lorsque l'historique le permet, conserver les traces utiles et présenter l'ensemble dans une interface compréhensible.

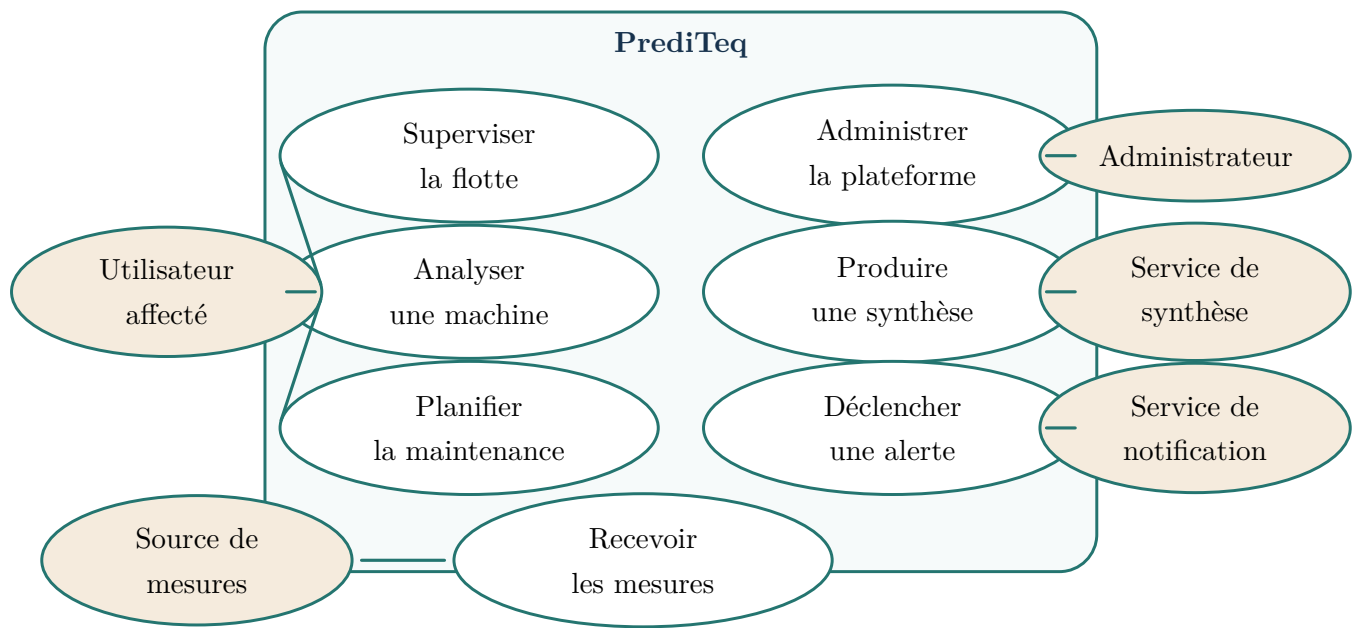
Cette analyse précise d'abord les acteurs. Elle identifie ensuite les besoins fonctionnels et non fonctionnels. Elle reformule ensuite les usages sous forme de user stories. Elle rappelle enfin les contraintes qui encadrent le projet et le cycle de surveillance visé.

3.2 Identification des acteurs

Table 3.1 – Acteurs du système

Acteur	Justification	Rôle principal
Administrateur	Gestion des comptes, des seuils, des machines et de l'espace de démonstration	Piloter la plateforme, paramétrer les machines, approuver les comptes, suivre les coûts et administrer les seuils terrain
Utilisateur affecté à une machine	Consultation des pages métier et accès limité à la machine concernée	Lire l'état de sa machine, suivre les alertes, consulter les tâches et exploiter les synthèses disponibles
Source de mesures	Ingestion en temps réel par voie industrielle, réseau ou acquisition expérimentale	Envoyer les mesures brutes du moteur vers la plateforme
Service de notification	Gestion des courriels et des traces d'envoi	Transmettre les alertes aux destinataires configurés
Service de synthèse	Génération de rapports à la demande	Produire une synthèse rédigée à partir de l'état courant de la machine

Deux familles d'acteurs apparaissent. La première agit directement sur la plateforme et prend les décisions de maintenance. La seconde alimente, relaie ou restitue l'information sans se substituer au décideur humain.

**Figure 3.1** – Cas d'utilisation principaux

3.3 Besoins fonctionnels

Les besoins fonctionnels sont regroupés dans le tableau 3.2. Ils décrivent la chaîne attendue, depuis la mesure brute jusqu'à l'action de maintenance.

Table 3.2 – Besoins fonctionnels retenus

Code	Fonction attendue	Effet attendu
BF1	Acquérir des mesures en temps réel	Recevoir les valeurs de vibration, de puissance, de température et d'humidité depuis une source réseau ou expérimentale
BF2	Interpréter l'état courant	Transformer les mesures brutes en indice de santé, zone d'état et indicateurs de diagnostic
BF3	Publier un pronostic exploitable	Estimer une durée de vie résiduelle avec intervalle de confiance lorsque l'historique requis est disponible
BF4	Conserver la traçabilité	Enregistrer les historiques de santé, de pronostic, d'alertes, de coûts et d'actions d'administration
BF5	Alerter les utilisateurs	Déclencher des alertes locales et des courriels selon l'état de la machine et les seuils terrain
BF6	Piloter la maintenance	Gérer des tâches, un calendrier, des priorités et des coûts de maintenance
BF7	Visualiser la flotte	Offrir un tableau de bord, une vue multi-machine, une page diagnostic et des indicateurs de synthèse
BF8	Produire des synthèses	Générer un rapport d'aide à la décision et une restitution lisible pour le technicien ou le responsable de maintenance
BF9	Administrer la solution	Gérer les comptes, l'affectation des machines, les seuils et les espaces de démonstration

Ces besoins montrent que la plateforme ne s'arrête pas au calcul d'un score. Elle doit convertir une mesure en information utile, puis cette information en décision, en trace historique et en restitution visuelle.

3.4 Besoins non fonctionnels

Table 3.3 – Besoins non fonctionnels

Code	Exigence	Justification
BNF1	Séparation claire des couches	L'apprentissage hors ligne, le backend temps réel et l'interface web doivent rester distincts pour faciliter la maintenance et l'évolution
BNF2	Sécurité d'accès	L'application doit imposer l'authentification, les rôles, le contrôle d'accès par machine et la protection des routes sensibles
BNF3	Traçabilité	Chaque décision importante doit laisser une trace : historiques, alertes, journaux de courriels et journaux d'audit
BNF4	Configurabilité terrain	Les seuils opérationnels doivent pouvoir évoluer sans réentraîner la chaîne hors ligne
BNF5	Fonctionnement multi-machine	Le service temps réel doit gérer une instance logique par machine et conserver un historique distinct
BNF6	Continuité de service	Le backend doit redémarrer en rechargeant ses artefacts et ses machines sans perdre la logique de supervision
BNF7	Lisibilité pour la démonstration	Le système doit fournir un simulateur et une page expérimentale afin de montrer la chaîne complète même sans campagne terrain longue

3.5 Contraintes du projet

Le projet est encadré par plusieurs contraintes de conception.

Table 3.4 – Contraintes majeures du projet

Contrainte	Conséquence sur la solution
Historique terrain encore limité	L'apprentissage repose d'abord sur une simulation de dégradation physiquement guidée
Apprentissage séparé du runtime	Le service temps réel recharge des artefacts exportés ; il n'entraîne pas lui-même les modèles
Seuils opérationnels variables	Les seuils utilisés en exploitation peuvent différer des seuils hors ligne car ils sont ajustables dans la base
Mode démonstration nécessaire	Le simulateur applique des scénarios publics stables pour garantir une lecture claire en soutenance

Ces contraintes fixent le niveau réel de maturité de PrediTeq. Le projet constitue une base solide pour la supervision, le diagnostic et la démonstration. Son apprentissage reste encore nourri principalement par une simulation physiquement guidée.

3.6 User stories prioritaires

Les besoins précédents peuvent aussi être formulés sous forme de user stories. Cette écriture relie l'usage attendu à la valeur rendue. Elle aide donc à vérifier que la plateforme ne se limite pas à calculer un indicateur, mais soutient bien une décision de maintenance.

Table 3.5 – User stories prioritaires

Acteur	User story	Valeur attendue	Priorité
Administrateur	En tant qu'administrateur, je veux enregistrer une machine, ses seuils et ses utilisateurs afin d'initialiser une supervision exploitable.	Mise en service cohérente de la plateforme	Haute
Utilisateur affecté	En tant qu'utilisateur affecté à une machine, je veux consulter le tableau de bord et le diagnostic avancé afin de savoir si une action est nécessaire.	Lecture rapide du risque et de son contexte	Haute
Utilisateur affecté	En tant qu'utilisateur affecté à une machine, je veux recevoir ou consulter une alerte lorsque la machine entre en zone dégradée ou critique afin de préparer l'intervention.	Réaction plus rapide à une dérive	Haute
Administrateur	En tant qu'administrateur, je veux planifier une tâche de maintenance et suivre son coût afin d'organiser l'intervention et son impact.	Passage du diagnostic à la décision	Moyenne
Source de mesures	En tant que source de mesures, je veux transmettre une mesure horodatée au service temps réel afin d'alimenter le calcul de l'état machine.	Alimentation continue de la supervision	Haute
Administrateur	En tant qu'administrateur, je veux générer une synthèse de l'état machine afin de partager rapidement une recommandation exploitable.	Communication plus claire avec les décideurs	Moyenne

3.7 Cycle de surveillance visé

Le besoin principal de la plateforme peut être résumé par le cycle de la figure 3.2. La logique est simple : recevoir une mesure, calculer l'état, publier un pronostic si le contexte le permet, historiser, puis alerter si nécessaire.

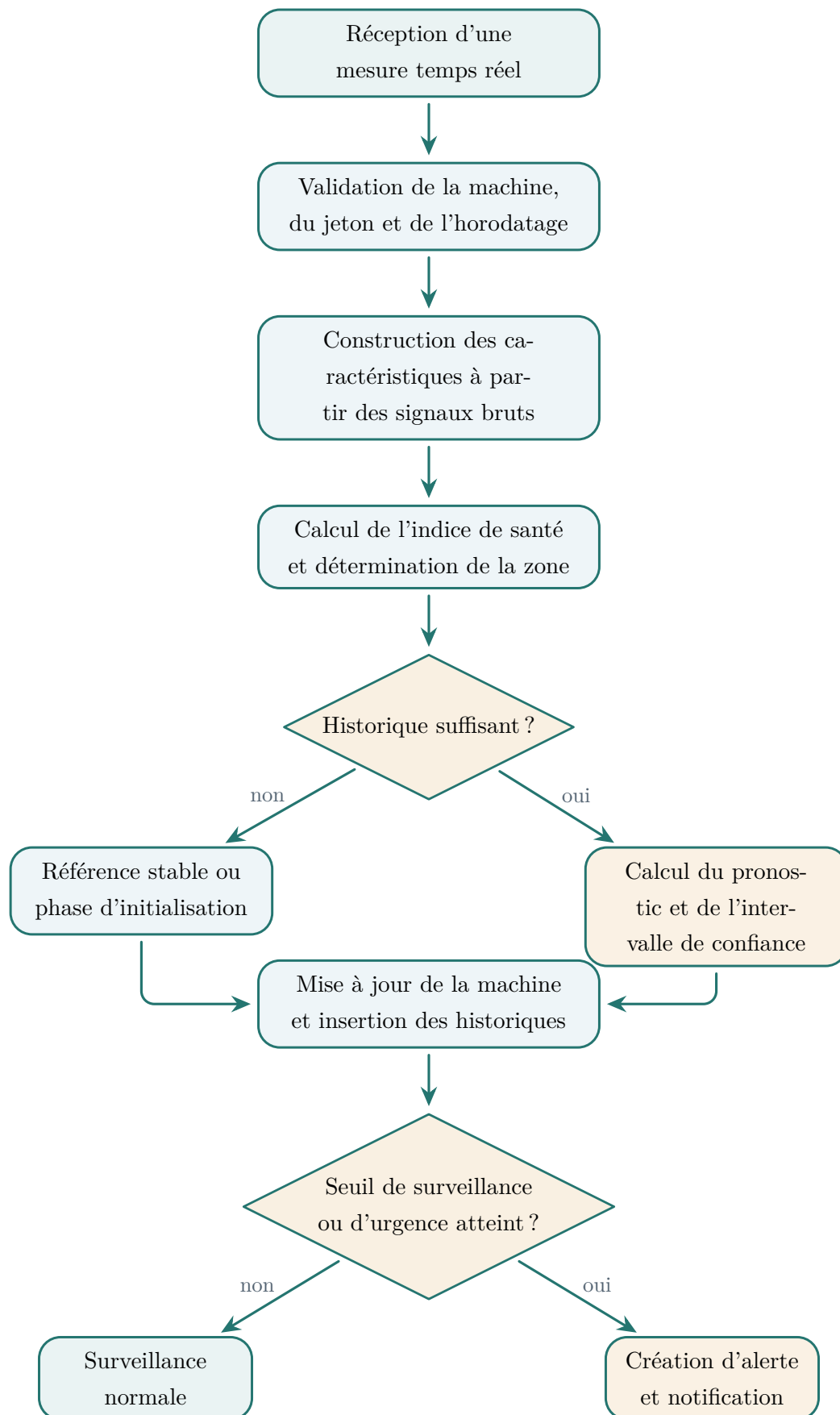


Figure 3.2 – Cycle de surveillance temps réel

3.8 Conclusion

Cette analyse montre que PrediTeq doit assurer une chaîne complète, depuis la mesure brute jusqu'à la décision de maintenance. Les acteurs sont identifiés, les fonctions principales sont définies, les user stories prioritaires traduisent les usages attendus et les contraintes du projet sont explicites. La valeur du système repose autant sur la supervision et la traçabilité que sur le calcul prédictif lui-même. Le chapitre suivant présente la conception de l'architecture et des flux qui rendent ces besoins réalisables.

Chapitre 4

Conception

4.1 Introduction

Après l'analyse des besoins, il faut organiser la solution en blocs cohérents. La conception de PrediTeq sépare clairement l'apprentissage hors ligne, la supervision temps réel, la persistance et l'interface web. Cette séparation améliore la lisibilité du système et facilite son évolution.

Ce chapitre présente l'architecture générale, le noyau runtime, l'organisation de la base de données, la séquence de supervision et la logique d'accès de l'interface.

4.2 Architecture générale de la solution

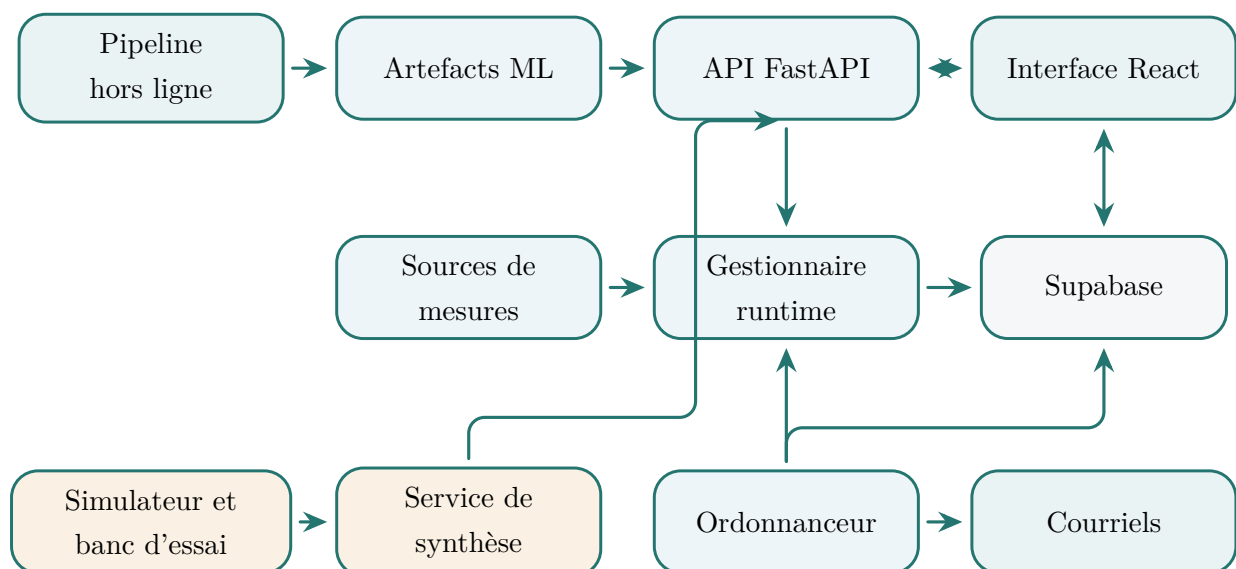


Figure 4.1 – Architecture générale de PrediTeq

L'architecture fait apparaître trois découpages structurants :

- la chaîne hors ligne produit les modèles, les paramètres et les figures de résultats ;
- le backend recharge ces artefacts, maintient une logique par machine et écrit l'état courant dans la base ;
- l'interface web lit ensuite ces informations pour présenter l'état de la flotte, les alertes, les tâches et les synthèses.

Cette organisation respecte la logique du projet. Le backend n'entraîne pas les modèles. Il exploite des artefacts déjà exportés par la chaîne hors ligne.

4.3 Noyau runtime

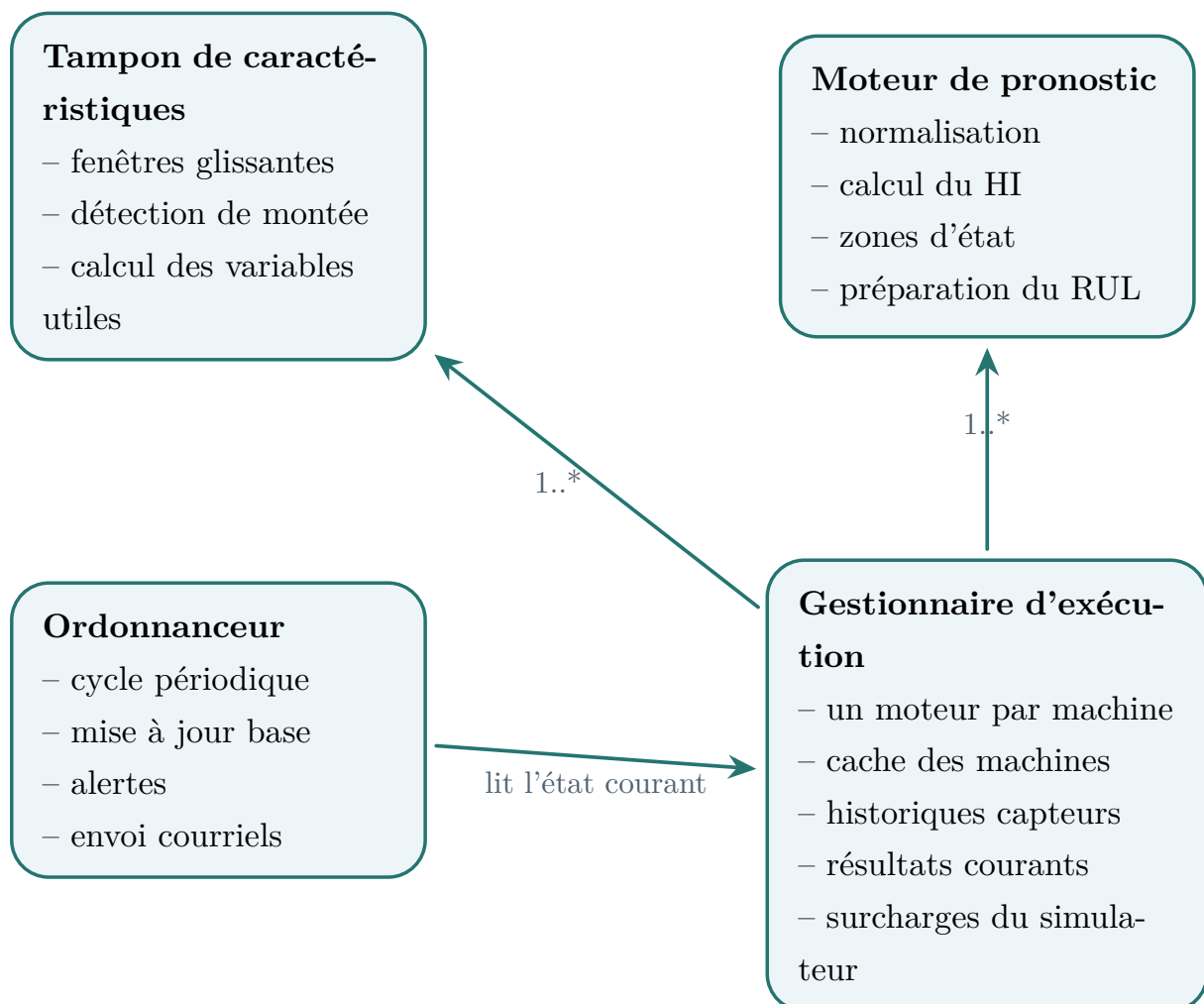


Figure 4.2 – Diagramme de classes simplifié du noyau runtime

Le noyau applicatif suit une logique simple. Le tampon transforme les mesures brutes en variables exploitables. Le moteur calcule ensuite l'indice de santé, la zone et le contexte de pronostic. Le gestionnaire coordonne une instance logique par machine. Enfin, l'ordonnanceur persiste l'état du système et pilote les alertes.

4.4 Conception de la persistance

Table 4.1 – Structures principales de persistance

Structure	Rôle	Information principale
Vue courante des machines	Image instantanée de la flotte	Code machine, statut, HI courant, RUL courant, localisation et dernière mise à jour
Profils utilisateurs	Gouvernance des accès	Utilisateur, rôle, statut d’approbation et machine affectée
Historique de santé	Mémoire de l’état machine	Valeur HI, score d’anomalie et statut daté
Historique du pronostic	Mémoire du RUL	RUL en jours, bornes basse et haute, horodatage
Centre d’alertes	Priorisation opérationnelle	Type, sévérité, acquittement et date de création
Tâches de maintenance	Pilotage des interventions	Titre, priorité, technicien, date planifiée et statut
Suivi budgétaire	Lecture économique	Main-d’oeuvre, pièces et total mensuel
Historique des synthèses	Restitution métier	Période, langue, titre et contenu généré
Paramétrage terrain	Réglage des seuils	Frontières du HI et du RUL, destinataires de notification
Journaux système	Traçabilité	Historique d’envoi et traces des actions sensibles

La persistance sépare la vue instantanée de la mémoire longue du système. Cette répartition convient à une application de supervision. Elle permet de consulter rapidement l’état courant tout en conservant l’historique nécessaire au diagnostic, aux alertes et à l’audit.

4.5 Séquence de supervision temps réel

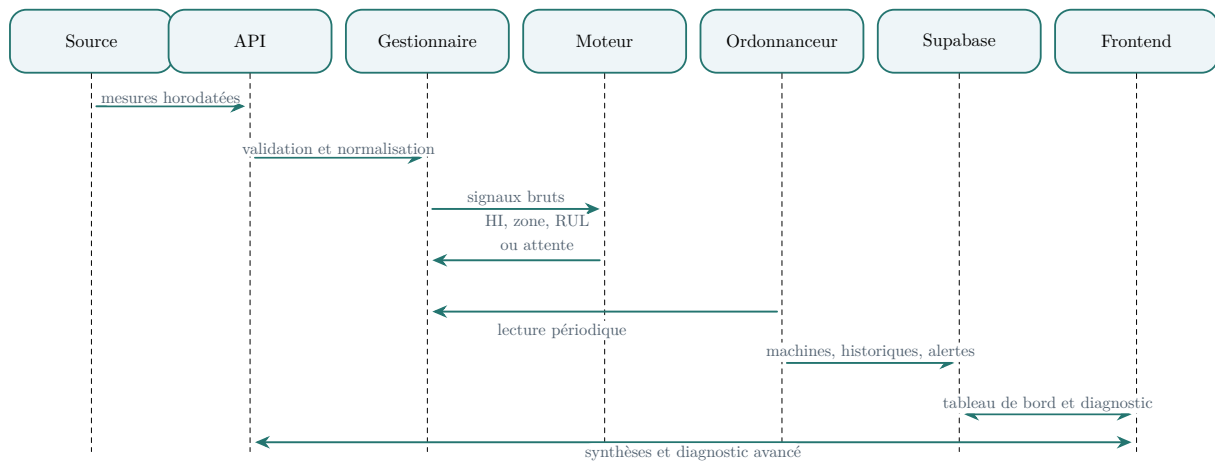


Figure 4.3 – Séquence simplifiée de la supervision temps réel

La mesure ne va jamais directement vers l'interface. Elle est d'abord validée, transformée et interprétée par le coeur runtime. L'ordonnanceur transforme ensuite cet état en historiques, en alertes et en données persistées.

4.6 Conception de l'interface et des droits

Table 4.2 – Organisation fonctionnelle de l'interface

Espace	Finalité	Accès
Tableau de bord	Lecture globale de la flotte, du HI, du RUL et des décisions courantes	Administrateur et utilisateur
Diagnostic avancé	Lecture détaillée des signaux, des explications et des recommandations	Administrateur et utilisateur
Maintenance et calendrier	Suivi des tâches, des dates planifiées et des fenêtres d'intervention	Administrateur et utilisateur
Alertes	Consultation et acquittement des alertes machine	Administrateur et utilisateur
Vue géographique et synthèses	Restitution synthétique et lecture multi-site	Administrateur et utilisateur
Gestion des machines, coûts, seuils, simulateur, expérience et administration	Paramétrage global, démonstration et gouvernance de la plateforme	Administrateur seulement

Une seconde règle est importante. Lorsqu'un utilisateur n'est pas administrateur, ses données sont limitées à la machine qui lui est affectée. Cette contrainte est cohérente avec l'usage métier et renforce la sécurité de lecture.

4.7 Conclusion

La conception de PrediTeq repose sur une architecture modulaire et lisible. La chaîne hors ligne produit les artefacts, le backend les exploite en temps réel, la base conserve l'état et l'interface restitue la décision. Le coeur runtime est organisé autour d'un tampon de caractéristiques, d'un moteur de pronostic, d'un gestionnaire d'exécution et d'un ordonnanceur périodique. Le chapitre suivant présente la réalisation complète du projet, depuis le pipeline scientifique jusqu'à la plateforme de supervision.

Chapitre 5

Réalisation

5.1 Introduction

Ce chapitre présente la réalisation de PrediTeq dans sa forme complète. Il rassemble le noyau scientifique, le service de supervision, l'interface web, les essais fonctionnels et l'ouverture industrielle.

La même logique doit convaincre deux lectures. Un non-spécialiste doit comprendre pourquoi chaque bloc existe. Un spécialiste doit retrouver des choix rigoureux, des métriques vérifiées et des limites clairement formulées.

5.2 Environnement matériel et logiciel

5.2.1 Environnement de réalisation

Table 5.1 – Environnement de réalisation

Bloc	Technologies visibles	Rôle principal
Simulation et apprentissage	 NumPy  Pandas  Scikit-learn  Joblib  Matplotlib  Seaborn	Générer les trajectoires, construire les variables, entraîner les modèles, produire les métriques et exporter les artefacts
Service applicatif temps réel	 FastAPI  Uvicorn  APScheduler  gmqtt  Pydantic  Supabase	Charger les artefacts, recevoir les mesures, calculer l'état machine, persister les historiques et piloter les alertes
Plateforme web	 React  Vite  React Router  React Query  Tailwind  Recharts	Restituer l'état de la flotte, le diagnostic, les alertes, la planification et l'administration
Acquisition expérimentale	 ESP32  ACS712  MPU6050	Montrer une acquisition courte et visible de courant et de vibration
Messagerie et intégration	 MQTT  HTTP	Recevoir un flux externe, le normaliser et l'injecter dans la chaîne temps réel

Cet environnement traduit une idée simple. La valeur du projet ne vient pas d'un modèle isolé. Elle vient d'une chaîne complète qui passe de l'apprentissage hors ligne à un produit de supervision exploitable.

5.3 Réalisation du noyau scientifique

5.3.1 Base physique et simulation

Le projet prend pour référence un moteur SITI FC100L1-4 utilisé pour l'entraînement d'un stockeur vertical rotatif. Les constantes retenues donnent un cadre industriel plausible [5]. Elles servent à relier la charge, l'effort électrique, l'échauffement et la dégradation.

Table 5.2 – Constantes de référence

Paramètre	Valeur	Intérêt pour le projet
Puissance nominale	2,2 kW	Cadre la lecture de la puissance et de l'effort
Vitesse nominale	1410 tr/min	Replace l'étude dans un contexte moteur industriel plausible
Tension nominale	400 V	Stabilise la lecture du courant et du facteur de puissance
Facteur de puissance	0,80	Intervient dans la logique de charge et d'effort électrique
Courant nominal	4,85 A	Fournit un repère de lecture pour la consommation du moteur
Nombre de niveaux	19	Structure la logique de charge du stockeur
Charge maximale par niveau	15 kg	Sert à construire les cas de charge
Charge totale maximale	285 kg	Définit la borne haute de la simulation

La simulation a été retenue pour une raison pratique. Le projet ne disposait pas d'un historique long, annoté et instrumenté de défaillances réelles. Une base physiquement guidée permet donc de commencer l'apprentissage sans attendre plusieurs années de mesures.

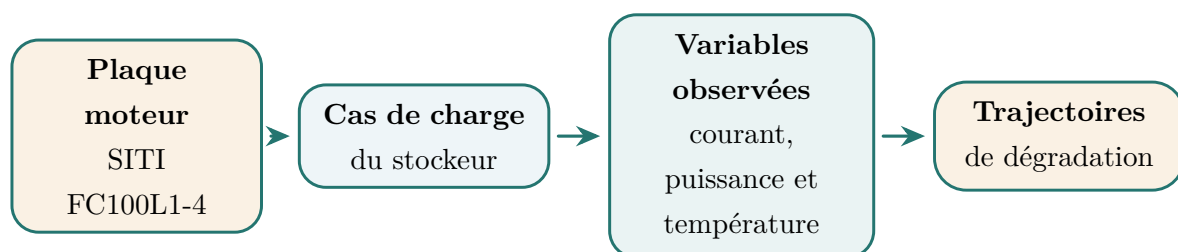


Figure 5.1 – Base physique de la simulation

Le simulateur hors ligne génère actuellement **200 trajectoires**. Ce point est important, car il correspond à l'état réel du pipeline vérifié dans les artefacts du projet.

Table 5.3 – Paramètres de simulation

Paramètre	Valeur	Pourquoi ce choix
Nombre de trajectoires	200	Compromis entre diversité et coût de calcul
Profils de dégradation	4	Couvrir plusieurs formes de vieillissement
Longueur nominale	800 min-sim	Couvrir la dégradation et les zones critiques
Cas de charge	20	Représenter toute la plage utile de 0 à 285 kg
Cycle de fonctionnement	44 s	Reprendre un rythme plausible pour le stockeur
Fréquence d'échantillonnage	1 Hz	Assez fine pour la dynamique, assez simple pour le pipeline

Les quatre profils couvrent une usure régulière, une usure accélérée en fin de vie, une dégradation par paliers et une usure plus bruitée. Cette diversité évite qu'un modèle n'apprenne un seul scénario idéalisé.

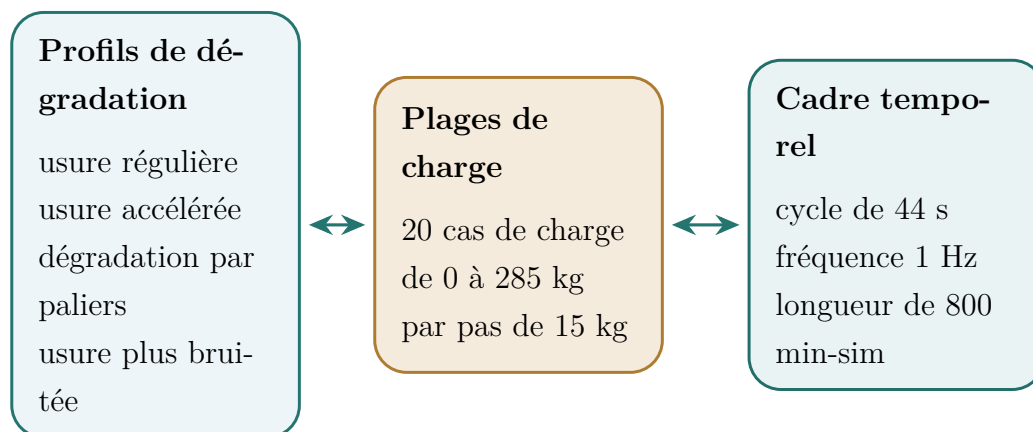


Figure 5.2 – Profils et charges

5.3.2 Ordre de la chaîne d'apprentissage

La chaîne suit la séquence vérifiée dans l'inventaire du projet [15]. Cet ordre est important. Il sépare clairement la simulation, l'extraction des variables, le diagnostic, le pronostic, l'évaluation et l'export runtime.

Table 5.4 – Étapes de la chaîne d'apprentissage

Étape	Bloc	Rôle
1	Simulation de dégradation	Générer les trajectoires synthétiques du moteur
2	Prétraitement des signaux	Transformer les mesures en variables exploitables
3	Détection d'anomalies	Produire un score d'écart au comportement sain
4	Construction du HI	Calculer l'indice de santé à partir des signaux observables
5	Modèle de RUL	Entraîner le régresseur de durée de vie résiduelle
6	Évaluation interne	Produire les métriques et les figures principales
6B	Validation externe	Vérifier la logique sur le benchmark NASA CMAPSS
6C	Calibration	Évaluer la confiance des intervalles
7	Export runtime	Préparer les artefacts rechargés par le backend

Les valeurs retenues dans ce mémoire proviennent des artefacts exportés les plus récents [16, 17, 18, 19, 20]. Cette règle évite d'introduire des chiffres moins fiables que les sorties générées par la chaîne elle-même.

5.3.3 Prétraitement et variables

Le prétraitement transforme les signaux bruts en variables plus stables. Cette étape est décisive, car un bon pronostic commence par une lecture cohérente du comportement de la machine.

Le pipeline exploite plusieurs fenêtres glissantes, notamment 60 secondes, 300 secondes et 3600 secondes. La vibration réagit vite. La température et certaines corrélations utiles évoluent plus lentement. Il faut donc observer plusieurs échelles de temps à la fois.

Table 5.5 – Variables principales extraites par le pipeline

Famille	Variable suivie	Interprétation
Vibration	Niveau vibratoire courant	Intensité mécanique observée à l'instant présent
Vibration	Vitesse de variation vibratoire	Rapidité de montée ou de baisse de la vibration
Vibration	Variabilité vibratoire récente	Stabilité ou instabilité de la vibration sur fenêtre courte

Famille	Variable suivie	Interprétation
Puissance	Puissance moyenne récente	Niveau moyen d'effort électrique
Puissance	Puissance quadratique moyenne	Intensité énergétique récente
Puissance	Vitesse de variation de la puissance	Rapidité de changement de charge électrique
Cycle	Énergie consommée par cycle	Dépense énergétique associée au fonctionnement
Cycle	Allongement du cycle	Écart entre cycle normal et cycle ralenti
Température	Température moyenne récente	Niveau thermique moyen observé
Température	Vitesse de variation thermique	Rapidité de montée ou de baisse de la température
Humidité	Variabilité hygrométrique	Stabilité ou variation de l'environnement
Croisée	Corrélation température-puissance	Lien entre échauffement et effort électrique sur fenêtre longue

Le modèle de pronostic ne se limite pas à ces douze variables. L'artefact d'évaluation confirme qu'il exploite au total **17 caractéristiques**. Il combine ces variables capteurs avec des descripteurs de dérive de l'indice de santé [17].

5.3.4 Détection d'anomalies et indice de santé

Le premier besoin est de dire si le comportement s'éloigne d'une zone saine. L'Isolation Forest répond à cette question en apprenant surtout la normalité, puis en signalant les écarts [2].

Les métriques du détecteur seul sont les suivantes [16] :

- précision : 0,521 ;
- rappel : 1,000 ;
- F1 : 0,685.

Cette lecture est volontairement sensible. Elle repère bien les dérives, mais elle reste trop large si elle est utilisée seule.

Le projet conserve donc un repère vibratoire direct. La combinaison du score multivarié et de ce repère produit les résultats suivants [16] :

- lecture vibratoire seule : précision 0,852, rappel 1,000, F1 0,920 ;

— ensemble hybride : précision 0,971, rappel 0,958, F1 0,964.

Le score hybride donne ainsi le meilleur compromis entre rigueur et lisibilité. Il reste proche de la physique observée, tout en profitant d'une lecture plus riche des signaux.

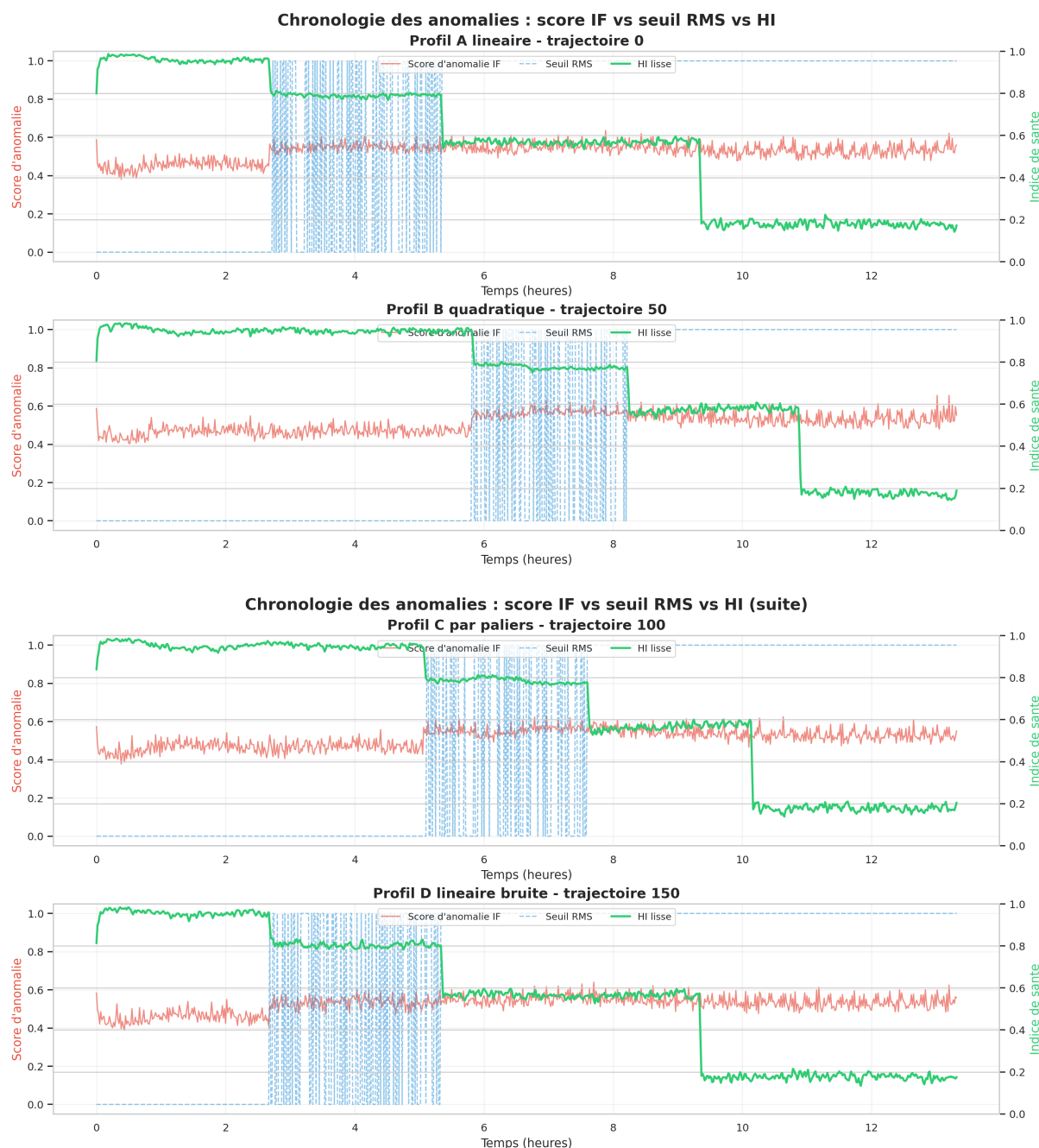


Figure 5.3 – Chronologie des anomalies

L'indice de santé, ou *Health Index* (HI), n'est pas une variable cachée livrée telle quelle au produit. Il est reconstruit à partir des signaux et du score d'anomalie, puis lissé dans le temps afin qu'un point isolé ne provoque pas une lecture brutale.

Trois seuils structurent ensuite l'interprétation :

— 0,8 pour la zone saine ;

- 0,6 pour la zone correcte à surveiller ;
- 0,3 pour la zone critique.

Le seuil de 0,3 joue un rôle central. Il marque la zone critique et sert aussi à construire la cible du pronostic.

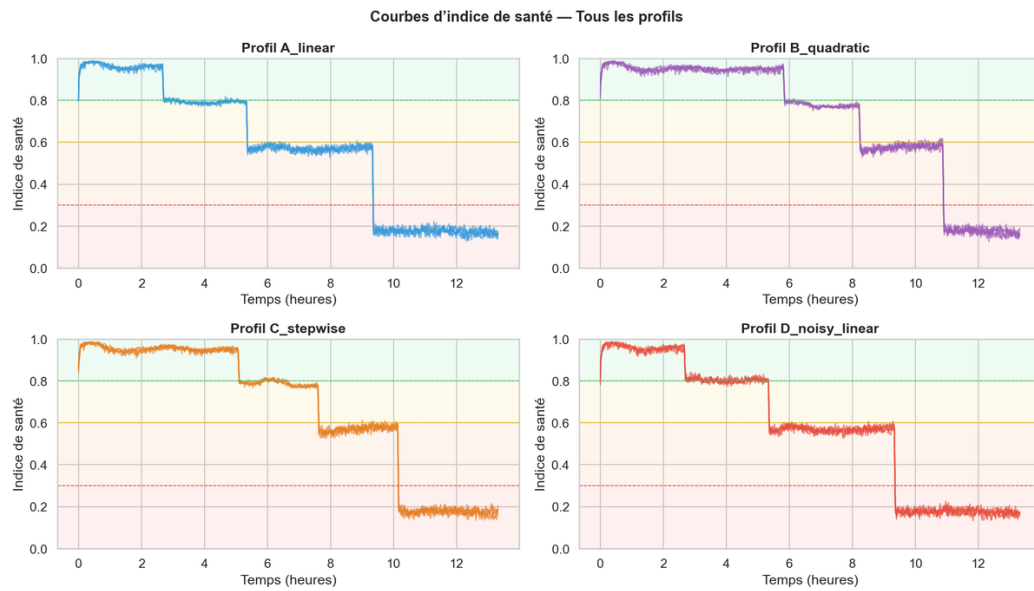


Figure 5.4 – Courbes d'indice de santé

5.3.5 Pronostic de durée de vie résiduelle

Le pronostic doit rester utile en exploitation, et non seulement en laboratoire. La cible finale de durée de vie résiduelle, ou *Remaining Useful Life* (RUL), est donc construite à partir d'un signal de santé observable et lissé, puis d'un franchissement persistant du seuil critique sur trois points consécutifs.

Ce choix a une conséquence importante. Le modèle n'apprend pas directement sur une vérité cachée inaccessible en exploitation. Il apprend sur ce que le futur service temps réel pourra réellement observer.

Le modèle de RUL repose sur une forêt aléatoire de 300 arbres, avec une profondeur maximale de 16 et une taille minimale de feuille de 10 [7]. Le nombre d'arbres améliore la stabilité. La profondeur et la taille minimale de feuille limitent le sur-apprentissage.

Table 5.6 – Paramétrage du pronostic

Choix	Valeur	Rôle
Historique HI requis	60 points	Une heure de contexte avant publication du RUL
Variables d'entrée	17	12 caractéristiques capteurs et 5 descripteurs du HI
Algorithme	Forêt aléatoire	Modèle non linéaire robuste pour le pronostic
Nombre d'arbres	300	Compromis retenu après convergence de la courbe hors sac
Profondeur maximale	16	Régularisation du modèle
Feuille minimale	10	Limitation du sur-apprentissage
Séparation des données	80/20 par profil	160 trajectoires d'entraînement et 40 de test
Validation croisée	GroupKFold, $k = 5$	Protection contre la fuite entre trajectoires

Les trajectoires d'entraînement et de test ont donc été séparées par trajectoire complète. La validation croisée par groupes évite ainsi qu'un modèle reconnaisse simplement une trajectoire déjà vue [8]. Pour un spécialiste, c'est un garde-fou méthodologique. Pour l'exploitant, c'est une manière de dire que le modèle ne triche pas.

Table 5.7 – Résultats principaux du pronostic

Bloc	R^2	Erreur	Lecture
Test séparé	0,980	RMSE 2,49 jours ; MAE 1,65 jour	Le modèle suit fortement la variabilité du temps restant
Validation croisée par groupes	0,982	RMSE moyen 2,32 jours	La performance reste stable d'un pli à l'autre
Régression linéaire	0,941	RMSE 4,23 jours	La solution finale apporte un gain net
Référence moyenne	~ 0	RMSE 17,43 jours	Une baseline naïve reste très éloignée
Capteurs seuls	0,967	–	Les signaux portent déjà une forte information prédictive
Modèle complet	0,980	–	L'indice de santé améliore encore la qualité du pronostic

Le test séparé actuel exploite 160 trajectoires dans l'apprentissage et 40 trajectoires dans le test. L'artefact détaillé indique 85 140 échantillons d'apprentissage et 21 388 échantillons de test [17].

L'étude d'ablation confirme ensuite le rôle de l'indice de santé. Le modèle complet à 17 variables atteint $R^2 = 0,980$. La version limitée aux seules variables capteurs tombe à $R^2 = 0,967$. Le HI n'est donc pas seulement un indicateur d'affichage. Il apporte une information utile au pronostic.

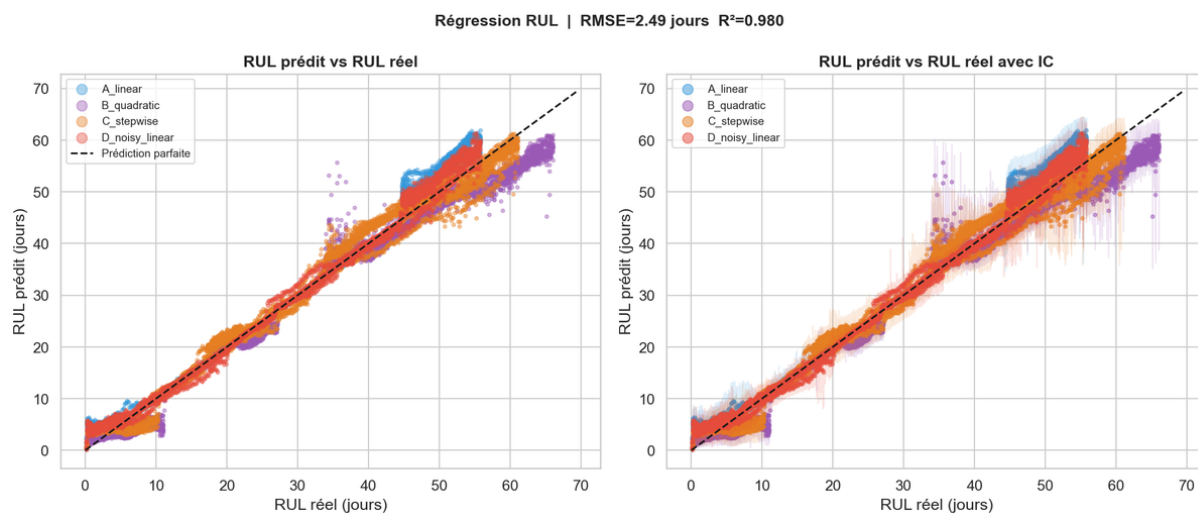


Figure 5.5 – RUL réelle et RUL prédite

5.3.6 Validation scientifique complémentaire

Un bon score ne suffit pas. Il faut aussi comprendre pourquoi le modèle réagit, tester sa stabilité et vérifier sa portée hors du cas interne.

Le projet produit d'abord un résumé d'explications additives de Shapley (SHAP). Cette lecture montre quelles variables poussent le pronostic vers un temps restant plus court ou plus long. Elle aide donc à justifier la décision, au lieu de seulement l'afficher.

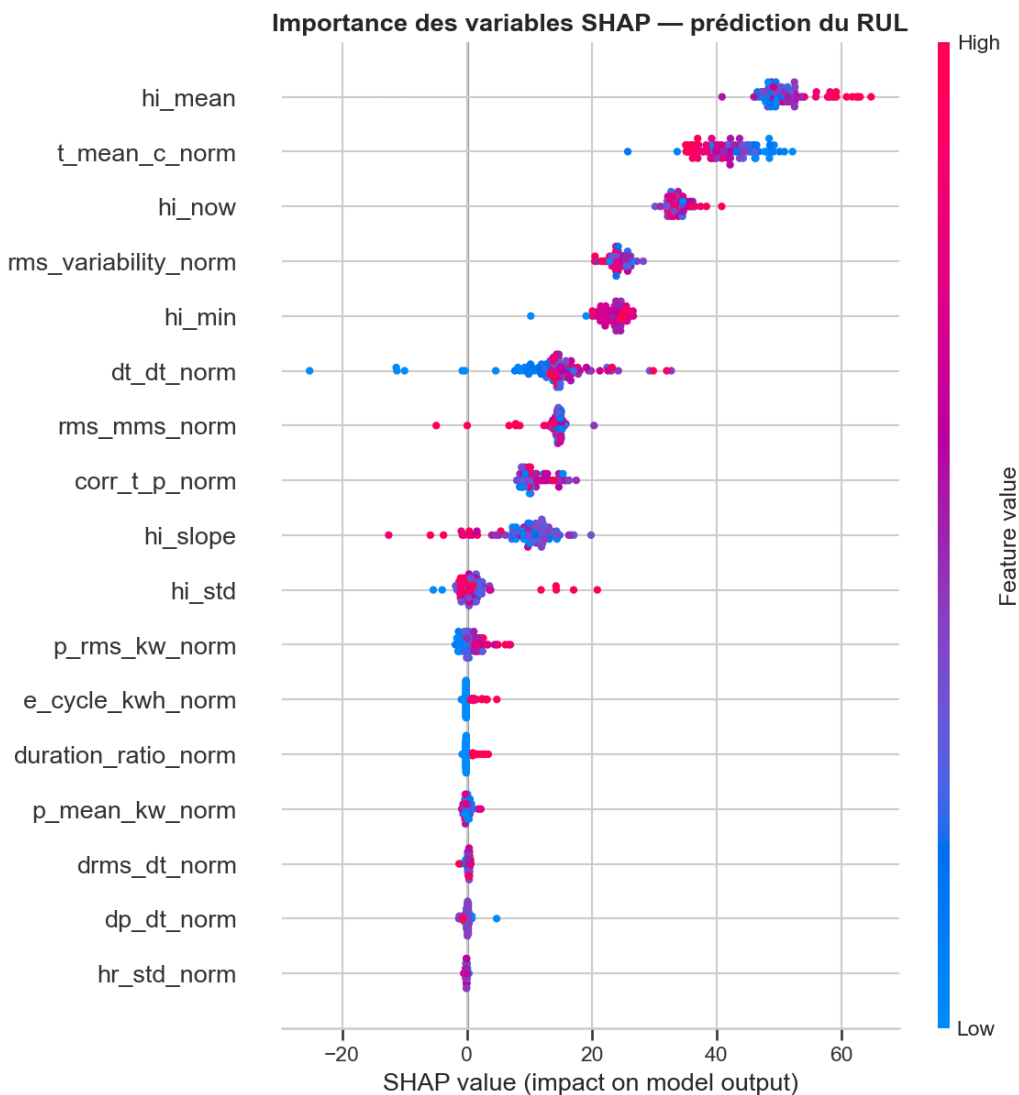


Figure 5.6 – Résumé d'explicabilité

Le projet ajoute aussi une carte de sensibilité des réglages du détecteur. Cette vue ne cherche pas à multiplier les figures. Elle vérifie que la qualité du diagnostic ne s'effondre pas pour une petite variation des paramètres.

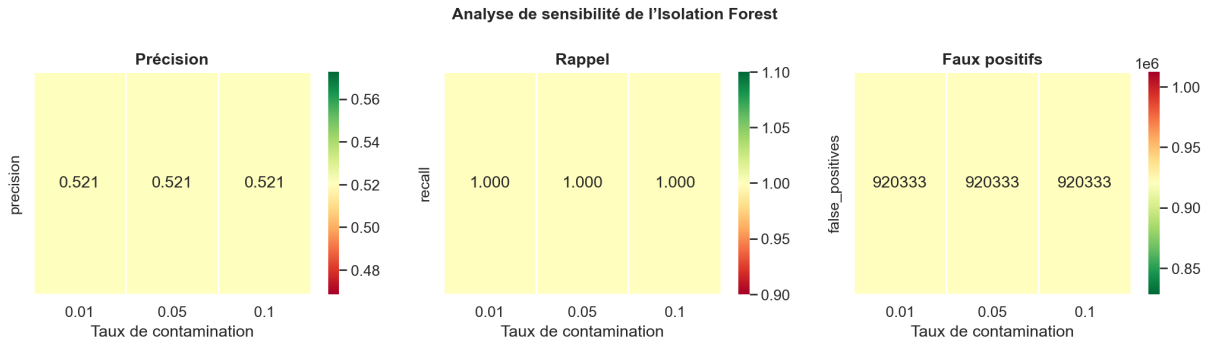


Figure 5.7 – Sensibilité des réglages

La validation externe sur NASA CMAPSS, ou *Commercial Modular Aero-Propulsion System Simulation* (CMAPSS), ne dit pas que le stockeur et un moteur aéronautique sont identiques. Elle vérifie plus simplement que la logique du pipeline reste utile hors de la seule simulation interne [6].

Les métriques exportées sont les suivantes [18] :

- R^2 : 0,886 ;
- RMSE : 14,11 cycles ;
- MAE : 9,64 cycles.

Ces résultats dépassent les objectifs de référence associés à cette étape. Ils renforcent donc la crédibilité scientifique du projet.

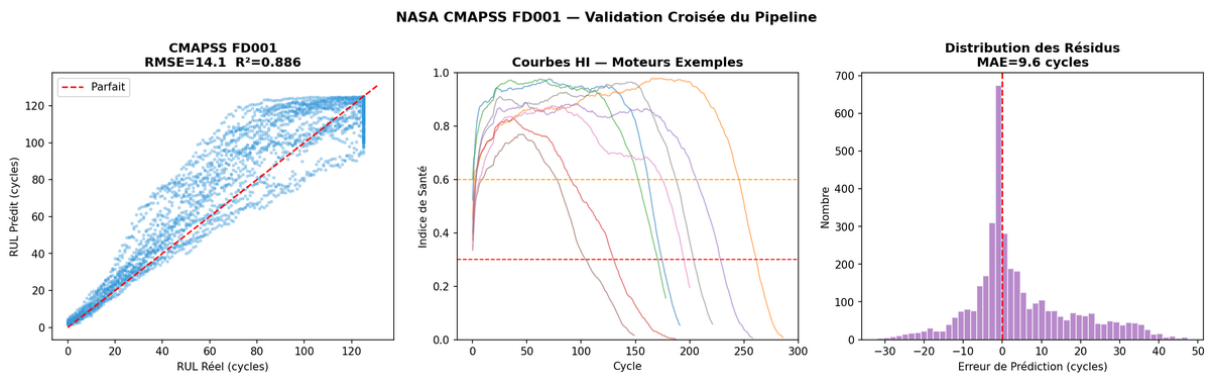


Figure 5.8 – Validation NASA CMAPSS

La calibration des intervalles répond à une autre question. Un intervalle utile n'est pas seulement un intervalle étroit. C'est un intervalle honnête, dont le niveau de confiance correspond à ce qu'il annonce.

Les valeurs les plus utiles sont les suivantes [19, 20] :

- couverture native de l'intervalle à 80 % : 0,872 ;
- couverture empirique au niveau 80 % : 0,881 ;
- erreur de calibration attendue : 0,088 ;

- biais moyen : +1,05 jour ;
- temps d'avance utile moyen : 564,9 minutes-simulation, calculé sur 194 trajectoires analysées sur 200.

Ces chiffres traduisent un modèle légèrement prudent. Dans un contexte de maintenance, cette prudence reste acceptable, car elle laisse en moyenne une fenêtre d'action avant la zone critique.

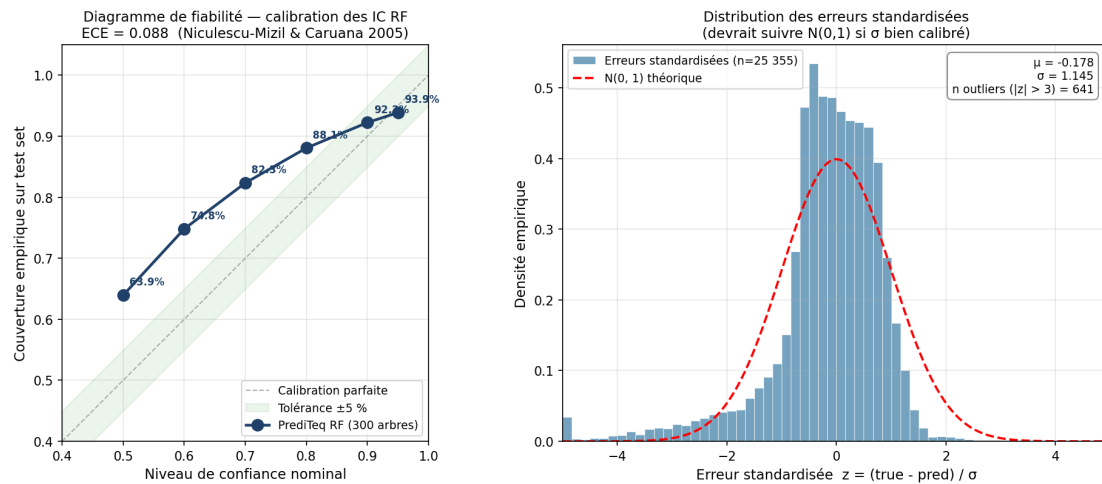


Figure 5.9 — Calibration des intervalles

5.4 Réalisation de la plateforme de supervision

5.4.1 Positionnement produit

PrediTeq n'est pas une interface ajoutée autour d'un modèle. C'est une plateforme logicielle en mode service, ou *Software as a Service* (SaaS), qui relie le diagnostic, le pronostic, la restitution métier et la gouvernance applicative.

La logique d'usage suit une séquence simple : observer, comprendre, décider, planifier, puis agir. Cette progression donne au produit une lecture claire pour l'exploitant comme pour le responsable de maintenance.

Table 5.8 – Parcours utilisateur principal

Étape	Question utilisateur	Espaces support
Voir	Quelles machines demandent une attention immédiate ?	Tableau de bord, vue flotte, indicateurs rapides et centre d'alertes
Comprendre	Pourquoi cette machine est-elle en zone de risque ?	Diagnostic avancé, historiques, lecture du HI, du stress et du RUL
Décider	Faut-il surveiller, intervenir ou reconfigurer ?	Espaces d'analyse, synthèse assistée et gestion des seuils
Planifier	Quand agir et avec quel impact ?	Calendrier, tâches de maintenance, priorisation et budget
Agir	Comment démontrer, configurer ou alimenter le système ?	Simulateur, page Experiment, gestion des machines et administration

Table 5.9 – Pages principales de la plateforme

Page	Usage principal
Tableau de bord	Vue de flotte, état machine, résumé rapide et priorités
Diagnostic avancé	Lecture détaillée des indicateurs et aide à l'interprétation
Synthèse et planification	Recommandation, priorisation et préparation des actions
Calendrier de maintenance	Projection des interventions dans le temps
Coûts et budget	Lecture économique du risque et des actions prévues
Centre d'alertes	Regroupement et priorisation des alertes
Gestion des machines	Inventaire, affectations et contexte flotte
Administration et seuils	Comptes, droits, seuils et gouvernance
Simulateur	Pilotage de la démonstration à trois machines
Experiment	Acquisition live centrée sur vibration et courant

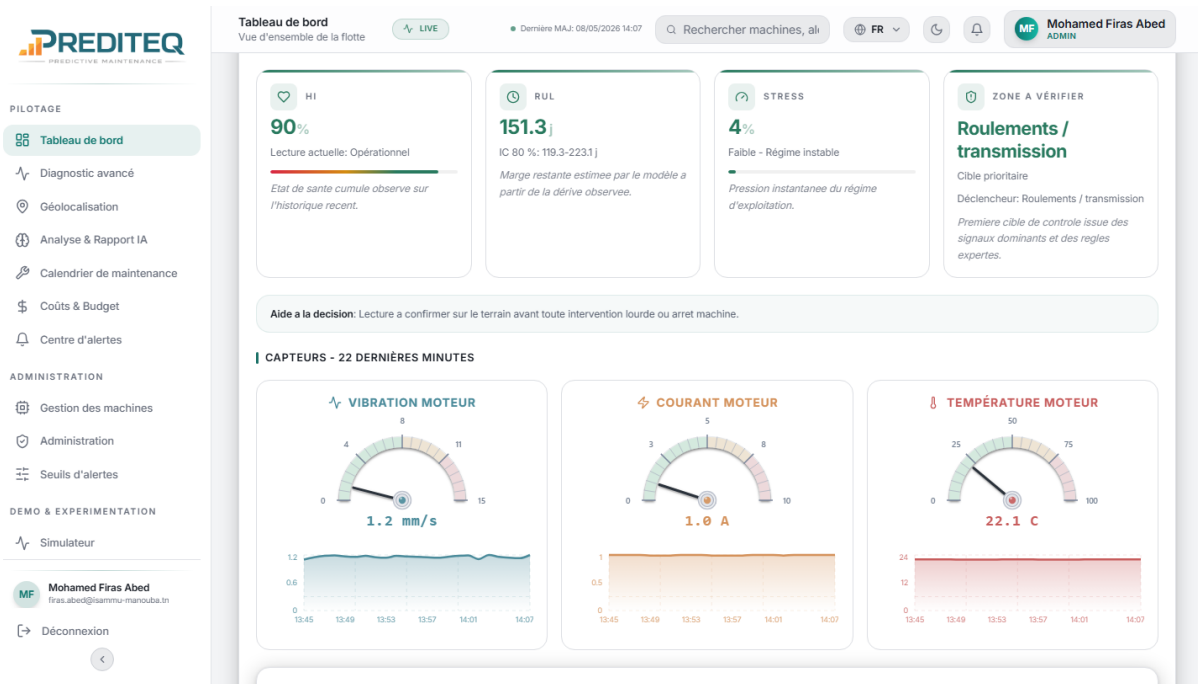


Figure 5.10 – Tableau de bord PrediTeq

Le tableau de bord joue le rôle de point d'entrée. Il permet de distinguer rapidement les machines saines, surveillées et critiques. Le diagnostic avancé complète ensuite cette vue par des indicateurs plus détaillés et par une lecture causale du risque.

5.4.2 Service applicatif temps réel

Le service applicatif charge les artefacts au démarrage, crée un moteur d'inférence par machine, relit les machines depuis la base, tente une connexion MQTT, lit les seuils terrain et démarre l'ordonnanceur. Cette séquence montre que PrediTeq est conçu comme un service complet, et non comme un simple démonstrateur.

L'acquisition live est prise en charge par deux canaux. Le premier est un canal MQTT. Le second est une route HTTP protégée par jeton d'ingestion. Dans les deux cas, les mesures sont ramenées à une forme commune, validées, puis injectées dans le gestionnaire runtime.

Le coeur applicatif calcule ensuite le HI, la zone, le contexte de pronostic et les indicateurs de diagnostic. L'ordonnanceur relit régulièrement l'état courant, met à jour la table des machines, insère les historiques, déclenche les alertes et envoie les notifications.

Une nuance importante doit rester visible. Les seuils utilisés en exploitation ne sont pas toujours strictement ceux du pipeline hors ligne. Le service temps réel peut utiliser des valeurs paramétrables stockées dans la base. Cette souplesse permet d'ajuster la lecture opérationnelle sans réentraîner toute la chaîne scientifique.

Le service intègre aussi une couche de gouvernance cohérente avec l'usage métier. Il gère l'authentification, les rôles, les affectations par machine, les traces d'audit et une

partie des notifications. La plateforme ne se contente donc pas d’afficher un score. Elle encadre aussi son usage [9, 10].

5.4.3 Simulateur et démonstration guidée

Le simulateur occupe une place centrale dans la démonstration produit. Il rejoue trois scénarios machine explicites, puis injecte ces signaux dans le moteur runtime [21]. La lecture publique reste ainsi simple, stable et cohérente.

Table 5.10 – Scénarios machine de démonstration

Scénario	État cible	Charge et usage	HI cible	Lecture produit
Cas sain	Opérationnelle	Charges légères à moyennes, usage modéré	0,96	Machine saine, lecture calme et stable
Cas de surveillance	Sous surveillance	Trafic mixte, demi-charges et pics modérés	0,62	Machine à suivre, dérive visible et action à préparer
Cas critique	Critique	Charges lourdes, ambiance sévère et forte intensité	0,10	Machine critique, alerte prioritaire et intervention rapprochée

Ces trois cas sont volontairement contrastés. Plus l’usage est sévère, plus la charge est forte et plus l’indice de santé visé est bas. La démonstration reste donc lisible sans devenir arbitraire.

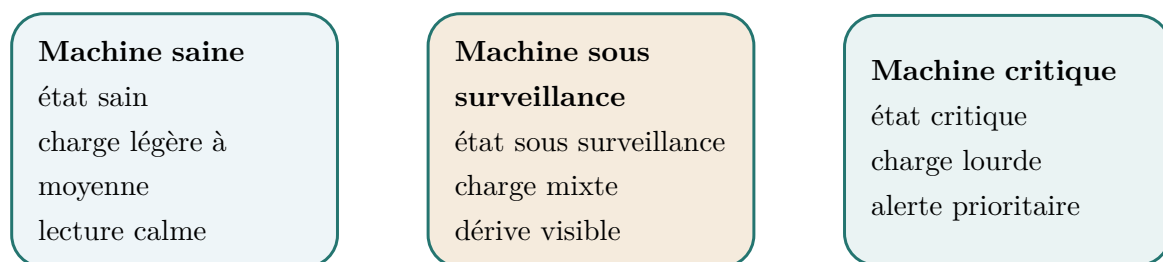


Figure 5.11 – Structure du simulateur

Le simulateur ne doit toutefois pas être confondu avec un essai industriel long. Il injecte

bien une télémétrie dans le moteur runtime, mais certaines valeurs d’affichage peuvent être bornées ou ajustées pour garder une narration stable pendant la soutenance. Cette précision ne fragilise pas le projet. Elle clarifie simplement son niveau réel de maturité.

5.5 Validation fonctionnelle, discussion critique et ouverture industrielle

5.5.1 Banc d’essai expérimental

Le projet comprend aussi une démonstration d’acquisition physique autour de deux capteurs simples : un capteur de courant ACS712 5A et un capteur inertiel MPU6050. Ce banc a un objectif clair. Il montre que l’application sait recevoir un flux live court, afficher des jauges cohérentes et relier le produit à un montage visible.

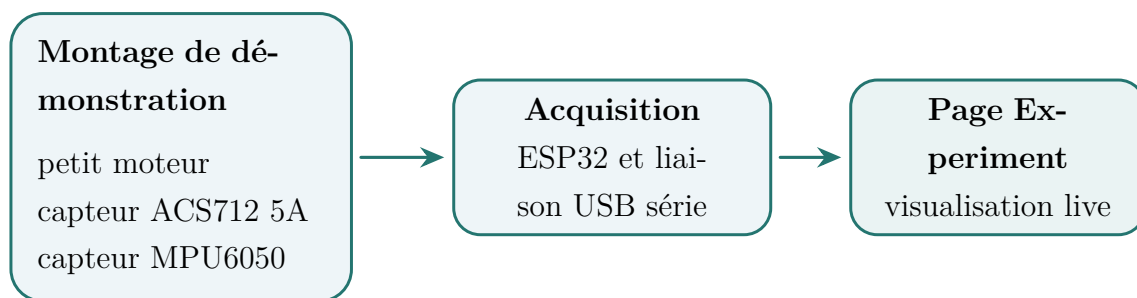


Figure 5.12 – Banc d’essai expérimental

5.5.2 Page Experiment

La page Experiment répond à une question concrète : l’application peut-elle afficher un flux live simple issu d’un montage physique visible [22] ? La réponse est oui, mais dans un cadre de démonstration fonctionnelle et non encore d’industrialisation complète.

La démonstration suit cinq phases de 7 secondes. Cette durée a été retenue pour deux raisons. Elle laisse le temps aux jauges et aux courbes de se stabiliser visuellement. Elle garde aussi une séquence complète suffisamment courte pour une démonstration de soutenance.

1. moteur arrêté ;
2. moteur en marche sans charge ;
3. moteur arrêté ;
4. moteur en marche avec charge ;
5. moteur arrêté.

Les valeurs visibles ont été réglées pour rester cohérentes avec le petit moteur utilisé :

- courant libre autour de 0,092 A ;
- courant en charge autour de 0,225 A ;
- vibration libre autour de 0,19 ;
- vibration en charge autour de 0,39 ;
- seuil courant visible à 0,16 A ;
- seuil vibration visible à 0,34.

Ces valeurs ne cherchent pas à reproduire un moteur industriel. Elles sont ajustées sur le banc pour garder une séparation nette entre le fonctionnement libre et le fonctionnement en charge. Les seuils de 0,16 A et de 0,34 sont donc placés entre les deux régimes observés. Ils rendent le franchissement lisible sans provoquer d'alerte permanente lorsque le moteur tourne à vide.

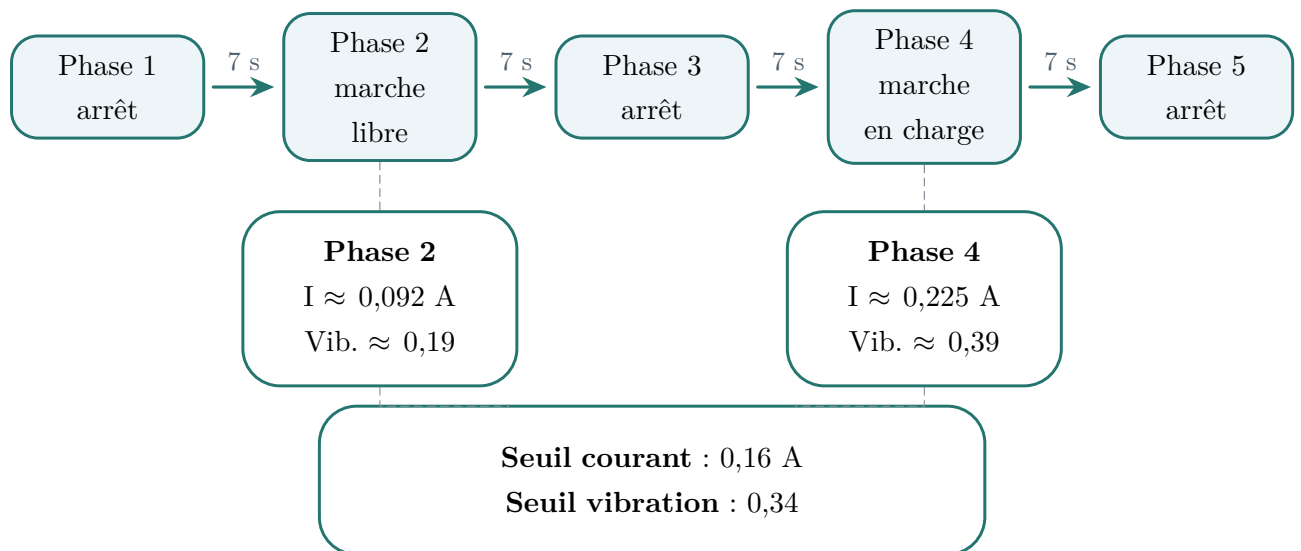


Figure 5.13 – Scénario Experiment

Cette page constitue donc un test fonctionnel d'acquisition dans l'application. Elle montre que le produit sait recevoir, afficher et synchroniser un flux simple de courant et de vibration. Elle ne remplace pas encore le contrat live complet attendu pour une intégration industrielle durable.

5.5.3 Validation de bout en bout

Une première chaîne live de bout en bout a été préparée et vérifiée fonctionnellement à l'aide de deux postes. Le premier héberge le backend PrediTeq et l'interface web. Le second joue le rôle d'une source externe et publie des mesures simulées via MQTT. Cette configuration a été retenue pour prouver le schéma source externe → transport → backend → application avant tout raccordement industriel réel.

Les captures conservées à la figure 5.14 illustrent trois moments de cette chaîne : l'émission côté source, la réception côté backend et la restitution applicative sur la machine de référence affichée dans l'interface. Elles doivent être lues comme une preuve de faisabilité fonctionnelle de la chaîne live, et non comme une validation terrain finale.

Cette étape ne remplace pas encore la qualification sur données réelles directement issues de la machine industrielle. Elle confirme en revanche que l'architecture de transport, d'ingestion et d'affichage fonctionne déjà de bout en bout sur un essai fonctionnel maîtrisé.

5.5.4 Maturité actuelle

Table 5.11 – Maturité actuelle des briques PrediTeq

Brique	Statut	Preuve principale	Limite actuelle	Étape suivante
Simulation du moteur	Validée	200 trajectoires, cohérence physique et figures exportées	Pas encore confrontée à un historique terrain long	Comparer progressivement les hypothèses à des signaux réels instrumentés
Pipeline HI et RUL	Validé	Test séparé, validation croisée, calibration et benchmark NASA	Données d'apprentissage encore majoritairement synthétiques	Raffiner avec télémétrie réelle et cas labellisés
Service applicatif temps réel	Démontré	Artefacts chargés, routes métier, moteur runtime, seuils terrain et essai live à deux postes	Le contrat terrain d'une première machine réelle reste à stabiliser	Qualifier la chaîne sur CSV réel fourni par l'industriel

Brique	Statut	Preuve principale	Limite actuelle	Étape suivante
Plateforme web	Démontrée	Tableau de bord, diagnostic, alertes, planification et administration	Certaines vues gardent une logique de démonstration guidée	Consolider les parcours avec davantage d'usage terrain
Simulateur à trois machines	Démontré	Scénarios contrastés et zones cohérentes	Ne remplace pas l'observation continue d'une flotte réelle	Le conserver comme outil de démonstration et de vérification produit
Page Experiment	Démontrée	Test live autour de deux capteurs simples	Ne couvre pas encore tout le contrat live du backend	Relier la page à une source télémétrique plus riche
Intégration industrielle	Préparée et testée fonctionnellement	Pont CSV, essai live à deux postes, backend cloud et frontend public	Qualification terrain encore dépendante du contrat CSV réel	Confirmer chemin, en-tête, ligne type et mise à jour en marche

```
Starting PrediTeq CSV bridge...
Source locale -> normalisation CSV -> MQTT prive
Machine id: ARO-01
Broker securise en TLS 8883
Topic: prediteq/ARO-01/sensors
Transport: mqtt
Machine id: ARO-01
Target: prediteq/ARO-01/sensors
[2026-05-13T23:43:04.003535+00:00] sent prediteq/ARO-01/sensors rms=1.315 power=1.129 temp=27.0 hum=56.7
[2026-05-13T23:43:05.103632+00:00] sent prediteq/ARO-01/sensors rms=1.374 power=1.129 temp=27.0 hum=58.1
[2026-05-13T23:43:06.184581+00:00] sent prediteq/ARO-01/sensors rms=1.494 power=1.166 temp=26.7 hum=58.4
[2026-05-13T23:43:07.284845+00:00] sent prediteq/ARO-01/sensors rms=1.535 power=1.150 temp=26.3 hum=58.8
[2026-05-13T23:43:08.368216+00:00] sent prediteq/ARO-01/sensors rms=1.512 power=0.987 temp=26.2 hum=59.5
[2026-05-13T23:43:09.450089+00:00] sent prediteq/ARO-01/sensors rms=1.525 power=0.840 temp=25.6 hum=60.1
[2026-05-13T23:43:10.549100+00:00] sent prediteq/ARO-01/sensors rms=1.421 power=0.693 temp=25.1 hum=61.0
[2026-05-13T23:43:11.646307+00:00] sent prediteq/ARO-01/sensors rms=1.291 power=0.583 temp=24.8 hum=60.6
[2026-05-13T23:43:12.744875+00:00] sent prediteq/ARO-01/sensors rms=1.176 power=0.511 temp=24.5 hum=61.0
```

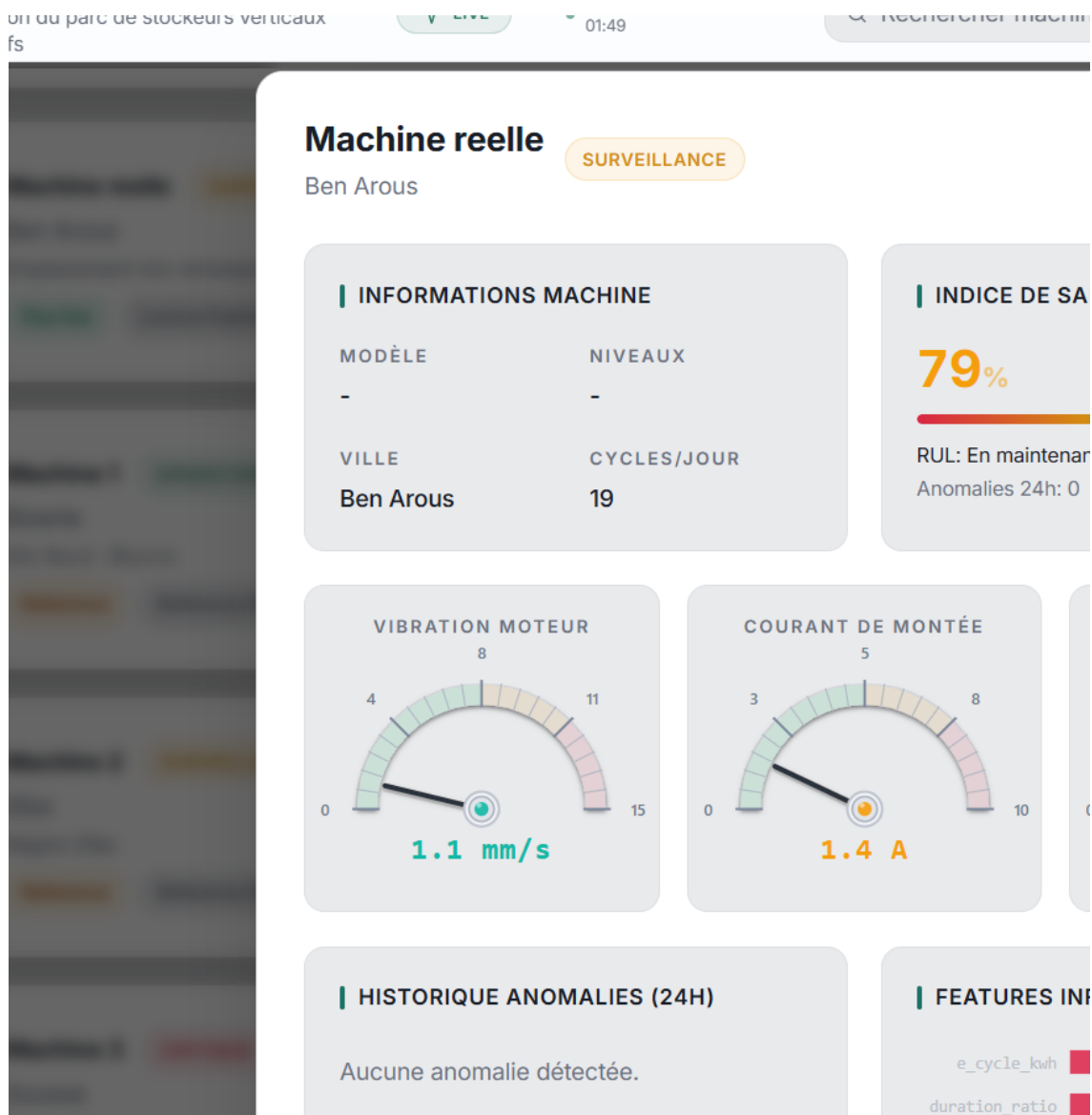
(a) Poste source

```
PS C:\workspace\prediteq_api>
python -m uvicorn main:app --reload
INFO: Will watch for changes in ['app']
INFO: Uvicorn running on http://127.0.0.1:8000
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [11856] using WatchFiles
INFO: Started server process [7888]
INFO: Waiting for application startup.
02:49:49 | INFO | prediteq | Starting PrediTeq API ...
02:49:49 | INFO | core.supabase.client | Supabase client initialized
02:49:49 | INFO | httpx | GET https://supabase-instance/rest/v1/machines ... "HTTP/2 200 OK"

ml\models
02:49:49 | INFO | ml.loader | Loaded isolation_forest.pkl
02:49:49 | INFO | ml.loader | Loaded random_forest_rul.pkl (300 trees)
02:49:49 | INFO | ml.loader | Loaded scaler_params.json (12 features)
02:49:49 | INFO | ml.loader | Loaded hi_params.json (p5=0.0522, p95=0.8631)
02:49:49 | INFO | ml.loader | Loaded hybrid_params.json (alpha=0.20)
02:49:49 | INFO | ml.engine_manager | EngineManager initialized
02:49:50 | INFO | ml.engine_manager | Registered 4 machines: ['ARO-01', 'ASC-A1', 'ASC-B2', 'ASC-C3']
02:49:50 | INFO | routers.mqtt | Dedicated bridge account loaded
02:49:50 | INFO | routers.mqtt | MQTT connected - subscribed to prediteq/+sensors
02:49:50 | INFO | routers.mqtt | MQTT connecting to broker TLS 8883 ...
```

(b) Réception backend

Figure 5.14 – Validation fonctionnelle de la chaîne live à deux postes



(c) Restitution applicative

Figure 5.14 – Validation fonctionnelle de la chaîne live à deux postes (suite)

5.5.5 Déploiement et reproductibilité

La pile de déploiement s'appuie sur un frontend public déployé sur Vercel, un backend cloud hébergé sur Render, une persistance Supabase et un broker MQTT dédié pour la chaîne live. Cette organisation rend la plateforme partageable et relançable [14].

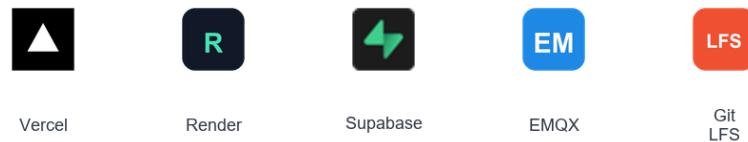


Figure 5.15 – Technologies de déploiement, de messagerie et de versionnement retenues.

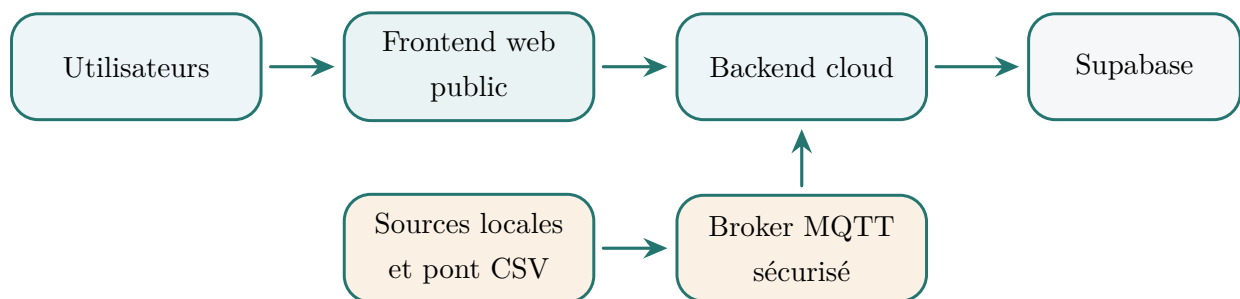


Figure 5.16 – Architecture de déploiement

Le projet reste reproductible parce qu'il sépare clairement l'apprentissage hors ligne, le service applicatif et l'interface. Il prévoit aussi des tests unitaires, des tests de bout en bout et des contrôles de syntaxe côté backend. Les détails de reprise sont regroupés en annexe.

Le modèle de pronostic occupe enfin un volume important. La gestion de ce type d'artefacts dépasse les limites habituelles d'un dépôt Git classique. Le recours à Git Large File Storage (Git LFS) permet donc de versionner proprement les fichiers volumineux sans rompre la chaîne de déploiement [12, 13].

5.5.6 Limites et trajectoire vers la donnée réelle

Quatre limites doivent être formulées avec clarté.

La première concerne la base d'apprentissage. Les données principales restent synthétiques, même si la simulation est physiquement guidée.

La deuxième concerne le simulateur. Il garantit une lecture stable des scénarios, mais il ne remplace pas l'observation continue d'une flotte réelle.

La troisième concerne la page Experiment. Elle démontre une acquisition live simple, mais elle ne couvre pas encore toute la richesse du contrat attendu par le backend industriel.

La quatrième concerne non plus le principe de la chaîne, mais sa qualification terrain. Une première chaîne live a déjà été préparée et vérifiée fonctionnellement. Ce qui manque encore pour une première machine réelle, c'est la fourniture, par l'industriel, du CSV attendu : chemin exact, en-tête confirmé, ligne d'exemple et confirmation que le fichier est bien mis à jour lorsque la machine fonctionne.

La poursuite la plus concrète consiste donc à brancher ce CSV réel sur la chaîne déjà en place, sans changer l'architecture produit : source locale, pont de normalisation, broker MQTT privé, backend cloud et frontend public. Une fois ce premier raccordement validé sur une machine de référence, le même schéma pourra être reproduit sur d'autres équipements.

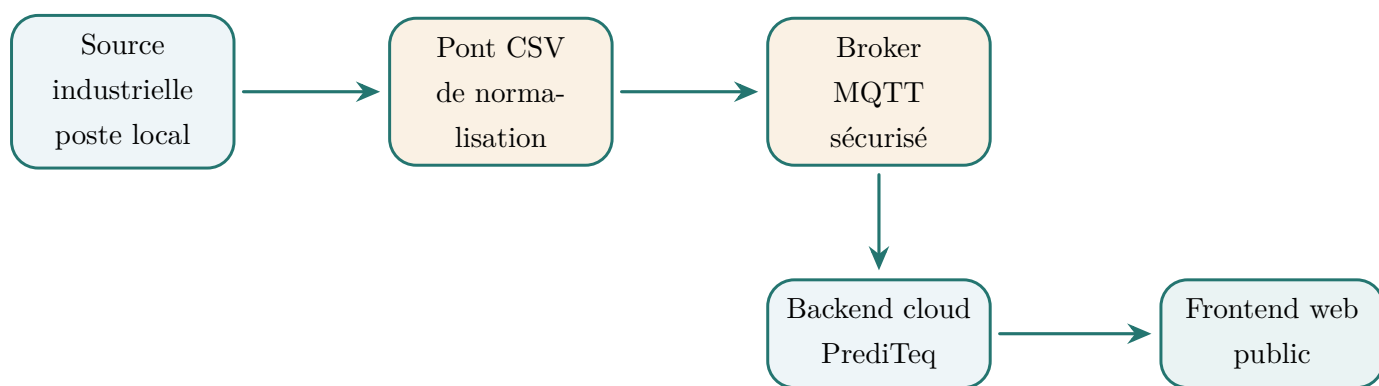


Figure 5.17 – Chaîne préparée d'intégration réelle

5.6 Conclusion

La réalisation de PrediTeq relie donc un noyau scientifique rigoureux, un service applicatif temps réel et une plateforme web exploitable. Le pipeline reste solide parce qu'il s'appuie sur une simulation physiquement guidée, une validation par groupes, une validation externe, une calibration des intervalles et une étude d'ablation explicite. Le produit reste lisible parce qu'il transforme ces sorties en diagnostic, en planification, en alertes et en démonstration cohérente. La conclusion générale peut maintenant synthétiser les acquis du projet, ses limites et ses prolongements naturels.

Conclusion générale et perspectives

PrediTeq a été conçu comme une plateforme complète de maintenance prédictive, et non comme un simple modèle isolé. Le travail réalisé montre qu'il est possible, dans le cadre d'un PFE MIME, de relier une logique physique de dégradation, un pipeline de Machine Learning, un backend temps réel, une interface de supervision et une trajectoire crédible vers la donnée réelle. Le moteur d'un stockeur vertical rotatif constitue ici le premier cas de référence. Il structure la simulation, le produit et la démonstration, sans enfermer l'ambition de la plateforme dans ce seul équipement.

Sur le plan scientifique, le projet atteint un niveau solide. Les artefacts vérifiés confirment une prédiction de durée de vie résiduelle robuste sur le protocole holdout, une validation croisée stable, une détection hybride performante et une validation externe convaincante sur NASA CMAPSS. Le point méthodologique le plus important reste la formulation du pronostic à partir d'un signal observable de santé. Cette construction rapproche l'apprentissage de ce qui sera réellement disponible en exploitation. La validation NASA CMAPSS ne démontre pas encore PrediTeq sur toute machine industrielle, mais elle apporte déjà un indice sérieux de transférabilité au-delà du seul cas interne.

Sur le plan produit, PrediTeq ne se limite pas à produire des métriques. La plateforme propose une lecture d'usage claire : suivre l'état d'une flotte, comprendre le risque, planifier l'action et afficher des scénarios de démonstration. Cette logique est pensée pour plusieurs machines. Le service temps réel accepte un contrat de données générique, et la chaîne d'apprentissage peut être réentraînée pour une autre famille d'équipements. Le simulateur à trois machines joue ici un rôle pédagogique. La page Experiment apporte une validation complémentaire en montrant que l'application sait recevoir et afficher un flux live simple de courant et de vibration.

Le niveau de maturité doit toutefois être formulé avec précision. Le pipeline est validé scientifiquement. Le produit est démontré fonctionnellement. Une première chaîne live cohérente avec l'architecture SaaS a été préparée et vérifiée fonctionnellement, notamment par un essai à deux postes. Le simulateur reste une démonstration guidée. La page Experiment constitue un test fonctionnel d'acquisition dans l'application. La validation terrain sur données réelles reste l'étape suivante.

Le prochain jalon opérationnel n'est donc plus de concevoir cette chaîne, mais de la qualifier sur une première machine de référence. Le raccordement réel attend encore, de

la part de l'industriel, le chemin exact du CSV, son en-tête confirmé, une ligne d'exemple et la confirmation que le fichier se met bien à jour lorsque la machine fonctionne. Une fois cette première qualification stabilisée, le même schéma pourra être reproduit sur d'autres machines AROTEQ afin d'obtenir une supervision centralisée de plusieurs équipements, puis une ouverture plus générale à d'autres usages industriels.

Ces limites ne diminuent pas la valeur du projet. Elles en précisent simplement le statut. PrediTeq est aujourd'hui une base scientifique solide, un produit logiciel cohérent, un support de démonstration lisible et un point de départ crédible pour une montée en réalisme industriel. La plateforme est conçue pour généraliser sa logique. Cette généralisation est déjà partiellement étayée par sa conception et par la validation externe, mais elle devra encore être confirmée famille de machines par famille de machines.

Apports personnels majeurs

Les contributions personnelles les plus structurantes peuvent être résumées ainsi :

- la consolidation du pipeline autour d’une simulation physiquement guidée en 200 trajectoires et d’une cible de pronostic construite sans fuite d’information ;
- la clarification du lien entre apprentissage hors ligne, service temps réel et comportement applicatif ;
- la reformulation de PrediTeq comme plateforme de supervision et non comme simple démonstrateur algorithmique ;
- la structuration de trois scénarios machine lisibles pour le simulateur ;
- la conception d’une page d’acquisition expérimentale centrée sur deux signaux visibles, courant et vibration ;
- la préparation d’une première chaîne live fonctionnelle et la clarification du contrat terrain restant à confirmer pour une première machine réelle.

Table 5.12 – Synthèse des apports personnels

Brique	Apport principal	Effet concret sur PrediTeq
Pipeline scientifique	Passage à 200 trajectoires et cible de pronostic fondée sur un signal observable	Base méthodologique plus solide pour le futur service temps réel
Architecture produit	Clarification des couches hors ligne, temps réel et interface	Lecture plus nette de la solution comme plateforme complète
Démonstration fonctionnelle	Structuration des trois scénarios et repositionnement de la page Experiment	Démonstration plus lisible et plus honnête
Ouverture industrielle	Préparation et validation fonctionnelle d’une chaîne live et clarification du contrat terrain	Transition vers la donnée réelle sans réécriture de l’architecture

Les perspectives les plus naturelles du projet sont les suivantes :

1. raccorder effectivement une première machine réelle de référence à la chaîne live déjà préparée via le CSV réel fourni par l’industriel ;
2. stabiliser cette première chaîne live, depuis l’identification machine jusqu’à la persistance cloud, sans modifier l’architecture produit ;

3. étendre ensuite le même schéma à d'autres machines ;
4. constituer un historique croisé de télémétrie et d'interventions de maintenance ;
5. recalibrer les seuils et réentraîner hors ligne les modèles lorsque les historiques annotés seront suffisants ;
6. mesurer enfin les gains d'exploitation en disponibilité, en planification et en qualité des alertes.

En définitive, PrediTeq montre qu'une approche de maintenance prédictive devient crédible lorsqu'elle relie la physique, l'apprentissage automatique, le logiciel et la démonstration. Le projet ne cherche pas seulement à produire une prédiction. Il cherche aussi à la rendre intelligible, défendable et exploitable.

Bibliographie et webographie

- [1] L. Breiman, *Random Forests*, Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [2] F. T. Liu, K. M. Ting, Z.-H. Zhou, *Isolation Forest*, Proceedings of the 8th IEEE International Conference on Data Mining, 2008.
- [3] IEEE, *IEEE Std 1856-2017 : Framework for Prognostics and Health Management of Systems*, IEEE Standards Association, 2017.
- [4] ISO, *ISO 10816-3 :2009 Mechanical vibration — evaluation of machine vibration by measurements on non-rotating parts*, International Organization for Standardization, 2009.
- [5] IEC, *IEC 60034-1 : Rotating electrical machines — Part 1 : Rating and performance*, norme internationale mobilisée dans le projet pour le cadrage électrotechnique du moteur de référence.
- [6] A. Saxena, K. Goebel, *Turbofan Engine Degradation Simulation Data Set*, NASA Ames Prognostics Data Repository, 2008.
- [7] P. Probst, A.-L. Boulesteix, *To tune or not to tune the number of trees in random forest*, Journal of Machine Learning Research, 18(181), 1–18, 2018.
- [8] M. Kuhn, K. Johnson, *Applied Predictive Modeling*, Springer, 2013.
- [9] OWASP Foundation, *OWASP Top 10 : The Ten Most Critical Web Application Security Risks*, édition 2021 du référentiel mobilise pour la conception sécurisée et la gouvernance applicative de la plateforme.
- [10] Union européenne, *Regulation (EU) 2016/679 — General Data Protection Regulation (GDPR)*, règlement européen de référence mobilisé ici pour la suppression définitive et la gouvernance des données de comptes.
- [11] K. Schwaber, J. Sutherland, *The Scrum Guide : The Definitive Guide to Scrum*, 2020.
- [12] GitHub Docs, *About large files on GitHub*, documentation officielle mobilisée pour la limite de taille GitHub et le seuil de blocage au-delà de 100 MiB.
- [13] GitHub Docs, *About Git Large File Storage*, documentation officielle mobilisée pour le mécanisme de pointeur Git LFS et la gestion des gros artefacts hors blob Git classique.

- [14] Render, *Deploying on Render*, documentation officielle mobilisée pour la chaîne de build et de déploiement du backend à partir d'un dépôt Git lié.
- [15] Projet PrediTeq, *INDEX_RESULTATS.md*, document interne du dépôt PrediTeq utilisé pour l'inventaire du pipeline, l'ordre des étapes offline et le repérage des artefacts vérifiés.
- [16] Projet PrediTeq, *prediteq_ml/outputs/metrics.json*, artefact exécuté de référence pour les métriques d'anomalies et la synthèse holdout du pipeline.
- [17] Projet PrediTeq, *prediteq_ml/outputs/rul_cv_scores.json*, artefact exécute détaillant holdout, GroupKFold, baselines et ablations du modele RUL.
- [18] Projet PrediTeq, *prediteq_ml/outputs/cmapss_metrics.json*, artefact exécuté de validation externe sur NASA CMAPSS FD001.
- [19] Projet PrediTeq, *prediteq_ml/outputs/calibration_metrics.json*, artefact exécuté d'évaluation de la calibration empirique des intervalles de confiance.
- [20] Projet PrediTeq, *prediteq_ml/outputs/lead_time.json*, artefact exécuté sur le temps d'avance utile avant état critique.
- [21] Projet PrediTeq, *prediteq_api/demo_scenarios.py*, source interne définissant les trois scénarios machine utilisés par le simulateur.
- [22] Projet PrediTeq, *prediteq_frontend/src/components/pages/ExperimentPage.tsx*, source interne de la page live d'acquisition ACS712 5A + MPU6050.

Chapitre 6

Annexes pratiques

6.1 Annexe A : cartographie résumée du dépôt

Table 6.1 – Cartographie fonctionnelle du dépôt

Dossier	Rôle principal
<code>prediteq_ml/</code>	Simulation offline, preprocessing, entraînement, évaluation, export des artefacts.
<code>prediteq_api/</code>	Backend runtime, simulateur, scheduler, routes métier, gouvernance.
<code>prediteq_frontend/</code>	Interface React, pages produit, vues de démonstration et d'administration.
<code>docs/</code>	Guides de lecture, déploiement, démo, reprise de code.
<code>final_report/</code>	Source du présent mémoire et livrable Overleaf.

Pour une reprise technique du projet, les points d'entrée les plus utiles restent les fichiers qui portent directement la logique : `prediteq_ml/steps/` pour la chaîne offline, `prediteq_api/main.py` et `prediteq_api/ml/loader.py` pour le runtime backend, `prediteq_api/routers/mqtt.py` pour l'ingestion live, `prediteq_frontend/src/pages/Index.tsx` pour la cartographie frontend, et `render.yaml` pour la configuration de déploiement.

6.2 Annexe B : commandes utiles

6.2.1 Lancement local

```
cd prediteq_api
pip install -r requirements.txt
uvicorn main:app --reload
```

```
cd prediteq_frontend
npm install
npm run dev
```

6.2.2 Vérification frontend

```
cd prediteq_frontend
npm run build
npm run test
npx playwright test
```

6.2.3 Séquence offline du pipeline

```
cd prediteq_ml
python steps/step1_simulate.py
python steps/step2_preprocess.py
python steps/step3_isolation_forest.py
python steps/step4_health_index.py
python steps/step5_rul_model.py
python steps/step6_evaluate.py
python steps/step6b_cmapss.py
python steps/step6c_calibration.py
python steps/step7_export.py
```

6.3 Annexe C : trois machines du simulateur

Table 6.2 – Détails utiles des scénarios machine

Code	état	Charge type	Site	Lecture démo
ASC-A1	Opérationnel	Légère à moyenne	Bizerte	Machine calme, peu surchargée, HI cible très élevé.
ASC-B2	Sous surveillance	Mixte / demi-charge	Sfax	Machine moyenne, dérive visible, RUL de référence non nulle.
ASC-C3	Critique	Lourde / proche max	Sousse	Machine sévère, biais vibratoire fort, état critique immédiatement lisible.

Cette annexe décrit uniquement les scénarios du simulateur. La première machine réelle visée pour l'intégration terrain reste, elle, la machine ARO-01 située à Ben Arous.

6.4 Annexe D : protocole de démonstration

1. Ouvrir la page `/experiment`.
2. Placer le téléphone de façon à voir simultanément le moteur et l'écran.
3. Cliquer sur *Connecter l'ESP32*.
4. Lancer un chrono.
5. 0 à 7 secondes : moteur OFF.
6. 7 à 14 secondes : moteur ON sans charge.
7. 14 à 21 secondes : moteur OFF.
8. 21 à 28 secondes : moteur ON avec roue ou charge.
9. Après 28 secondes : moteur OFF et stabilisation finale.

Conseil de démonstration : fixer le **MPU6050** sur le corps du moteur ou sur son support rigide, et garder le **ACS712 5A** en série avec l'alimentation du moteur. La disposition expérimentale reste ainsi physiquement crédible pour un observateur.

6.5 Annexe E : chaîne v1 d'intégration réelle

6.5.1 Chaîne déployée

Source locale industrielle ou poste local -> CSV local -> pont de normalisation -> broker MQTT privé TLS 8883 -> backend cloud PrediTeq -> frontend public sur domaine propre

Les paramètres d'accès au broker, les identifiants dédiés, les mots de passe et les éventuels certificats restent hors du mémoire et doivent être fournis par variables d'environnement ou champs à compléter.

6.5.2 Champs live minimaux actuellement attendus par le runtime

- machine_id
- observed_at ou timestamp
- rms_mms
- power_kw
- temp_c
- humidity_rh

6.5.3 Éléments restant à confirmer pour ARO-01

- le chemin exact du CSV écrit par la source industrielle ;
- l'en-tête final validé par l'industriel ;
- une ligne d'exemple issue du flux réel ;
- la confirmation que le fichier se met bien à jour lorsque la machine fonctionne.

6.5.4 Exemple de source CSV puis de payload à compléter

```
machine_id,observed_at,rms_mms,power_kw,temp_c,humidity_rh,current_a,
load_kg,status
ARO-01,<horodatage ISO 8601>,<rms_mms>,<power_kw>,<temp_c>,<humidity_rh
>,<current_a>,<load_kg>,<status>
```

Le bridge lit cette ligne source, normalise les champs utiles, puis publie un message du type :

```
{  
  "machine_id": "ARO-01",  
  "observed_at": "<horodatage ISO 8601>",  
  "rms_mms": <rms_mms>,  
  "power_kw": <power_kw>,  
  "temp_c": <temp_c>,  
  "humidity_rh": <humidity_rh>,  
  "current_a": <current_a>,  
  "load_kg": <load_kg>,  
  "status": "<status>",  
  "source": "csv_bridge"  
}
```

6.6 Annexe F : traçabilité user stories, pages et preuves de test

Table 6.3 – Trace simple entre besoins, pages produit et preuves de validation

User story / besoin	Page ou brique associée	Preuve ou test disponible
Visualiser rapidement l'état de la flotte	Tableau de bord, centre d'alertes	Captures frontend, build frontend, tests Vitest et cohérence des données de simulation runtime
Comprendre pourquoi une machine est à risque	Diagnostic avancé, backend runtime, routes métier	Figures HI / RUL, routes de diagnostic, artefacts holdout / GroupK-Fold / calibration
Comparer trois situations machine lisibles	Simulateur backend et page simulateur	Scénarios ASC-A1 / ASC-B2 / ASC-C3, captures de démonstration et lecture de zones cohérente
Observer un flux live dans l'application	Page Experiment, montage ACS712 5A + MPU6050	Démonstration filmée, protocole chrono, page live à deux cartes et scénario OFF / ON / OFF / ON charge / OFF
Configurer des seuils et gouverner la plateforme	Administration, gestion des machines, seuils runtime	Capture administration, routes backend associées et distinction claire entre seuils offline et runtime
Planifier une action de maintenance	Planner, calendrier, budget	Parcours produit documenté, captures de pages clés et liens entre risque, priorité et action
Préparer une intégration terrain réelle	Bridge MQTT, backend live, documentation LabVIEW / PLC	Documentation de payload, test mock sur poste distant et chaîne cible documentée

6.7 Annexe G : origine des logos technologiques

Les visuels technologiques insérés dans le mémoire sont centralisés dans le dossier `images/logos/`. Le manifeste local `images/logos/sources.json` conserve, pour chaque visuel, l'origine publique retenue lors de la préparation du rapport. Les panneaux graphiques insérés dans les figures et dans la table de synthèse ont ensuite été composés localement à partir de ces visuels.

Table 6.4 – Principe de gestion des visuels technologiques

Catégorie	Principe retenu
Icônes officielles disponibles	Le mémoire réutilise l'icône publique diffusée par le site officiel ou la documentation officielle de la technologie concernée.
Technologies sans marque autonome exploitable	Le mémoire emploie une vignette typographique neutre générée localement, afin d'éviter l'usage d'un logotype non vérifié tout en conservant un rendu homogène.

Chapitre 7

Dossier ultime de soutenance

Ce chapitre complete le memoire principal par une lecture transversale, directement exploitable pendant la soutenance. Il sert a quatre objectifs : reconstituer toute la chaine en ordre, relier chaque partie du produit aux fichiers qui la pilotent, distinguer clairement la demo Experiment de la vraie ingestion live, et proposer un discours a la fois defendable devant un jury technique et intelligible pour un jury mixte.

Bloc	Fichiers de reference	Ce qui se passe et pourquoi cela compte
------	-----------------------	---

7.1 Vue d'ensemble ordonnee du projet

Table 7.1 – Chaîne complete PrediTeq, du moteur au produit

Bloc	Fichiers de reference	Ce qui se passe et pourquoi cela compte
Contexte physique	<code>prediteq_ml/config.py</code> , <code>prediteq_ml/PIPELINE_EXPLAINED.txt</code>	Le projet part d'un moteur reel SITI FC100L1-4. La vitesse et la tension restent fixes ; la charge fait varier puissance, courant, echaufrage et usure. Cette base physique justifie toute la suite.
Step 1 : simulation	<code>prediteq_ml/steps/step1_simulate.py</code>	Generation de trajectoires synthetiques a partir de 20 cas de charge, 4 profils de degradation, un cycle de 44 s, du bruit capteur et un climat plausible. L'objectif n'est pas de remplacer le terrain, mais de construire un premier socle PHM exploitable avant de disposer d'annees de pannes reelles.
Step 2 : pretraitement	<code>prediteq_ml/steps/step2_preprocess.py</code> , <code>prediteq_api/ml/engine_manager.py</code>	Les signaux bruts deviennent 12 variables techniques : <code>rms_mms</code> , <code>drms_dt</code> , <code>rms_variability</code> , <code>p_mean_kw</code> , <code>p_rms_kw</code> , <code>dp_dt</code> , <code>e_cycle_kwh</code> , <code>duration_ratio</code> , <code>t_mean_c</code> , <code>dt_dt</code> , <code>hr_std</code> , <code>corr_t_p</code> . Le backend reproduit ce schema en runtime.
Step 3 : anomalies	<code>prediteq_ml/steps/step3_isolation_forest.py</code>	Un Isolation Forest apprend d'abord la zone saine. Le score final combine cette vision multivariee et la vibration RMS pour mieux capter a la fois la derive globale et le signal vibratoire dominant.
Step 4 : HI	<code>prediteq_ml/steps/step4_health_index.py</code>	Les anomalies deviennent un Health Index lisse entre 0 et 1, puis quatre zones lisibles : excellent, bon, degrade, critique. Cette traduction est capitale pour les non-specialistes.

Bloc	Fichiers de reference	Ce qui se passe et pourquoi cela compte
Step 5 : RUL	<code>prediteq_ml/steps/step5_rul_model.py</code> , <code>prediteq_ml/models/prediteq_engine.py</code>	Le modele RUL n'apprend pas sur un label cache de simulation. Il apprend une cible derivee de <code>hi_smooth</code> observable, avec une logique de franchissement persistant qui evite la fuite de donnees. Les entrees utiles sont 12 features normalisees plus 5 resumes HI.
Steps 6, 6B, 6C	<code>prediteq_ml/steps/step6_evaluate.py</code> , <code>prediteq_ml/steps/step6b_cmapss.py</code> , <code>prediteq_ml/steps/step6c_calibration.py</code>	Le pipeline est mesure sur holdout, en GroupKFold par trajectoire, sur NASA CMAPSS FD001, puis par calibration d'intervalles. C'est la preuve que le travail ne se limite pas a une demo visuelle.
Step 7 : export	<code>prediteq_ml/steps/step7_export.py</code> , <code>prediteq_ml/models/</code> , <code>prediteq_ml/outputs/mqtt_schema.json</code>	Les artefacts exportes deviennent le contrat de production : modeles, scaler, schema MQTT et classe <code>PrediteqEngine</code> . L'API ne reentraine rien ; elle consomme ce qui a ete preparee offline.
Backend runtime	<code>prediteq_api/main.py</code> , <code>prediteq_api/ml/engine_manager.py</code> , <code>prediteq_api/core/decision_snapshot.py</code>	Au demarrage, FastAPI verifie la configuration, initialise Supabase, charge les artefacts ML, cree un moteur par machine, tente MQTT, charge les seuils live, puis lance le scheduler. La decision snapshot consolide ensuite HI, RUL, stress, alertes et recommandations dans une seule verite produit.
Persistence et alertes	<code>prediteq_api/scheduler.py</code> , <code>prediteq_api/routers/seuils.py</code>	Toutes les 60 s, le scheduler persiste HI, RUL, historique, alertes et emails ; toutes les 1 h, il met a jour <code>power_avg_30j</code> et <code>cycles_avg_7j</code> . Les seuils runtime peuvent venir de la table Supabase <code>seuils</code> et donc differer des seuils offline.
Frontend produit	<code>prediteq_frontend/src/pages/Index.tsx</code> , <code>prediteq_frontend/src/components/pages/</code>	L'interface React ne montre pas seulement un score : elle permet de surveiller, diagnostiquer, planifier, estimer les couts, traiter les alertes, generer des rapports et administrer la flotte.

Bloc	Fichiers de reference	Ce qui se passe et pourquoi cela compte
Demo simulateur	<code>prediteq_api/routers/simulator.py</code> , <code>prediteq_api/demo_scenarios.py</code>	Le simulateur backend traverse bien le moteur runtime, mais il reste oriente soutenance : il peut maintenir l’affichage dans des bandes pedagogiques de HI, de zone et de RUL. Il ne faut donc pas le presenter comme un replay neutre du terrain.
Demo Experiment	<code>prediteq_frontend/src/components/pages/ExperimentPage.tsx</code> , <code>hardware/esp_bench_arduino712_simple/</code>	Cette page relie Chrome ou Edge a un ESP32 par Web Serial pour visualiser vibration et courant sur un banc local. Elle valide une integration capteur rapide, mais elle n’est pas la meme chaine que l’ingestion cloud PrediTeq.
Ingestion live	<code>prediteq_api/routers/mqtt.py</code> , <code>prediteq_api/routers/live_ingest.py</code> , <code>prediteq_api/scripts/mqtt_bridge_sender.py</code>	Pour le terrain, les mesures doivent passer par MQTT ou HTTP, etre rattachees a une machine connue, etre ingerees par le backend puis exposees au frontend. C’est cette voie qui prepare le raccordement d’un PC relais cote client, de LabVIEW, du PLC et de la machine ARO-01.

Les preuves quantitatives centrales restent les suivantes : F1 hybride d’anomalies = **0,964**, RUL holdout = **R^2 0,980** avec **RMSE 2,49 jours** et **MAE 1,65 jour**, validation GroupKFold = **R^2 moyen 0,982 $\pm$ 0,001**, validation NASA CMAPSS FD001 = **R^2 0,886** et **RMSE 14,11 cycles**, calibration native IC80 = **87,2 %** de couverture avec **ECE 0,088**. Le signal d’alerte arrive suffisamment tot sur la simulation actuelle : **194 trajectoires** analysees sur 200 donnent un **lead time moyen de 564,8 min**, soit environ **62,76 jours equivalents interface**. Ces chiffres viennent des artefacts sous `prediteq_ml/outputs/*.json`.

7.2 Demarche methodologique suivie

PrediTeq a ete mene comme un vrai projet PHM naissant : ne pas attendre un historique parfait pour commencer, mais ne pas masquer non plus ce qui reste a consolider sur le terrain. L’idée directrice est simple : partir d’un cas machine reel, construire un premier socle realiste quand les longues pannes annotees manquent, avancer etape par etape, puis preparer en parallele la collecte live et l’annotation future.

Table 7.2 – Demarche industrielle recommandee et reponse effective de PrediTeq

Principe	Bonne pratique	Comment PrediTeq y repond
Point de depart	Partir d'un cas industriel reel meme si l'historique est encore court.	Le projet part du moteur SITI FC100L1-4, de sa plaque signalétique, de ses charges, de son cycle de 44 s et d'un contexte usine reel. On n'a donc pas inventé un cas abstrait juste pour faire tourner du ML.
Donnees insuffisantes	Si les pannes reelles manquent, lancer le projet avec une simulation realiste.	Step 1 genere 200 trajectoires, 4 profils, 20 charges, du bruit capteur et un climat plausible. La simulation sert de premier socle PHM, pas de substitut definitif au terrain.
Repere externe	Utiliser un dataset voisin si disponible, ou au moins un benchmark externe.	PrediTeq n'emploie pas un dataset externe comme source principale du stockeur, mais Step 6B verifie l'architecture sur NASA CMAPSS FD001. Ce benchmark sert de preuve de generalisation methodologique, pas de raccourci marketing.
Signal puis indicateurs	Nettoyer, segmenter et resumer les signaux avant tout modele aval.	Step 2 detecte les phases et calcule 12 variables techniques. Pour un non-specialiste, une feature est simplement un indicateur derive des mesures brutes, comme une moyenne, une variation ou une correlation.
Anomalies avant RUL	D'abord reconnaitre l'ecart au comportement sain, puis seulement estimer le temps restant.	Step 3 construit le score d'anomalie, Step 4 le traduit en HI lisible, puis Step 5 estime le RUL. Le projet n'a donc pas saute directement d'un signal brut a un pronostic opaque.
Validation progressive	Verifier chaque bloc avant de passer au suivant.	Holdout, GroupKFold par trajectoire, benchmark NASA, calibration et lead time montrent que la chaine a ete mesuree a plusieurs niveaux avant d'etre defendue en soutenance.

Principe	Bonne pratique	Comment PrediTeq y repond
Collecte terrain	Organiser en parallele la collecte live et le journal d'evenements.	La voie LabVIEW / PLC -> PC relais cote client -> MQTT ou HTTP -> /ingest/live est deja preparee, et le backend historise deja HI, RUL, alertes et contexte. En revanche, un vrai journal terrain annote reste la prochaine marche de maturite.
Expertise terrain	S'appuyer sur les techniciens pour choisir variables et hypotheses.	Les constantes moteur, la charge maximale, le cycle et la logique charge -> puissance -> courant -> echauffement viennent de la machine reelle et des echanges terrain. Cette expertise explique pourquoi les variables retenues sont defendables.

Autrement dit, PrediTeq repond bien a la demarche saine attendue dans un projet industriel avec peu de donnees initiales : on demarre par une simulation realiste, on avance bloc par bloc, on valide chaque niveau, puis on organise deja la collecte live et l'historisation qui permettront plus tard de raffiner les seuils, de recalibrer les sorties et de reentrainer hors ligne.

Choix	Ce que c'est	Pourquoi ce choix et ce qu'il change
-------	--------------	--------------------------------------

7.3 Choix structurants et justification des methodes

Table 7.3 – Choix techniques majeurs, justification et effet pratique

Choix	Ce que c'est	Pourquoi ce choix et ce qu'il change
200 trajectoires, 4 profils, 20 charges	<code>config.py</code> fixe <code>N_TRAJECTORIES</code> = 200, <code>N_PROFILES</code> = 4, 20 cas de charge de 0 a 285 kg.	Le modele voit toute la plage utile, de l'ascenseur a vide a la pleine charge, avec plusieurs formes d'usure. On reduit ainsi le risque de confondre "forte charge normale" et "derive anormale".
Cycle machine de 44 s	12 s montee, 12 s descente, 20 s pause.	Cette periodicite structure les signatures electriques et vibratoires. Sans elle, la simulation serait moins credible et le runtime disposerait de moins de repere pour lisser proprement les lectures.
Bruit capteur et climat	Bruit sur vibration, puissance, temperature, humidite ; plage climatique Ben Arous.	Le pipeline apprend sur des signaux moins "propres" et donc plus proches du terrain. Cela reduit le risque d'un systeme excellent en simulation mais fragile devant de vrais capteurs.
4 capteurs live minimaux	<code>rms_mms</code> , <code>power_kw</code> , <code>temp_c</code> , <code>humidity_rh</code> .	Ce noyau minimal est deja assez riche pour calculer HI, stress et, apres historique suffisant, RUL. Le courant et la charge restent utiles, mais le backend fonctionne deja autour de ce minimum.
Cadence live	Schema MQTT a 1 Hz, HI sur 120 points, buffer RUL sur 60 points, prediction toutes les 60 s.	On se cale sur un rythme simple, lisible et robuste : assez frequent pour detecter une derive, assez stable pour eviter les oscillations d'une seconde a l'autre.
12 features runtime	Features derivees calculees dans <code>step2_preprocess.py</code> et <code>engine_manager.py</code> .	On ne donne pas les donnees brutes telles quelles au modele. Les derivees, variabilites, correlations et energies de cycle rendent la physique de degradation plus visible pour l'algorithme.

Choix	Ce que c'est	Pourquoi ce choix et ce qu'il change
Isolation Forest	IF_N_ESTIMATORS = 100, IF_CONTAMINATION = 0.05.	L'Isolation Forest delimite une zone saine sans exiger un catalogue complet de pannes etiquetees. C'est un bon compromis quand les labels de defaillance reelle sont encore rares.
Score hybride $\alpha = 0,2$	20 % score IF, 80 % score RMS.	Dans la simulation actuelle, la vibration RMS porte l'essentiel du signal utile, mais l'Isolation Forest garde une sensibilit� aux derives multicapteurs. L'ensemble obtient un meilleur compromis precision/rappel que chaque bloc seul.
HI a 4 zones	Seuils 0,8 / 0,6 / 0,3 et lissage 120 s.	Le HI traduit le jargon PHM en langage metier. Un responsable comprend plus vite "excellent / bon / degrade / critique" qu'une suite de scores d'anomalie. Le lissage limite aussi les oscillations visuelles.
Cible RUL observable	Franchissement persistant de <code>hi_smooth</code> , persistance 3.	Ce choix evite la fuite de donnees : la cible provient d'un signal observable par le runtime, pas d'un etat cache de simulation inaccessible en exploitation.
Random Forest RUL	300 arbres, profondeur 16, <code>min_samples_leaf = 10</code> , <code>max_features = sqrt</code> .	Le Random Forest gere bien les relations non lineaires, reste stable sur un dataset synthetique heterogene et s'exporte facilement vers la couche runtime.
Validation GroupKFold	5 folds par trajectoire.	Une trajectoire entiere reste d'un seul cote du split. On evite donc un faux bon score cree par des points presque jumeaux de la meme machine presentes dans train et validation.
Validation NASA + calibration	<code>step6b_cmapss.py</code> et <code>step6c_calibration.py</code> .	NASA CMAPSS montre que l'architecture generale ne depend pas uniquement du stockeur. La calibration, elle, verifie si les intervalles de confiance racontent quelque chose de raisonnable.
Seuils runtime depuis Supabase	<code>prediteq_api/routers/seuils.py</code> .	Les seuils visibles en exploitation peuvent etre ajustes sans recompiler le modele. C'est pratique en phase pilote, mais il faut le dire franchement pour ne pas melanger seuils offline et seuils terrain.

Pour un jury non technique, la justification peut se resumer ainsi : *nous avons choisi des methodes assez solides pour tenir debout scientifiquement, mais assez simples pour rester explicables et exportables dans un vrai produit.* Pour un jury technique, la phrase juste est : *PrediTeq privilegie une chaine PHM robuste, sans fuite de donnees, mesuree sur plusieurs protocoles, et directement raccordable au runtime.*

Couche	Technologies	Fichiers repere	Role reel
--------	--------------	-----------------	-----------

7.4 Cartographie technologique exacte

Table 7.4 – Technologies utilisees et role reel dans le projet

Couche	Technologies	Fichiers repere	Role reel
ML offline	Python, NumPy, Pandas, scikit-learn, joblib, SHAP	prediteq_ml/steps/, prediteq_ml/models/, prediteq_ml/outputs/	Simulation, preprocessing, entraînement, evaluation, calibration, explicabilite et export des artefacts.
Backend runtime	FastAPI, Pydantic, APScheduler, gmqtt, Supabase Python client, fpdf2, Groq	prediteq_api/main.py, prediteq_api/routers/, prediteq_api/core/	Chargement des artefacts ML, ingestion live ou simulee, diagnostics, rapports, alertes email, persistance et gouvernance de la flotte.
Frontend web	React 18, Vite, TypeScript, TanStack Query, React Router, Recharts, Leaflet, Radix UI, Supabase JS	prediteq_frontend/src/App.tsx, prediteq_frontend/src/pages/Index.tsx	Experience produit complete : tableau de bord, diagnostics, carte, alertes, maintenance, couts, administration, simulateur, Experiment et rapports IA.
Couche decision	Python metier et hooks frontend	prediteq_api/core/decision_snapshot.py, prediteq_frontend/src/hooks/useFleetPredictiveInsights.ts	Une seule synthese de decision alimente dashboard, planner, couts, maintenance et alertes pour eviter les verites contradictoires d'un ecran a l'autre.
IoT de demo	ESP32, ACS712 5A, MPU6050, Web Serial API	hardware/esp_bench_acs712_simple/, ExperimentPage.tsx	Banc local visible en direct dans le navigateur pour vibration et courant. C'est une preuve capteur rapide, distincte de la chaine cloud de production.

Couche	Technologies	Fichiers repere	Role reel
Integration terrain	MQTT, JSON, CSV, HTTPS avec token	prediteq_api/scripts/mqtt_bridge_sender.py, routers/mqtt.py, routers/live_ingest.py	Raccordement d'une source externe telle qu'un PC relais cote client, LabVIEW ou un PLC vers le backend PrediTeq.
Stockage et temps reel	Supabase	prediteq_api/sql/, prediteq_frontend/src/lib/runtimeDataRepository.ts	Machines, alertes, taches GMAO, couts, seuils, rapports, historique HI, historique RUL et canaux temps reel pour l'interface.
Qualite frontend	Vitest, Playwright	prediteq_frontend/package.json	Validation unitaire et E2E de l'application web.
Rendu final	LaTeX, Overleaf compatible, Tectonic portable	final_report/prediteq_overleaf_report/	Production du memoire et de ce dossier de soutenance au format PDF.

7.5 Ce que les utilisateurs peuvent faire dans l'application

7.5.1 Commandes communes visibles sur presque tout le produit

- **Topbar** : indicateur live, date de derniere mise a jour, recherche machine/page, selecteur de langue FR/EN/AR, bascule theme, raccourci vers les alertes et resume du profil connecte.
- **Sidebar** : navigation par section, badge du nombre d'alertes actives, lien vers les zones metier, bouton deconnexion et bouton de reduction/extension tout en bas.
- **Scoping** : un compte **admin** voit toute la flotte; un compte **user** approuve est borne a la machine qui lui est assignee.

7.5.2 Pages publiques avant connexion

Table 7.5 – Pages visibles avant authentification

Route	Acces	Ce que l'on peut y faire
/ et /landing	public	Voir la proposition de valeur, les blocs produit, les scenarios de demo, les offres, puis naviguer vers /login ou /signup. La page expose aussi une navigation marketing et un changement de theme.
/login	public	Se connecter, changer langue et theme, afficher ou masquer le mot de passe, ouvrir la page mot de passe oublie ou la page d'inscription.
/signup	public ou admin-create	Demander un compte, choisir un role, saisir les informations d'identite, et pour un administrateur deja connecte creer un nouveau compte via le mode admin-create .
/forgot-password	public	Demander la reinitialisation du mot de passe puis revenir a la connexion.
/reset-password	public via lien de reset	Choisir un nouveau mot de passe et confirmer la mise a jour.
/pending	compte en attente	Voir l'etat "en attente d'approbation", se deconnecter ou revenir vers l'inscription.

Route	Acces	Ce que l'utilisateur peut faire
-------	-------	---------------------------------

7.5.3 Pages metier pour les comptes approuves

Table 7.6 – Pages metier accessibles apres connexion

Route	Acces	Ce que l'utilisateur peut faire
/dashboard	admin + user	Voir la flotte, les cartes machine, le HI, la zone, les tendances et la synthese predictive. L'ecran permet un rafraichissement manuel, l'ouverture d'un contexte machine, l'ouverture d'un panneau d'explication, le saut vers le diagnostic detaille, et, pour l'admin, un demarrage rapide du simulateur ou l'ouverture des controles du simulateur. Le ChatWidget apparait aussi ici.
/diagnostics	admin + user	Consulter le payload de diagnostic le plus riche : RUL calibre, intervalle de confiance, Stress Index, causes probables, regles expertes, disclosures et explication SHAP du pronostic. Un bouton renvoie au dashboard.
/geo	admin + user	Afficher la carte Leaflet et focaliser une machine a partir de son point geographique.
/alertes	admin + user	Lire les alertes issues du pronostic, les grouper, filtrer, ouvrir le planner ou la maintenance, ouvrir le diagnostic d'une machine et acquitter les alertes. L'historique peut etre affiche ou masque.
/maintenance et /calendrier	admin + user	Gerer le calendrier GMAO : changer de mois, exporter CSV, creer une tache, ouvrir un jour, editer une tache, changer son statut ou la supprimer. Les taches approuvees depuis le planner arrivent ici. Le ChatWidget apparait aussi sur cette page.
/ia	admin + user	Ouvrir la couche IA du produit. Pour un compte standard, l'usage principal reste le rapport. Pour un administrateur, la page peut aussi basculer vers le planner.
/rapport-ia	admin + user	Choisir une machine ou toute la flotte, une periode, une langue, generer un rapport, exporter le PDF, relire l'historique et telecharger les PDF deja generees.

7.6 Surfaces backend et logique partagee

Table 7.8 – Principales surfaces backend exposees par l’API

Surface	Repere code	Role produit
Sante et demarrage	<code>routers/health.py</code> , <code>main.py</code>	Demarre Supabase, charge les artefacts ML, initialise le gestionnaire multi-machine, tente MQTT, charge les seuils live, puis demarre le scheduler.
Authentification	<code>core/auth.py</code> , <code>routers/auth.py</code>	Verifie les JWT Supabase, distingue <code>admin</code> et <code>user</code> , exige un compte approuve, et limite un utilisateur standard a sa machine assignee.
Machines	<code>routers/machines.py</code>	Retourne la flotte, les dernieres mesures, le RUL live public, l’historique capteur, et permet le reset post-maintenance.
Diagnostic complet	<code>routers/diagnostics_rul.py</code>	Publie <code>/diagnostics/{code}/all</code> avec RUL calibre, intervalle, Stress Index, regles expertes, SHAP et modes <code>prediction</code> , <code>initializing</code> ou <code>reference_only</code> .
Explainabilite SHAP	<code>routers/explain.py</code>	Retourne les contributions SHAP des 12 features d’anomalie pour la derniere lecture utile d’une machine.
Chat IA	<code>routers/chat.py</code>	Fournit un assistant conversationnel base sur Groq avec outils metier pour interroger l’etat machine, les alertes, les taches et l’explicabilite selon un public jury, technicien ou mixte.
Alertes	<code>routers/alerts.py</code> , <code>scheduler.py</code>	Gere le centre d’alertes, l’historique email et le declenchement dedupe des alertes selon les transitions de zone et l’etat runtime.
Maintenance, couts, HI	<code>routers/runtime_data.py</code>	Sert le CRUD des taches GMAO, la lecture des couts, l’historique HI et d’autres donnees runtime exploitees par Maintenance et Costs.
Planner	<code>routers/planner.py</code>	Retourne du JSON deterministe pour classer le risque, proposer des actions, puis inserer la tache approuvee dans <code>gmao_taches</code> .
Rapports	<code>routers/report.py</code> , <code>report_engine.py</code>	Produit des rapports LLM via Groq et des rapports template sans LLM, avec historique et export PDF.

Surface	Repere code	Role produit
Seuils	<code>routers/seuils.py</code>	Charge les seuils depuis Supabase, expose une version publique et une version admin, et connecte les destinataires d'alertes.
Simulateur	<code>routers/simulator.py</code> , <code>demo_scenarios.py</code>	Alimente la demo avec trois histoires distinctes : ASC-A1 plutot sain, ASC-B2 sous surveillance, ASC-C3 critique. Les sorties restent pilotees pour la demonstration.
Ingestion HTTP	<code>routers/live_ingest.py</code>	Accepte un payload protege par LIVE_INGEST_TOKEN via <code>/ingest/live</code> , verifie que la machine existe, puis l'ingere dans le moteur runtime.
Ingestion MQTT	<code>routers/mqtt.py</code>	Souscrit a <code>prediteq/+sensors</code> , peut recharger une machine connue depuis Supabase lors du premier message live, puis injecte les mesures dans le moteur.

Le point architectural le plus important est la **synthese de decision partagee**. La fonction `build_machine_decision_snapshot()` reconcilie HI, RUL, stress, alertes, taches ouvertes, fraicheur des donnees et recommandations en une seule vue. Cette vue alimente ensuite le dashboard, le planner, la maintenance, les couts et le reporting. Autrement dit, les ecrans ne reinventent pas chacun leur propre verite.

7.7 IoT, Experiment, MQTT et PC relais cote client

7.7.1 Il y a en realite deux chaines capteur distinctes

Table 7.9 – Difference entre la page Experiment et la vraie chaine live PrediTeq

Aspect	Page Experiment	Chaine live backend PrediTeq
Transport	USB direct via Web Serial dans Chrome ou Edge, a 115200 bauds.	MQTT sur <code>prediteq/{machine_id}/sensors</code> ou HTTP sur <code>/ingest/live</code> .
Source	ESP32 + ACS712 5A + MPU6050 sur un banc local.	PC relais cote client, LabVIEW, PLC, script pont JSON/CSV/custom, puis broker ou HTTPS.

Aspect	Page Experiment	Chaine live backend PrediTeq
But	Verifier rapidement une lecture capteur visible dans le navigateur.	Alimenter le backend cloud pour produire HI, diagnostic, alertes, historique et plus tard RUL.
Backend PrediTeq	Non, ou seulement de maniere indirecte par l'operateur.	Oui, c'est la voie officielle d'exploitation.
Commandes utilisateur	Connecter, deconnecter, recalibrer ACS (c), MPU (m) ou les deux (a).	Enregistrer la machine, lancer le sender, verifier MQTT ou HTTP, attendre le warmup RUL.
Nature de la preuve	Demo capteur et UX experimentale.	Validation fonctionnelle de bout en bout, puis integration industrielle progressive.

7.7.2 Ce que fait exactement la page Experiment

La page /experiment parse des trames texte de forme $I = \dots \text{ mA} \mid \text{Vib} = \dots \text{ g} \mid |a| = \dots \text{ g}$. Elle maintient deux cartes : **vibration** et **courant**. La vibration applique une hysteresis d'alarme a 0,30 g a l'entree et 0,20 g a la sortie. Le courant reste une carte d'affichage. Deux LEDs logiques sont visibles : verte si le banc est connecte sans alarme vibration, rouge clignotante si la vibration depasse le seuil.

Le montage materiel de reference reste : ESP32 + MPU6050 + ACS712 5A, avec sortie ACS712 OUT vers GPIO35 via un diviseur 10k / 20k, et protection moteur par diode 1N4007 et condensateur 1000 uF. Cette page est donc un excellent support de demonstration, mais elle ne doit pas etre decrite comme la preuve finale d'une chaine industrielle cloud complete.

7.7.3 Validation fonctionnelle a deux postes

Le depot contient une validation claire de la chaine PC2 source externe -> MQTT -> backend PrediTeq -> application sur PC1. Le but n'est pas de prouver LabVIEW ou le PLC eux-memes, mais de prouver qu'un poste externe peut deja alimenter PrediTeq. Dans ce test, le code machine AR0-01 est affiche publiquement sous le nom **Machine reelle**.

Il faut presenter ce test exactement pour ce qu'il est :

- une **validation fonctionnelle de bout en bout sur donnees simulees** ;

- une preuve que le schema reseau et logiciel est deja viable ;
- **pas encore** une validation industrielle longue duree sur la machine de Ben Arous.

7.7.4 Chemin le plus simple avec un PC relais cote client

Le plan documente le plus sobre est :

1. creer d'abord la machine reelle dans PrediTeq, par exemple **ARO-01** ;
2. configurer un broker prive MQTT ou, a default, la voie HTTP protegee par **LIVE_INGEST_TOKEN** ;
3. faire ecrire par LabVIEW ou par le PLC un **JSON** ou un **CSV** local mis a jour chaque seconde sur n'importe quel PC relais choisi cote client ;
4. faire lire ce fichier par `prediteq_api/scripts/mqtt_bridge_sender.py` ;
5. publier les valeurs vers MQTT ou `/ingest/live` ;
6. verifier dans PrediTeq que `last_sensors`, `hi_courant` et `zone_live` apparaissent ;
7. attendre ensuite environ **60 minutes d'historique HI** pour obtenir un RUL numerique stabilise.

Le minimum actuellement attendu par le backend est :

```
{
  "machine_id": "ARO-01",
  "observed_at": "2026-05-11T18:00:00Z",
  "rms_mms": 1.24,
  "power_kw": 0.88,
  "temp_c": 26.5,
  "humidity_rh": 57.0
}
```

Les champs `current_a`, `load_kg`, `status` et `source` restent utiles mais non obligatoires. En revanche, il faut dire franchement au jury qu'avec le backend actuel, l'absence de `power_kw` ou de `humidity_rh` pose un vrai probleme de coherence pour la chaine live.

7.7.5 Mode demo jury deja en place : donnees LabVIEW de demo sur la chaine live reelle

Pour la soutenance, nous pouvons deja **montrer une arrivee de donnees type LabVIEW depuis un PC relais cote client**, sans pretendre que le LabVIEW ou le PLC reel est termine. La position defendable devant le jury est la suivante : **la fin de chaine est deja la vraie fin de chaine PrediTeq**; seule la source PC relais reste encore une source de demonstration au format LabVIEW.

Pour cela, nous avons ajoute un mode demo reutilisable dans le depot : `prediteq_api/scripts/generate_labview_demo_csv.py`, `prediteq_api/scripts/replay_labview_demo_csv.py` et le gabarit `prediteq_api/scripts/sample_data/ARO-01_labview_demo_template.csv`. L’idee est de **ne pas changer la fin de chaine**. Nous remplaçons seulement, pour l’instant, l’écriture réelle LabVIEW/PLC par une écriture CSV de demonstration mais physiquement plausible.

Pendant la demo jury, le chemin raconte et montre donc : `template CSV -> writer LabVIEW demo -> fichier CSV live -> mqtt_bridge_sender.py -mode csv-last-row -> MQTT/HTTP -> backend -> frontend`. Cette strategie est importante pedagogiquement : le jour ou la vraie source industrielle existe, il suffit d’arreter le writer de demo et de laisser le vrai LabVIEW ou le vrai PLC ecrire le meme fichier CSV.

La phrase honnete a utiliser devant le jury est : *nous montrons deja la chaine PC relais -> CSV -> bridge -> MQTT/HTTP -> PrediTeq, mais la partie CSV est encore une source LabVIEW de demonstration en attendant le vrai export LabVIEW/PLC*. Ainsi, nous **faisons deja entrer un flux “PC relais cote client” dans l’application**, sans presenter a tort ce flux comme une acquisition industrielle finalisee.

Le gabarit CSV reste coherent avec la simulation PrediTeq :

- cycle machine de **44 s** (12 + 12 + 20);
- cas de charge discrets de **0 a 285 kg**;
- hausse de `motor_power` et de `current` lorsque la charge ou la degradation augmentent;
- vibration `vibration_mm_s` plus forte quand la fenetre HI se degrade;
- temperature et humidite lisses, au lieu d’un bruit purement aleatoire.

La sequence de demonstration correspondante est :

```
cd prediteq_api
python scripts/generate_labview_demo_csv.py --machine-id ARO-01 --
    scenario surveillance
python scripts/replay_labview_demo_csv.py --input scripts/sample_data/
    ARO-01_labview_demo_template.csv --output C:\labview\prediteq_log.csv
    --interval 1.0
```

```
python scripts/mqtt_bridge_sender.py --mode csv-last-row --machine-id
ARO-01 --csv-path C:\labview\prediteq_log.csv
```

Pour une soutenance plus courte, il est possible de compresser la demo avec `-interval 0.25`. Il faut alors le presenter honnêtement comme une **accélération de démonstration**, et non comme la fréquence cible industrielle. La cible réelle reste **1 Hz**, conformément au schema MQTT exporté par `prediteq_ml/outputs/mqtt_schema.json`.

Un extrait typique du CSV “PC relais cote client” version LabVIEW demo est :

```
machine_code,time,profile,scenario,phase,vibration_mm_s,motor_power,
temperature,humidity,current,charge,state,source
ARO-01,2026-05-19T16:18:26.695Z,B_quadratic,surveillance,ascent
,1.668,0.839,24.1,64.3,1.514,90.0,ascent,labview_demo_csv
```

Pour un non-spécialiste, cette ligne suffit à montrer l’idée : **un poste externe prépare une ligne de mesures, puis PrediTeq la lit comme s’il s’agissait déjà d’un PC relais cote client**. Pour un spécialiste, elle prouve en plus que les noms de colonnes cote PC relais peuvent être différents des noms internes PrediTeq, puisque `mqtt_bridge_sender.py` les renomme ensuite vers `machine_id`, `rms_mms`, `power_kw`, `temp_c`, `humidity_rh`, `current_a`, `load_kg` et `status`.

Il n’a pas été nécessaire d’ajouter une nouvelle page au frontend pour cette étape. L’application consomme déjà le flux live normal via MQTT ou HTTP, et la présentation machine sait déjà afficher un libelle de type LabVIEW / PC relais quand la source le signale.

7.8 Preuves par extraits de code

Le principe de cette section est simple : chaque fois que nous affirmons “nous avons simulé cela” ou “le pipeline ML fait cela”, nous montrons un extrait court du code correspondant, puis une lecture simple de ce que cela prouve. En plus des extraits retapés dans le rapport, nous ajoutons aussi une **capture directe du code source réel** afin que le jury voie que chaque affirmation correspond bien à un fichier concret du dépôt.

7.8.1 Preuve 1 : la simulation part d’un moteur et d’un cycle réels

Source : `prediteq_ml/config.py`

```
MOTOR_SPEED_RPM = 1410
MOTOR_POWER_KW = 2.2
MOTOR_VOLTAGE_V = 400
MOTOR_COSPHI = 0.80
```

```

N_FLOORS = 19
LOAD_PER_FLOOR_KG = 15
LOAD_MAX_KG = 19 * 15

T_ASCENT_S = 12
T_DESCENT_S = 12
T_PAUSE_S = 20
T_CYCLE_S = 44

N_TRAJECTORIES = 200

```

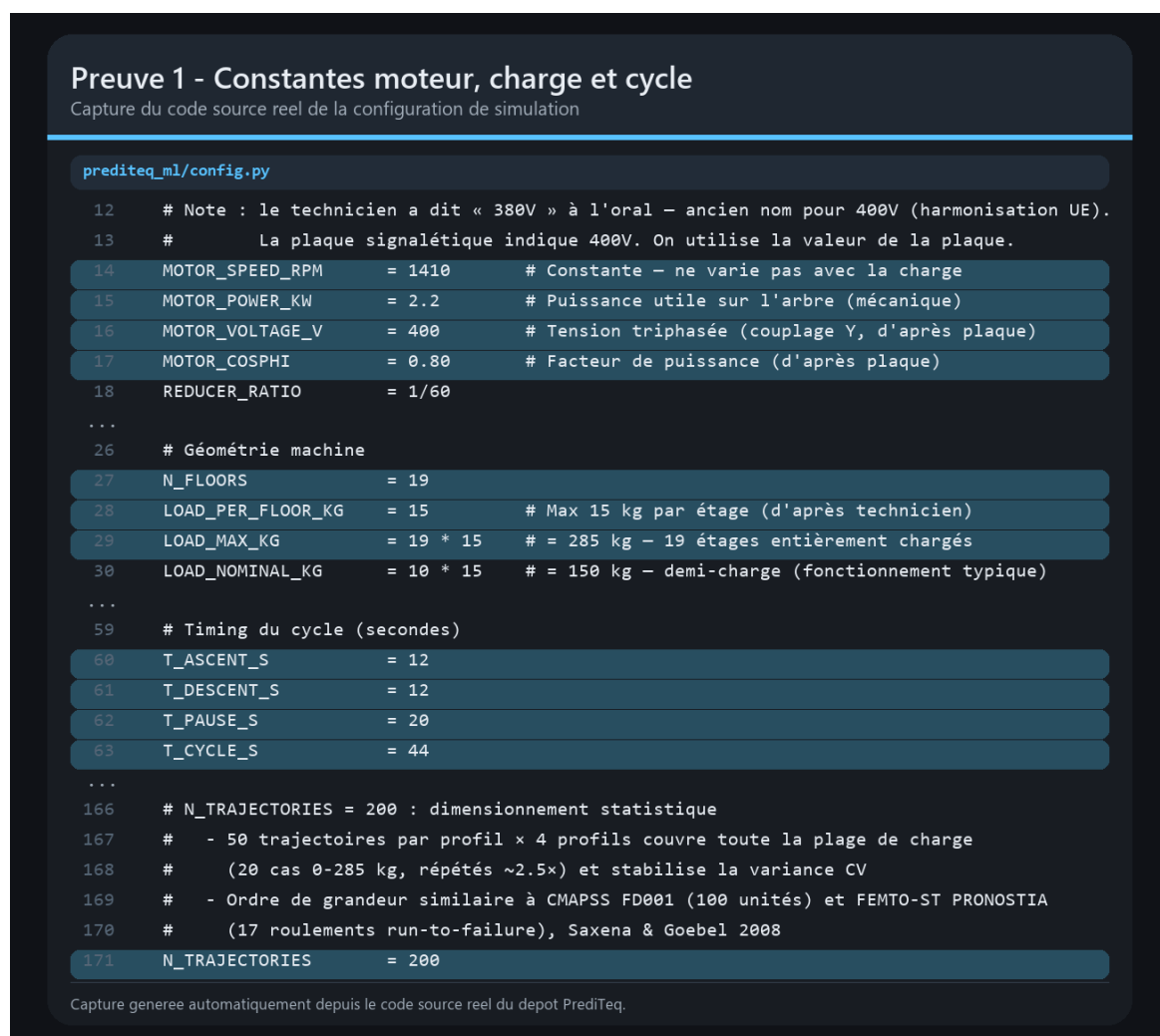


Figure 7.1 – Capture du code source réel montrant les constantes moteur, la charge maximale, le cycle 44 s et les 200 trajectoires

Lecture simple :

- ce code prouve que la simulation ne part pas d'un moteur imaginaire ;
- les constantes moteur, la charge maximale et le cycle de 44 s sont écrits en dur dans la configuration ;

— le jury peut donc voir d’où viennent les chiffres annonces dans la presentation.

7.8.2 Preuve 2 : la puissance et le courant sont bien relies a la charge et a l’etat de sante

Source : prediteq_ml/steps/step1_simulate.py

```
if pt < T_ASCENT_S:
    phase[i] = 'ascent'
    p_load = P_ASCENT_EMPTY_KW + P_ASCENT_LOAD_RANGE * load_ratio
    p_deg = P_ASCENT_DEG_RANGE * (1 - h)
    power[i] = np.clip(p_load + p_deg, P_ASCENT_EMPTY_KW,
                       P_ASCENT_DEG_KW)
elif pt < T_ASCENT_S + T_DESCENT_S:
    phase[i] = 'descent'
    power[i] = P_DESCENT_KW
else:
    phase[i] = 'pause'
    power[i] = P_PAUSE_KW

current = (power * 1000) / MOTOR_SQRT3_V_COSPHI
```

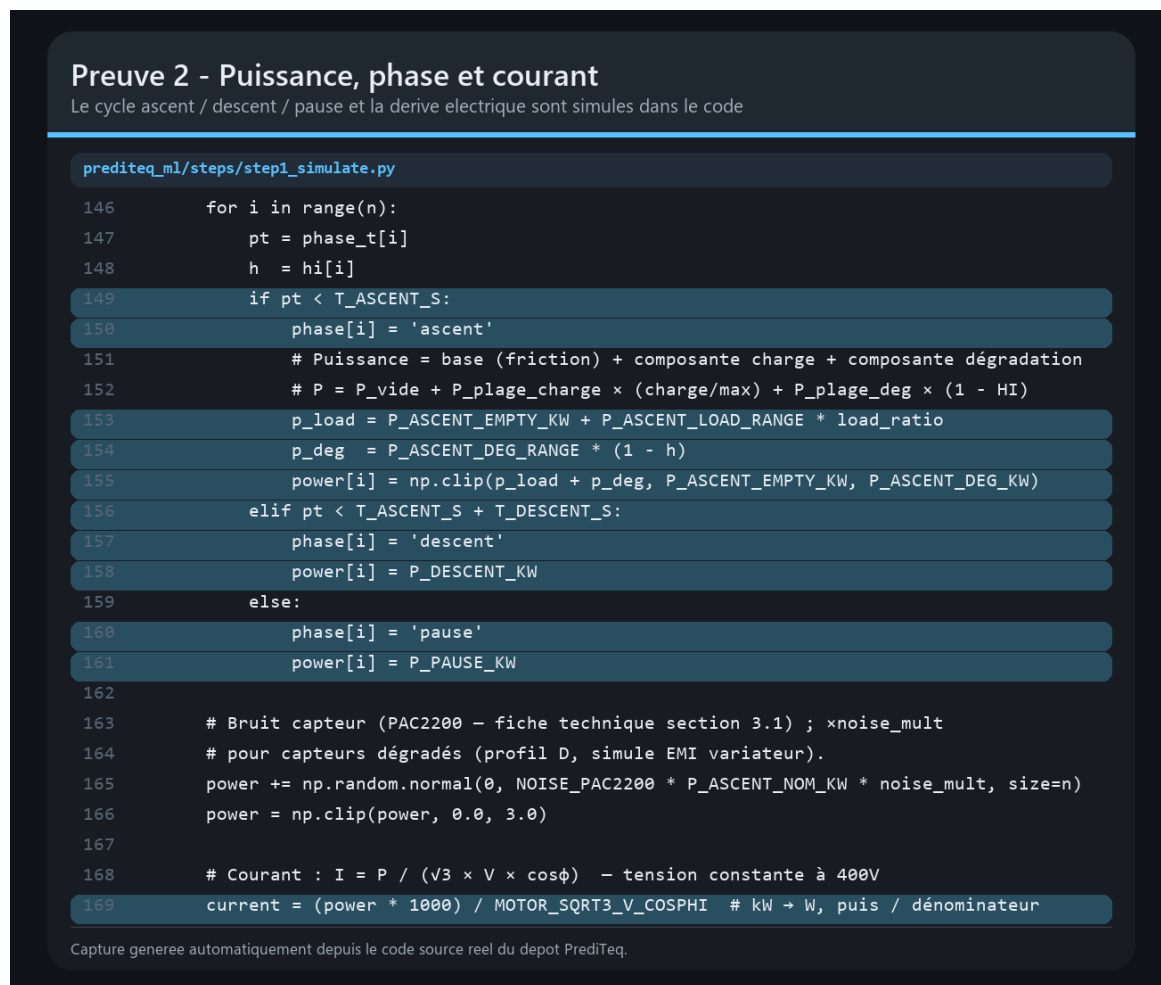


Figure 7.2 – Capture du code source reel montrant les phases ascent/descent/pause ainsi que le calcul de la puissance et du courant

Lecture simple :

- la montee, la descente et la pause ne consomment pas pareil ;
- la montee depend en plus de la charge et de la degradation ;
- le courant n'est pas invente au hasard : il est derive de la puissance electrique.

7.8.3 Preuve 3 : les donnees brutes sont transformees en variables plus parlantes pour le modele

Source : prediteq_ml/steps/step2_preprocess.py

```

feat_df = pd.DataFrame({
    'rms_mms': f1, 'drms_dt': f2, 'rms_variability': f3,
    'p_mean_kw': f4, 'p_rms_kw': f5, 'dp_dt': f6,
    'e_cycle_kwh': f7, 'duration_ratio': f8,
    't_mean_c': f9, 'dt_dt': f10, 'hr_std': f11, 'corr_t_p': f12,
})

```

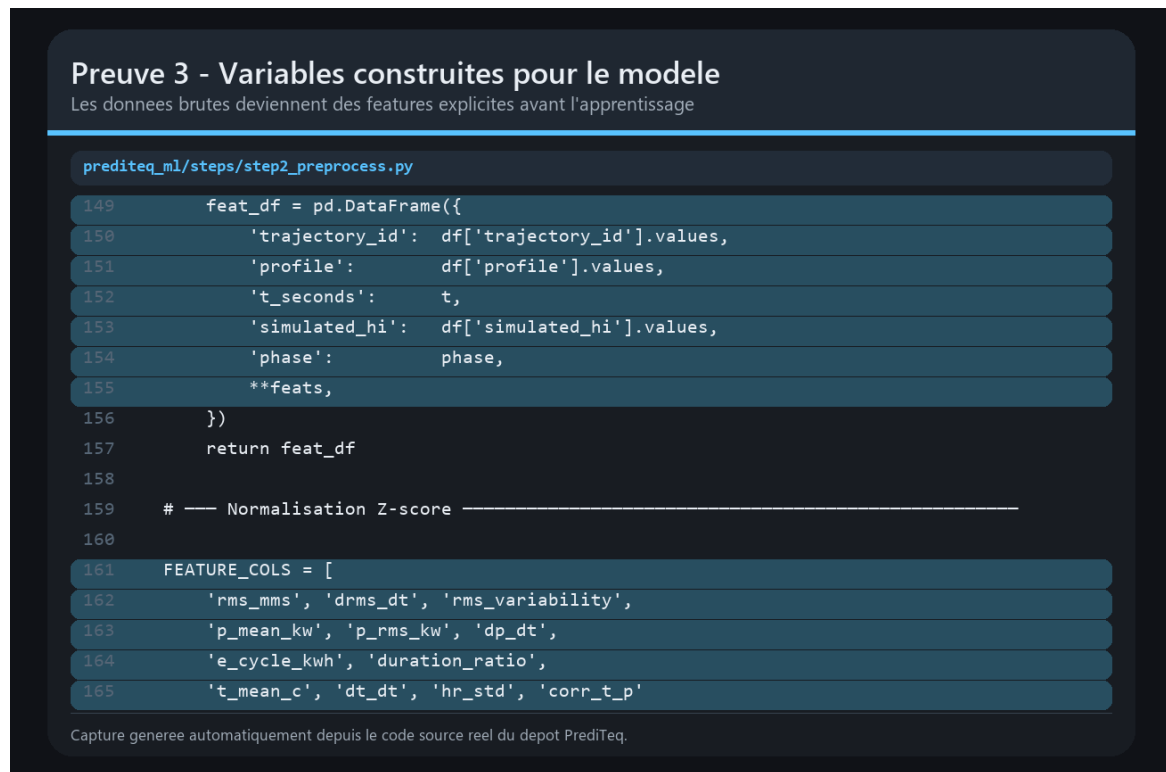


Figure 7.3 – Capture du code source reel montrant la construction du tableau de features et la liste des variables fournies au modele

Lecture simple :

- le modele ne voit pas seulement “une vibration” ou “une temperature”;
- il voit aussi des tendances, des variations, une energie par cycle et une correlation temperature/puissance;
- cela explique pourquoi PrediTeq peut raisonner plus finement qu’un simple seuil brut.

7.8.4 Preuve 4 : l'anomalie finale est bien un score hybride et pas une seule regle brute

Sources : `prediteq_ml/config.py` et `prediteq_ml/steps/step3_isolation_forest.py`

```
HYBRID_ALPHA = 0.2

alpha = HYBRID_ALPHA
scores_df['hybrid_score'] = alpha * sa_norm + (1 - alpha) * rz_norm
```

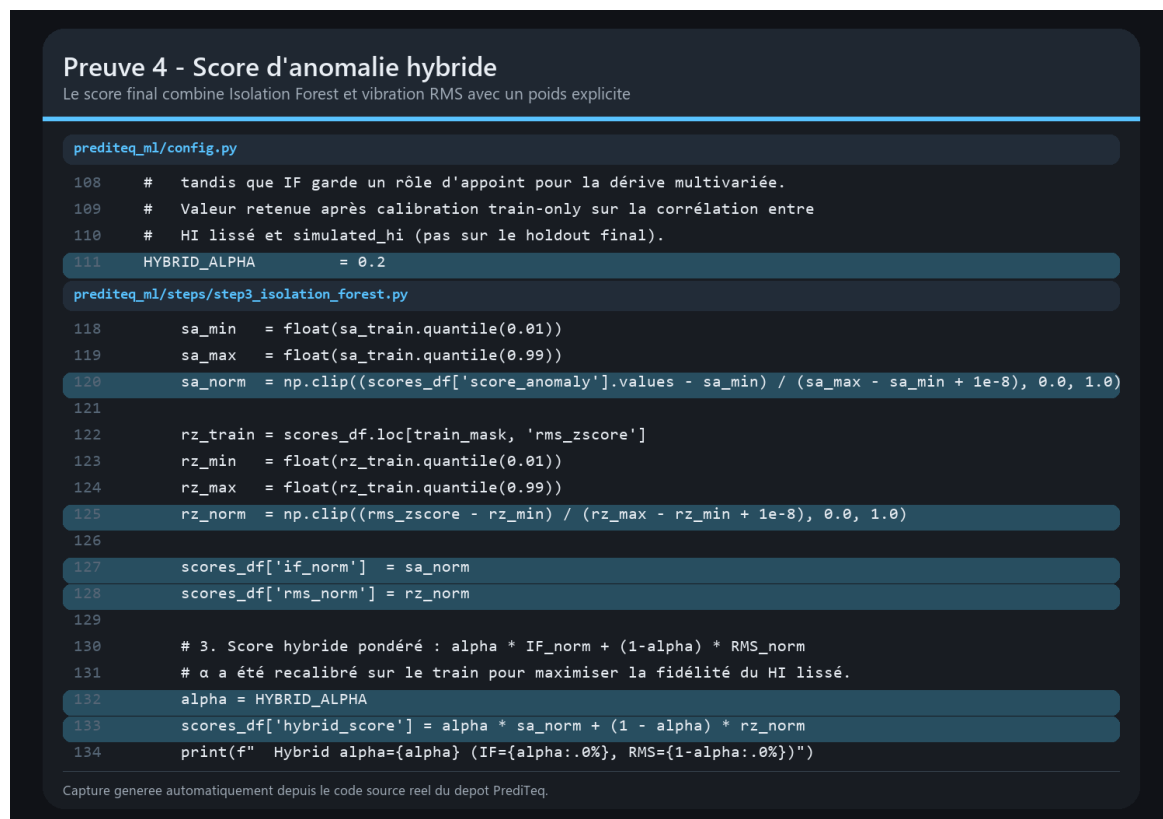


Figure 7.4 – Capture du code source réel montrant le poids `HYBRID_ALPHA` et la combinaison du score Isolation Forest avec la RMS vibratoire

Lecture simple :

- 0.2 signifie que l'on garde une part du score multicapteur Isolation Forest ;
- 0.8 revient à la vibration RMS, qui porte le signal principal dans notre simulation actuelle ;
- le système final est donc un compromis pondéré, pas une heuristique unique.

7.8.5 Preuve 5 : la cible RUL vient bien du HI observable, sans fuite de donnees

Source : prediteq_ml/steps/step5_rul_model.py

```
hi_obs = merged['hi_smooth'].values

t_fail_idx = find_threshold_crossing(
    hi_obs, HI_CRITICAL, RUL_CROSSING_PERSISTENCE
)

hi_now = hi_obs[i]
hi_mean = np.mean(window)
hi_std = np.std(window)
hi_min = np.min(window)
hi_slope = np.polyfit(np.arange(len(window)), window, 1)[0]
```

Preuve 5 - Cible RUL construite depuis hi_smooth

La prediction de duree de vie utilise le HI observable et des resumes de tendance

```
prediteq_ml/steps/step5_rul_model.py

118     hi_obs = merged['hi_smooth'].values
119
120     # Franchissement confirmé : N points consécutifs sous le seuil critique
121     t_fail_idx = find_threshold_crossing(
122         hi_obs, HI_CRITICAL, RUL_CROSSING_PERSISTENCE
123     )
124     if t_fail_idx is None:
125         n_skipped_no_cross += 1
126         continue
127
128     t_fail_min = merged.loc[t_fail_idx, 't_seconds'] / 60.0
129
130     # Générer des échantillons RUL du début (après la fenêtre lookback)
131     # jusqu'à l'instant de défaillance.
132     for i in range(RUL_LOOKBACK_MIN, t_fail_idx):
133         window = hi_obs[max(0, i - RUL_LOOKBACK_MIN):i]
134         if len(window) < RUL_LOOKBACK_MIN:
135             continue
136
137         t_now_min = merged.loc[i, 't_seconds'] / 60.0
138         rul_min = t_fail_min - t_now_min
139         if rul_min <= 0:
140             n_skipped_early += 1
141             continue
142
143         sensor_feats = merged.loc[i, NORM_COLS].values.astype(float)
144         hi_now = hi_obs[i]
145         hi_mean = np.mean(window)
146         hi_std = np.std(window)
147         hi_min = np.min(window)
148         hi_slope = np.polyfit(np.arange(len(window)), window, 1)[0]
149
150         row = np.concatenate([sensor_feats, [hi_now, hi_mean, hi_std, hi_min, hi_slope]])
```

Capture generee automatiquement depuis le code source reel du depot PrediTeq.

Figure 7.5 – Capture du code source reel montrant que la cible RUL est construite a partir de `hi_smooth` et de resumes de tendance observables

Lecture simple :

- le RUL apprend a partir de `hi_smooth`, donc d'un signal que le runtime connait vraiment ;
- la persistance a 3 points evite qu'un seul bruit de mesure soit pris pour une fin de vie ;
- les resumes `hi_now`, `hi_mean`, `hi_std`, `hi_min` et `hi_slope` donnent au modele une lecture de la tendance, pas seulement de l'instant courant.

7.8.6 Preuve 6 : la validation RUL garde des trajectoires entieres hors train

Source : `prediteq_ml/steps/step5_rul_model.py`

```
gkf = GroupKFold(n_splits=n_splits)
for fold_idx, (tr_idx, va_idx) in enumerate(gkf.split(X, y, groups), 1):
    ...
```

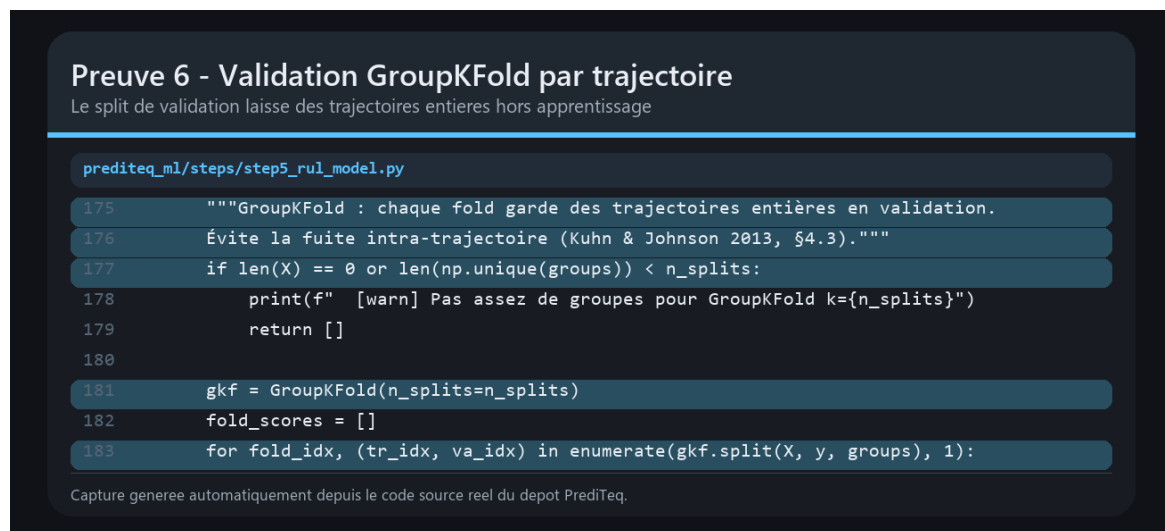


Figure 7.6 – Capture du code source reel montrant la validation GroupKFold qui garde des trajectoires entieres hors apprentissage

Lecture simple :

- une trajectoire complete reste d'un seul cote du test ;
- on evite ainsi un faux bon score cree par des morceaux quasi identiques d'une meme trajectoire ;
- c'est la preuve que les chiffres de validation ne viennent pas d'un melange train/test trop favorable.

7.9 Meme projet, deux niveaux de discours

Table 7.10 – Comment presenter le meme sous-systeme a un public non technique ou technique

Bloc	Version simple pour jury mixte	Version technique pour jury specialise
Simulation	Nous avons construit des vies de machine credibles parce que nous ne possedons pas encore des annees de pannes captees sur site.	La simulation part de la plaque moteur, des cas de charge, du cycle 44 s, du bruit capteur et d'une chaine courant vers echauffement vers degradation, avec 200 trajectoires et 4 profils.
Anomalies et HI	Le systeme compare la machine a un comportement sain puis transforme l'ecart en un indice de sante facile a lire.	L'Isolation Forest delimite le sain, le score hybride ajoute la RMS vibratoire dominante, puis le HI applique un lissage 120 s et des seuils 0,8 / 0,6 / 0,3.
RUL	Le systeme estime en combien de jours la machine risque de devenir vraiment preoccupante.	Le Random Forest utilise 17 variables, dont 5 resumes HI, apprend un franchissement persistant de <code>hi_smooth</code> sans fuite de donnees, puis publie un intervalle et un niveau de confiance.
Backend	Le serveur central recoit les mesures, calcule l'etat machine, enregistre l'historique et declenche des actions.	FastAPI charge les artefacts <code>prediteq_ml/models/</code> , instancie un moteur par machine, gere MQTT ou HTTP, maintient les buffers, publie diagnostics, planner, rapports, scheduler et persistance Supabase.
Frontend	L'application ne montre pas un score abstrait ; elle montre quoi surveiller, quoi planifier et quand intervenir.	React consomme les routes backend, reutilise une decision snapshot partagee entre dashboard, diagnostics, planner, maintenance, alertes et couts, avec des droits admin ou user.
IoT et MQTT	Nous savons deja faire parler une source externe avec PrediTeq ; la prochaine etape est de remplacer la source simulee par le poste industriel reel.	Le pont <code>mqtt_bridge_sender.py</code> sait lire JSON, CSV ou une source custom, puis publier vers MQTT ou <code>/ingest/live</code> . Le test a deux PCs valide deja le schema logiciel de bout en bout.

7.10 Points d'honneur, limites et ordre conseille pour la presentation

7.10.1 Ce qu'il faut dire clairement au jury

- Les donnees d'entrainement du pipeline principal restent **synthetiques**, meme si elles sont ancrees dans une machine reelle et dans une logique physique defendable.
- Le **simulateur backend** est un outil de demonstration : il fait passer la telemetrie dans le moteur runtime, mais il peut aussi maintenir l'affichage dans des bandes pedagogiques de HI, de zone et de RUL.
- Le **backend n'entraine pas** les modeles. L'entrainement vit dans `prediteq_ml/`; l'API charge seulement les artefacts exportes.
- Les **seuils runtime** peuvent differer des seuils offline parce que l'application lit aussi la table Supabase **seuils**.
- Un **RUL numerique n'apparait pas instantanement** : il faut environ 60 minutes d'historique HI avant que le runtime publie une prediction exploitable.
- La page `/experiment` valide un **banc local USB** ; elle n'est pas la preuve finale de la chaine cloud.
- Le test **MQTT a deux postes** prouve la faisabilite logicielle du flux externe, mais pas encore l'industrialisation finale LabVIEW ou PLC sur plusieurs mois.

7.10.2 Ordre conseille pour presenter tout le travail

1. Commencer par le besoin industriel : eviter d'intervenir trop tard ou trop tot sur un stockeur vertical rotatif.
2. Montrer la base physique : charge, puissance, courant, echauffement, degradation.
3. Expliquer pourquoi la simulation etait necessaire et pourquoi elle reste raisonnable.
4. Derouler ensuite les 9 scripts de la chaine ML, du Step 1 au Step 7 en passant par 6B et 6C.
5. Donner les chiffres qui prouvent que le pipeline n'est pas seulement "joli" : F1, RUL holdout, GroupKFold, NASA, calibration, lead time.
6. Basculer vers le runtime : artefacts exportes, backend FastAPI, moteur par machine, scheduler, alertes et seuils live.
7. Ouvrir l'application web : dashboard, diagnostics, planner, maintenance, couts, alertes, administration.
8. Montrer la difference entre `/simulateur`, `/experiment` et la vraie ingestion live MQTT ou HTTP.

9. Expliquer la preuve a deux PCs puis le plan le plus simple avec un PC relais cote client et la machine ARO-01.
10. Finir par les limites honnetes et le prochain jalon logique : historique terrain annote, recalibrage prudent, puis reentrainement offline.

En une phrase de cloture, PrediTeq peut etre presente ainsi : *nous avons construit une chaine complete, mesurable et montrable, qui va de la physique du moteur a l'action de maintenance dans le navigateur, tout en laissant une place claire a l'etape suivante indispensable : la validation terrain sur donnees reelles.*